IndicDoc Verify

Deep-Learning Layout Analysis for Pre-Print Checking of Indian Question Papers and Their Translations

EXPT.-11 · SDG-BASED DEEP LEARNING MINI PROJECT · COMPLETE SUBMISSION

Team

Name	Roll No.
Riddhi Patil	39
Mariam Badure	40
Prashant Jha	41
Aditya Soni	42

Project guide Dr. Joanne Gomes

SDGs SDG 4 Quality Education · SDG 9 Industry, Innovation and Infrastructure · SDG 10 Reduced Inequalities · SDG 16 Peace, Justice and Strong Institutions

Repository `github.com/va-ada/IndicDoc-AI`

Date 27 October 2026

This volume contains the dataset, the code and implementation, the repository, both phase presentations and the research paper (70 pages).

Contents

Part	Contents	Pages
1	**Dataset** IndicDLP subset: splits, languages, domains, the 42 labels, data files, sample pages; the verification benchmark	3–8
2	**Project Code and Implementation** Architecture, technology stack, repository structure, running the system, REST API, tests, the working product, source code of the core modules	9–43
3	**GitHub Repository** Files, branches, languages and commits of the repository; README	45–46
4	**Phase-1 Presentation: Literature Survey** 19 slides, two per page	47–56
5	**Phase-2 Presentation: Implementation** 20 slides, two per page	57–66
6	**Research Paper** IndicDoc Verify: layout-aware pre-print checking of Indian question papers and their translations (IEEE format)	67–70

Printing. Print the whole file double-sided on A4 (flip on long edge). Every part begins on the front of a new sheet, so the printed stack separates into the parts; pages marked “intentionally left blank” keep it that way. The presentations are printed two slides per page.

PART 1

Dataset

The IndicDLP subset used to train and test the layout detectors, the data files delivered with the project, and the benchmark used to measure IndicDoc Verify.

1.1 Source: IndicDLP

IndicDLP is a human-labelled dataset for document layout parsing in Indian languages, released by IIT Madras and IIIT Hyderabad at ICDAR 2025. Each page is labelled with rectangular regions of 42 kinds, including the question-paper labels this project depends on: first-, second- and third-level questions, options and answers.

Property	Details
Paper	O. Nath, S. Kukkala, M. Khapra, R. K. Sarvadevabhatla, "IndicDLP: A foundational dataset for multilingual and multi-domain document layout parsing," ICDAR 2025, LNCS, pp. 23–39
Download	`huggingface.co/datasets/ai4bharat/indicdlp` (gated; also listed on AIKosh, the IndiaAI dataset platform)
Size	119,806 pages; Parquet shards (train 73.3 GB, validation 8.9 GB, test 8.7 GB)
Annotation	PNG page image; boxes `[x, y, width, height]` in pixels; class ids 0–41
Languages (12)	Assamese, Bengali, English, Gujarati, Hindi, Kannada, Malayalam, Marathi, Odia, Punjabi, Tamil, Telugu
Domains (12)	acts & rules, brochures, forms, magazines, manuals, newspapers, notices, novels, question papers, research papers, syllabi, textbooks
Licence	MIT (Hugging Face dataset page)

1.2 Subset used in this project

A language-balanced subset is drawn from the official splits by `scripts/prepare_indicdlp.py`, so no test page was seen in training, by us or by the dataset authors. It has 3,317 pages and 49,995 labelled regions. Boxes under 1 px after clipping to the page were dropped (165 of 33,832 training boxes, 0.5%); 15 training pages without labelled elements are kept as background examples.

Split	Pages	Boxes	Boxes per page
Train	2,293	33,667	14.7
Validation	342	5,815	17.0
Test	682	10,513	15.4
Total	**3,317**	**49,995**	**15.1**

Pages by language

Language	Train	Val	Test	Total
Assamese	129	30	60	219
Bengali	200	30	60	290
English	188	30	60	278
Gujarati	200	30	60	290
Hindi	200	21	22	243
Kannada	200	21	60	281
Malayalam	200	30	60	290
Marathi	200	30	60	290
Odia	200	30	60	290
Punjabi	188	30	60	278
Tamil	200	30	60	290
Telugu	188	30	60	278
Total	**2293**	**342**	**682**	**3317**

Pages by domain

Domain	Train	Val	Test	Total
acts & rules	230	30	68	328
brochures	200	24	66	290
forms	125	28	46	199
magazines	244	39	79	362
manuals	242	26	58	326
newspapers	118	26	36	180
notices	112	17	36	165
novels	248	39	69	356
question papers	142	13	40	195
research papers	229	31	68	328
syllabi	161	24	54	239
textbooks	242	45	62	349
Total	**2293**	**342**	**682**	**3317**

1.3 Data files delivered

The folder `dataset/` of the repository describes every page and box used, so the subset can be rebuilt exactly; the 2.3 GB of images are rebuilt from Hugging Face by `scripts/prepare_indicdlp.py` and are not stored in git.

File	One row per	Columns
`pages.csv`	page (3,317)	split, file_name, language, domain, width, height, num_boxes
`boxes.csv`	labelled region (49,995)	split, file_name, class_id, class_name, x, y, width, height (pixels, top-left origin)
`class_counts.csv`	label (42)	class_id, class_name, train, val, test, total
`language_split.csv`	language (12)	language, train, val, test, total
`samples/`	language (12)	one real test page with its ground-truth boxes drawn

First rows of pages.csv

split	file_name	language	domain	width	height	num_boxes
train	nt_mr_000617.png	marathi	notices	612	1008	1
train	rp_ta_000350_1.png	tamil	research papers	510	680	9
train	qp_hi_000016_0.png	hindi	question papers	1700	2200	10
train	qp_pa_000374_1.png	punjabi	question papers	595	842	25

First rows of boxes.csv

split	file_name	class_id	class_name	x	y	width	height
train	nt_mr_000617.png	36	table	48.0	36.0	489.0	943.0
train	rp_ta_000350_1.png	26	section-title	33.0	91.2	183.8	45.2
train	rp_ta_000350_1.png	21	paragraph	32.9	55.9	221.5	35.1

1.4 The 42 layout labels

Labelled regions per label and split (class-id order of the authors' `indic.yaml`). The question-paper labels used by IndicDoc Verify are in bold.

id	Label	Train	Val	Test	Total
0	advertisement	354	30	114	498
1	**answer**	72	17	37	126
2	author	409	113	90	612
3	chapter-title	173	23	42	238
4	contact-info	272	30	75	377
5	dateline	703	125	222	1,050
6	figure	1,914	321	517	2,752
7	figure-caption	707	64	141	912
8	**first-level-question**	911	64	252	1,227
9	flag	68	16	27	111
10	folio	1,848	252	611	2,711
11	footer	1,360	188	390	1,938
12	footnote	327	45	85	457
13	formula	347	48	98	493
14	header	1,436	230	457	2,123
15	headline	979	211	326	1,516
16	index	6	0	2	8
17	jumpline	91	32	42	165
18	**options**	871	87	251	1,209
19	ordered-list	956	159	281	1,396
20	page-number	1,840	277	562	2,679
21	paragraph	10,510	2,136	3,435	16,081
22	placeholder-text	1,632	336	623	2,591
23	quote	141	16	4	161
24	reference	109	32	37	178
25	**second-level-question**	251	32	73	356
26	section-title	1,771	346	625	2,742
27	sidebar	940	109	293	1,342
28	sub-headline	333	87	136	556
29	sub-ordered-list	424	65	122	611
30	sub-section-title	413	77	96	586
31	subsub-ordered-list	102	13	30	145
32	subsub-section-title	59	9	9	77
33	sub-unordered-list	13	1	2	16
34	subsub-headline	54	8	14	76
35	subsub-unordered-list	0	0	5	5
36	table	441	73	137	651
37	table-caption	180	34	61	275
38	table-of-contents	32	2	8	42
39	**third-level-question**	62	6	15	83
40	unordered-list	284	56	95	435
41	website-link	272	45	71	388

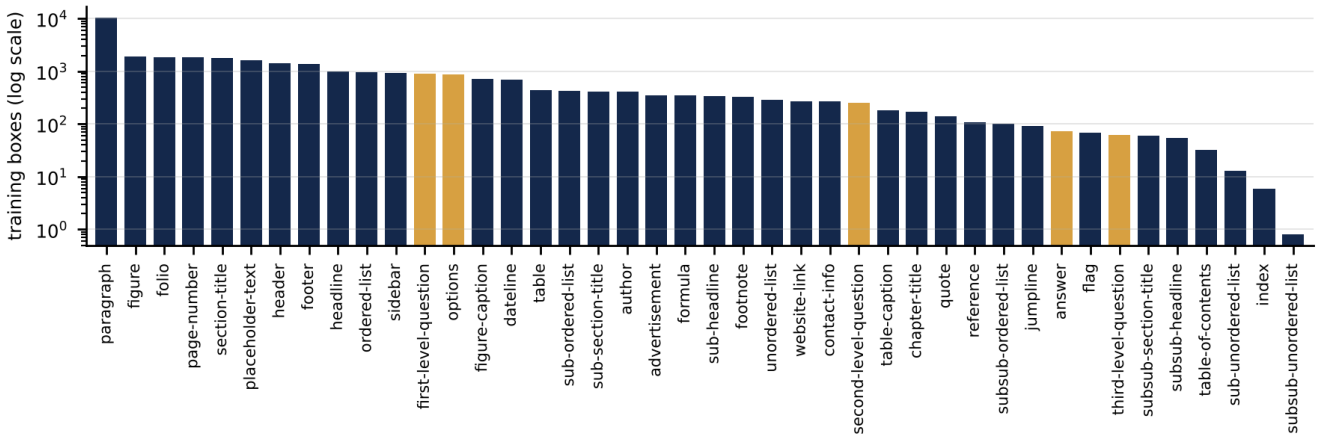
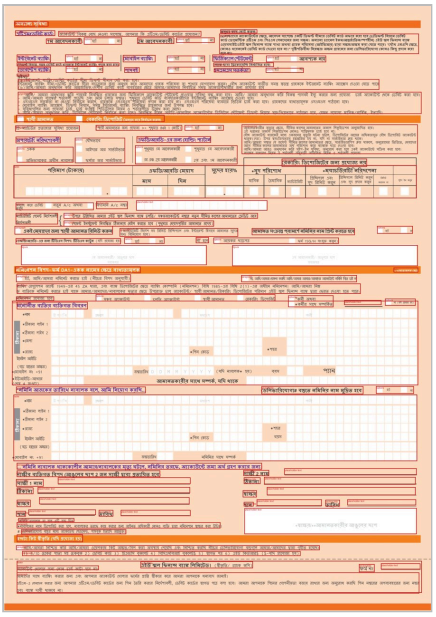


Figure 1.1. Training boxes per label, most frequent first (log scale); question-paper labels in gold. *paragraph* alone is a third of the training boxes; several labels have fewer than 20.

1.5 Sample test pages

One real IndicDLP test page per language with its ground-truth regions drawn (one colour per label).

Assamese, newspapers



Bengali, forms

English, newspapers

Gujarati, newspapers

Hindi, newspapers

Kannada, newspapers

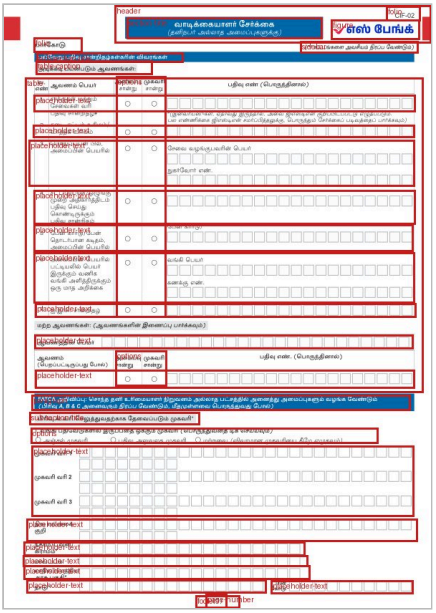
1.5 Sample test pages (continued)

Malayalam, newspapers

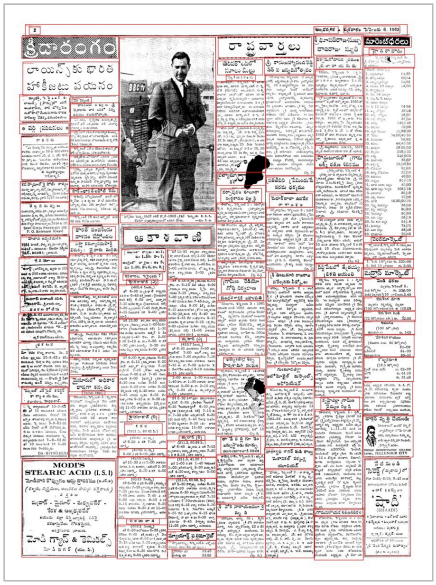
Marathi, newspapers

Odia, newspapers

Punjabi, brochures



Tamil, forms



Telugu, newspapers

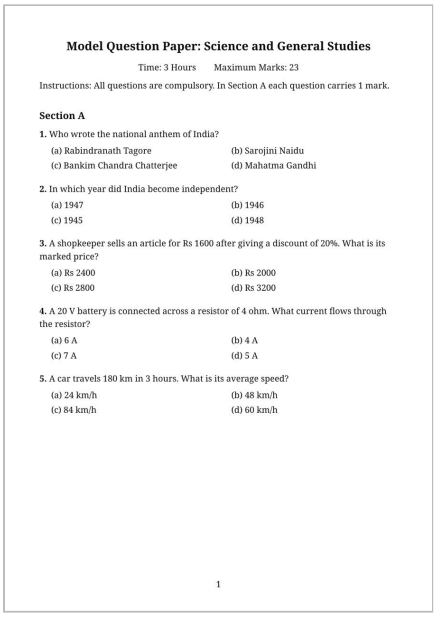
1.6 Verification benchmark

No public dataset has question papers with known errors in their translations, so the benchmark was built for this project (`scripts/verify_bench/`). Papers of 18 questions are composed from parallel English, Hindi and Tamil questions; each pair is an English source and a Hindi or Tamil version. One error of a kind that has happened in real exams is injected into the version, or none (a clean pair). The pages are rendered at 200 dpi with Noto fonts; half are degraded like scans (blur, noise, JPEG at quality 55), and every page is read with real OCR (Tesseract).

Injected error	Test pairs
Number changed in stem	12
Number changed in an option	12
Option missing	12
Question missing	12
Question printed twice	12
Marks differ	12
Wrong maximum marks	12
Question numbering error	12
Repeated from a past paper	12
No error (clean pair)	24
Total	**132**

Set	Pairs	Use
Development (seed 7)	48	building the parser and the checks
Test (seed 42)	132	run once, results reported

An error counts as found only when a finding of the matching kind points at the right question. On the test set 98.1% of the 108 injected errors were found, with 4.46 false alarms per clean pair (Part 6).



Model Question Paper: Science and General Studies

Time: 3 Hours Maximum Marks: 23

Instructions: All questions are compulsory. In Section A each question carries 1 mark.

Section A

1. Who wrote the national anthem of India?
(a) Rabindranath Tagore (b) Sarojini Naidu
(c) Bankim Chandra Chatterjee (d) Mahatma Gandhi

2. In which year did India become independent?
(a) 1947 (b) 1946
(c) 1945 (d) 1948

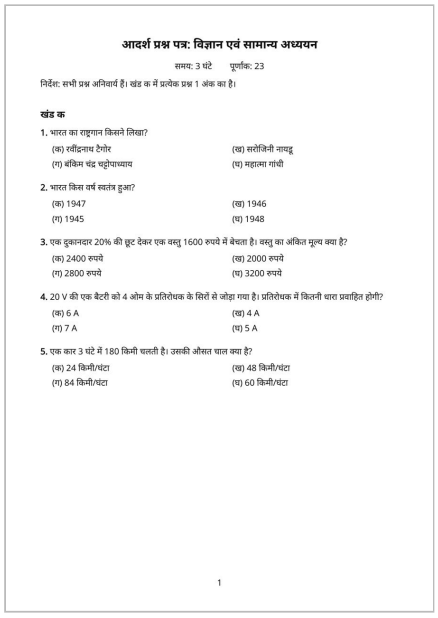
3. A shopkeeper sells an article for Rs 1600 after giving a discount of 20%. What is its marked price?
(a) Rs 2400 (b) Rs 2000
(c) Rs 2800 (d) Rs 3200

4. A 20 V battery is connected across a resistor of 4 ohm. What current flows through the resistor?
(a) 6 A (b) 4 A
(c) 7 A (d) 5 A

5. A car travels 180 km in 3 hours. What is its average speed?
(a) 24 km/h (b) 48 km/h
(c) 84 km/h (d) 60 km/h

1

English source



आदर्श प्रश्न पत्र: विज्ञान एवं सामान्य अध्ययन

समय: 3 घंटे पूर्णांक: 23

निर्देश: सभी प्रश्न अनिवार्य हैं। खंड क में प्रत्येक प्रश्न 1 अंक का है।

खंड क

1. भारत का राष्ट्रगान किसने लिखा?
(क) रवींद्रनाथ टैगोर (ख) सरोजिनी नायडू
(ग) बंकिम चंद्र चट्टोपाध्याय (घ) महात्मा गांधी

2. भारत किस वर्ष स्वतंत्र हुआ?
(क) 1947 (ख) 1946
(ग) 1945 (घ) 1948

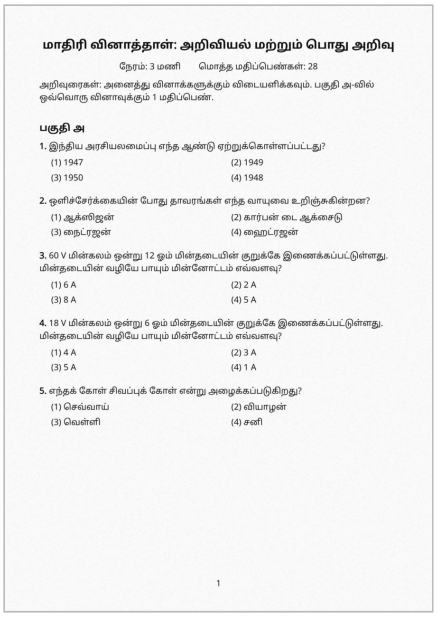
3. एक दुकानदार 20% की छूट देकर एक वस्तु 1600 रुपये में बेचता है। वस्तु का अंकित मूल्य क्या है?
(क) 2400 रुपये (ख) 2000 रुपये
(ग) 2800 रुपये (घ) 3200 रुपये

4. 20 V की एक बैटरी को 4 ओम के प्रतिरोधक के सिरों से जोड़ा गया है। प्रतिरोधक में कितनी धारा प्रवाहित होगी?
(क) 6 A (ख) 4 A
(ग) 7 A (घ) 5 A

5. एक कार 3 घंटे में 180 किमी चलती है। उसकी औसत चाल क्या है?
(क) 24 किमी/घंटा (ख) 48 किमी/घंटा
(ग) 84 किमी/घंटा (घ) 60 किमी/घंटा

1

Hindi version



மாதிரி வினாத்தாள்: அறிவியல் மற்றும் பொது அறிவு

நேரம்: 3 மணி மொத்த மதிப்பெண்கள்: 28

அறிவுரைகள்: அனைத்து வினாக்களுக்கும் விடையளிக்கவும். பகுதி அ-வில் ஒவ்வொரு வினாவுக்கும் 1 மதிப்பெண்.

பகுதி அ

1. இந்திய அரசியலமைப்பு எந்த ஆண்டு ஏற்றுக்கொள்ளப்பட்டது?
(1) 1947 (2) 1949
(3) 1950 (4) 1948

2. ஒளிச்சேர்க்கையின் போது தாவரங்கள் எந்த வாயுவை உறிஞ்சுகின்றன?
(1) ஆக்ஸிஜன் (2) கார்பன் டை ஆக்சைடு
(3) நைட்ரஜன் (4) ஹைட்ரஜன்

3. 60 V மின்கலம் ஒன்று 12 ஓம் மின்தடையின் குறுக்கே இணைக்கப்பட்டுள்ளது. மின்தடையின் வழியே பாயும் மின்னோட்டம் எவ்வளவு?
(1) 6 A (2) 2 A
(3) 8 A (4) 5 A

4. 18 V மின்கலம் ஒன்று 6 ஓம் மின்தடையின் குறுக்கே இணைக்கப்பட்டுள்ளது. மின்தடையின் வழியே பாயும் மின்னோட்டம் எவ்வளவு?
(1) 4 A (2) 3 A
(3) 5 A (4) 1 A

5. எந்தக் கோள் சிவப்புக் கோள் என்று அழைக்கப்படுகிறது?
(1) செவ்வாய் (2) வியாழன்
(3) வெள்ளி (4) சனி

1

Tamil version, scan-degraded

PART 2

Project Code and Implementation

How IndicDoc Verify is built: the architecture, the code base, how to run it, the API, the tests, the working product, and the source code of its core modules.

2.1 System architecture

IndicDoc Verify checks an Indian question paper, and its translations, before it is printed. A deep-learning layout detector (DocLayout-YOLO, fine-tuned on IndicDLP by its authors) finds the questions, sub-questions, options and answers on each page. Tesseract reads the text with the page script's own model, and a second *number pass* with the English model must agree on every digit. The questions are built from the layout and checked for structure, parity with the source version, repeats from earlier papers and format. Everything runs offline in one Docker image.

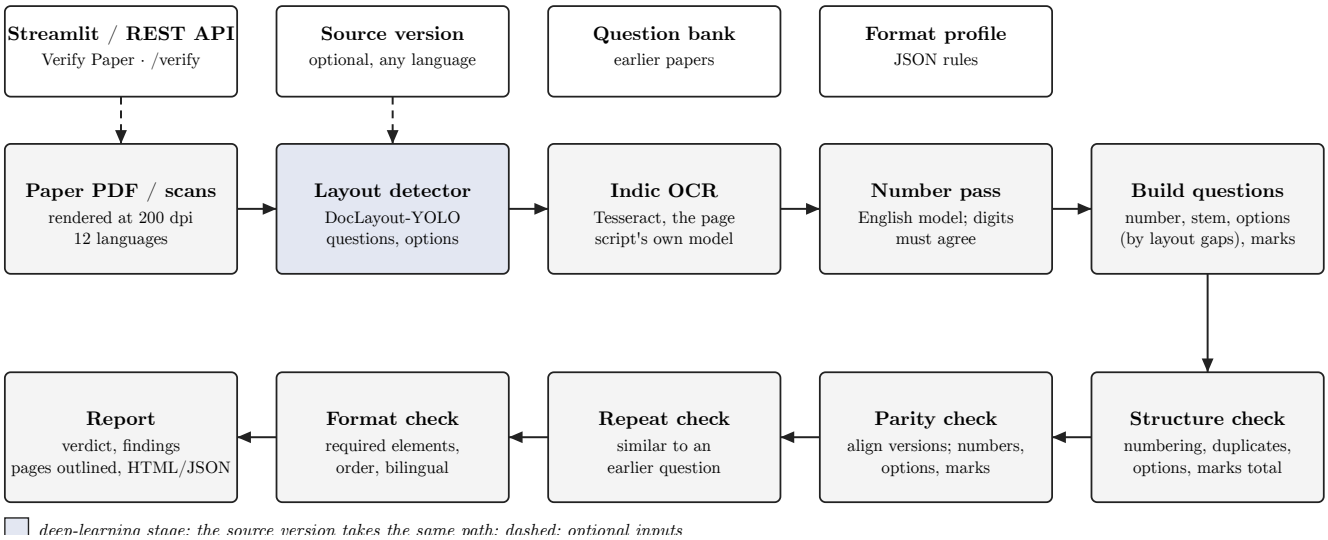


Figure 2.1. Processing pipeline of IndicDoc Verify.

2.2 Technology stack

Layer	Technology
Layout detection	DocLayout-YOLO (IndicDLP authors' weights); YOLOv8s and Faster R-CNN fine-tuned here (Ultralytics, PyTorch, torchvision)
OCR	Tesseract 5 with `tessdata_best` models for 12 languages (pytesseract)
Verification	Python 3.11: question parser, checks, Needleman–Wunsch alignment of versions, JSON format profiles, HTML/JSON report
Documents	PyMuPDF (PDF pages at 200 dpi), OpenCV and Pillow (pre-processing)
Interfaces	Streamlit web app (port 8501); FastAPI REST API (port 8000)
Evaluation	pycocotools (COCO mAP), the verification benchmark (Playwright rendering, tesseract.js)
Deployment, tests	Docker and docker-compose; pytest; node test runner for the React prototype

2.3 Repository structure

Path	Contents
app/streamlit_app.py	Streamlit web app: Verify Paper, Analyze, Dashboard, Model Insights, Evaluation, About
backend/main.py	FastAPI REST API
ml/config.py	paths, the 42 IndicDLP labels, defaults, device selection
ml/datasets/	COCO dataset loader, synthetic sample generator
ml/preprocessing/	transforms, deskew/CLAHE, PDF page rendering
ml/models/	Faster R-CNN and YOLOv8 builders, COCO-to-YOLO converter
ml/training/	train_baseline.py (Faster R-CNN), train_yolo.py (YOLOv8s)
ml/evaluation/	COCO mAP, precision/recall/F1, confusion matrix, per-language results
ml/inference/	load a checkpoint and predict regions
ml/ocr/	OCR of detected regions, script detection, number pass, reading order
ml/verify/	IndicDoc Verify: parsing, checks, parity, repeats, profiles, report
scripts/	data preparation, evaluation, benchmark, and the builders of every document
scripts/verify_bench/	benchmark generator, renderer, degradation, OCR
tests/	pytest suite
dataset/	the subset as CSV files, sample pages
outputs/	trained models, metrics, plots
docs/, paper/	documentation, report, presentations, research paper
Dockerfile, docker-compose.yml	one image with the app, the API, the models and Tesseract

The repository has 69 Python files with 12,363 lines in all. The core modules printed in Section 2.8 are:

File	Lines	Role
ml/verify/__init__.py	76	Entry point: verify() parses the paper, runs every check and returns the report
ml/verify/text.py	226	Text helpers: Indian digits, numerals, question numbers, marks, option splitting, similarity
ml/verify/paper.py	255	Builds the questions (number, stem, options, marks) from the layout regions and OCR words
ml/verify/checks.py	277	Structure checks, alignment of two language versions (translation parity), repeats
ml/verify/profiles.py	107	Format profiles: any document type described as JSON rules
ml/verify/profiles/question_paper.json	42	The question-paper profile
ml/verify/report.py	87	The report: findings outlined on the pages, HTML output
ml/ocr/__init__.py	415	Tesseract OCR of the detected regions, script detection, the number pass, reading order
ml/inference/predict.py	152	Loads a layout detector (DocLayout-YOLO, YOLOv8, Faster R-CNN) and predicts regions
ml/models/proposed_yolov8.py	68	YOLOv8s model builder and COCO-to-YOLO conversion
ml/training/train_yolo.py	88	Fine-tuning YOLOv8s on the IndicDLP subset

File	Lines	Role
ml/models/baseline_frcnn.py	35	Faster R-CNN (ResNet-50 FPN) baseline builder
backend/main.py	274	FastAPI REST API: /analyze (layout + OCR) and /verify (pre-print check)

2.4 Running the system

With Docker (recommended)

```
docker compose up --build -d
# web app  http://localhost:8501
# REST API http://localhost:8000   (interactive docs at /docs)
```

Local setup

```
python3.11 -m venv venv && source venv/bin/activate
pip install -r requirements.txt
python scripts/download_models.py      # DocLayout-YOLO + Tesseract language models (~300 MB,
once)
brew install tesseract                 # or: apt install tesseract-ocr

streamlit run app/streamlit_app.py     # web app
uvicorn backend.main:app --port 8000   # REST API
python -m pytest tests/                # test suite
```

Reproducing the results

```
python scripts/prepare_indicdlp.py                # IndicDLP subset -> data/raw/indicdlp
python -m ml.training.train_yolo                  # fine-tune YOLOv8s
python -m ml.training.train_baseline              # fine-tune Faster R-CNN
python scripts/evaluate.py                        # layout mAP on the test pages
python scripts/evaluate_verify.py --layout oracle  # verification benchmark
```

2.5 REST API

Method and path	Input	Output
GET /	—	service status
POST /analyze	page image or PDF; ocr, ocr_lang	regions (label, box, confidence) and their text
GET /metrics	—	evaluation metrics of the layout models
GET /verify/profiles	—	available format profiles
POST /verify	paper (PDF/images); optional source version, question bank, lang, source_lang, bilingual, profile	verdict and findings (question, page, box) as JSON, or the HTML report with the pages outlined

```
curl -F file=@paper_hi.pdf -F source=@paper_en.pdf \
     "http://localhost:8000/verify?lang=hin&source_lang=eng&output=html" > report.html
```

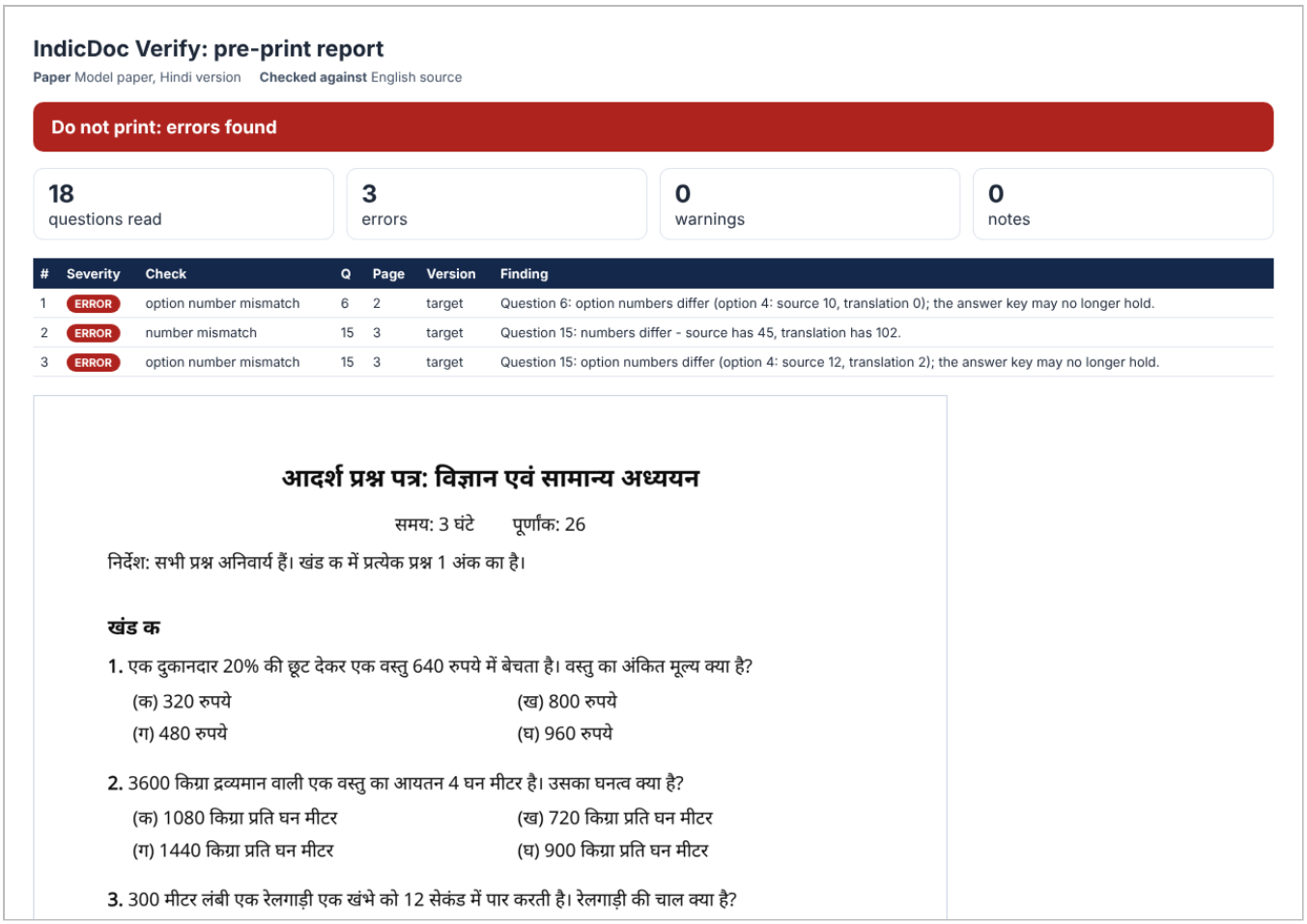
2.6 Testing

The test suite was run on 27 October 2026: **164 passed, 6 skipped, 0 failed**, plus 2 passing tests of the React prototype. The skipped tests need the Tesseract binary and the trained checkpoints, which were not in-

stalled on that machine; they run in the Docker image.

Test file	Tests	What is tested
`tests/test_accessible.py`	3	accessible HTML/EPUB export
`tests/test_app.py`	12	Streamlit app: tabs, upload, results (AppTest)
`tests/test_backend.py`	26	REST API: /analyze, /metrics, /verify, upload limits
`tests/test_coco_eval.py`	12	COCO mAP, precision/recall, confusion matrix
`tests/test_ocr.py`	27	OCR of regions, script detection, the number pass, reading order
`tests/test_pipeline.py`	64	models, datasets, training and inference on synthetic pages
`tests/test_prepare_indicdlp.py`	8	IndicDLP conversion and cleaning
`tests/test_verify.py`	18	question parsing, structure checks, translation parity, repeats, profiles
Total	**170**	

2.7 The working product



IndicDoc Verify: pre-print report

Paper Model paper, Hindi version **Checked against** English source

Do not print: errors found

18	3	0	0
questions read	errors	warnings	notes

#	Severity	Check	Q	Page	Version	Finding
1	ERROR	option number mismatch	6	2	target	Question 6: option numbers differ (option 4: source 10, translation 0); the answer key may no longer hold.
2	ERROR	number mismatch	15	3	target	Question 15: numbers differ - source has 45, translation has 102.
3	ERROR	option number mismatch	15	3	target	Question 15: option numbers differ (option 4: source 12, translation 2); the answer key may no longer hold.

आदर्श प्रश्न पत्र: विज्ञान एवं सामान्य अध्ययन

समय: 3 घंटे पूर्णांक: 26

निर्देश: सभी प्रश्न अनिवार्य हैं। खंड क में प्रत्येक प्रश्न 1 अंक का है।

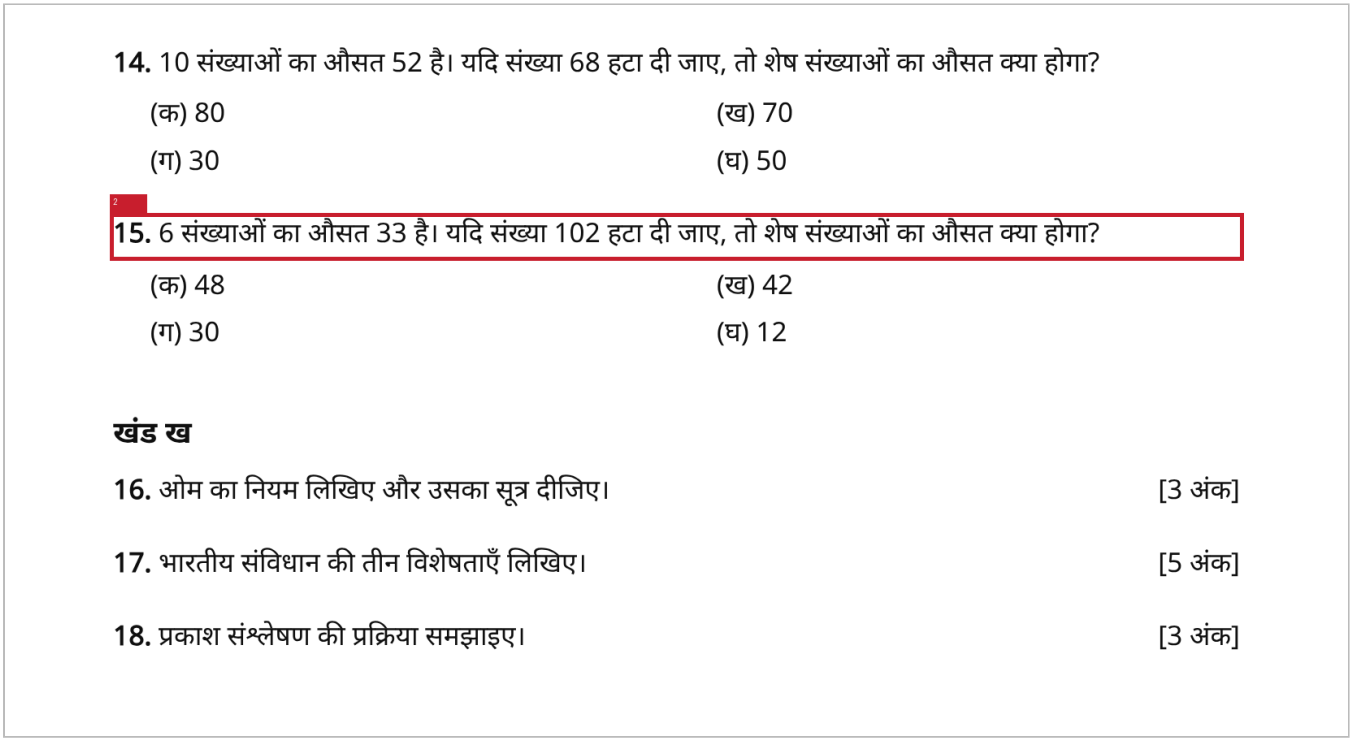
खंड क

1. एक दुकानदार 20% की छूट देकर एक वस्तु 640 रुपये में बेचता है। वस्तु का अंकित मूल्य क्या है?
   (क) 320 रुपये (ख) 800 रुपये
   (ग) 480 रुपये (घ) 960 रुपये

2. 3600 किग्रा द्रव्यमान वाली एक वस्तु का आयतन 4 घन मीटर है। उसका घनत्व क्या है?
   (क) 1080 किग्रा प्रति घन मीटर (ख) 720 किग्रा प्रति घन मीटर
   (ग) 1440 किग्रा प्रति घन मीटर (घ) 900 किग्रा प्रति घन मीटर

3. 300 मीटर लंबी एक रेलगाड़ी एक खंभे को 12 सेकंड में पार करती है। रेलगाड़ी की चाल क्या है?

Figure 2.2. Verification report: verdict, findings with question and page



14. 10 संख्याओं का औसत 52 है। यदि संख्या 68 हटा दी जाए, तो शेष संख्याओं का औसत क्या होगा?
(क) 80 (ख) 70
(ग) 30 (घ) 50

15. 6 संख्याओं का औसत 33 है। यदि संख्या 102 हटा दी जाए, तो शेष संख्याओं का औसत क्या होगा?
(क) 48 (ख) 42
(ग) 30 (घ) 12

खंड ख

16. ओम का नियम लिखिए और उसका सूत्र दीजिए। [3 अंक]

17. भारतीय संविधान की तीन विशेषताएँ लिखिए। [5 अंक]

18. प्रकाश संश्लेषण की प्रक्रिया समझाइए। [3 अंक]

Figure 2.3. A changed number found in the Hindi version (Q-level finding outlined on the page)

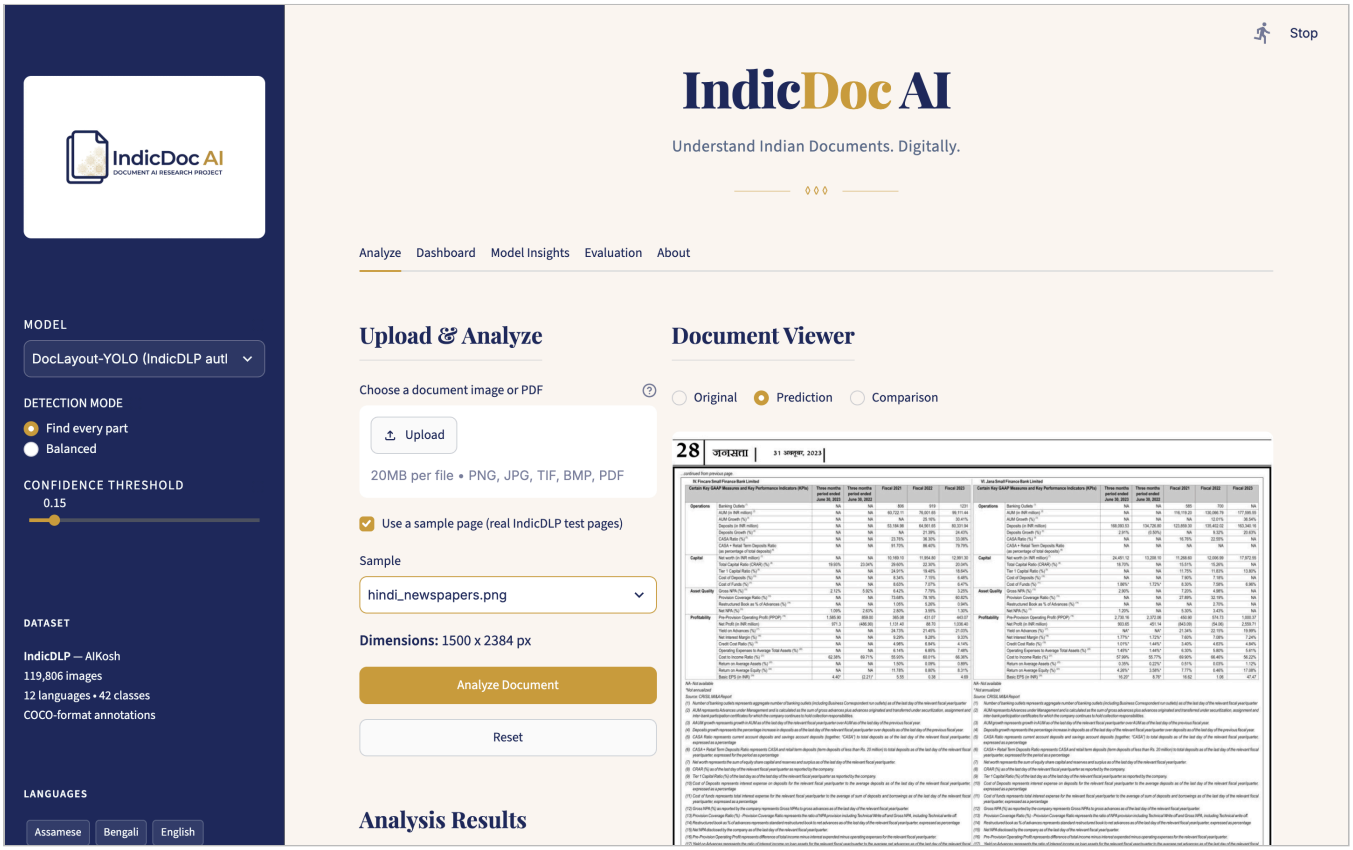


Figure 2.4. Streamlit app: uploading a document

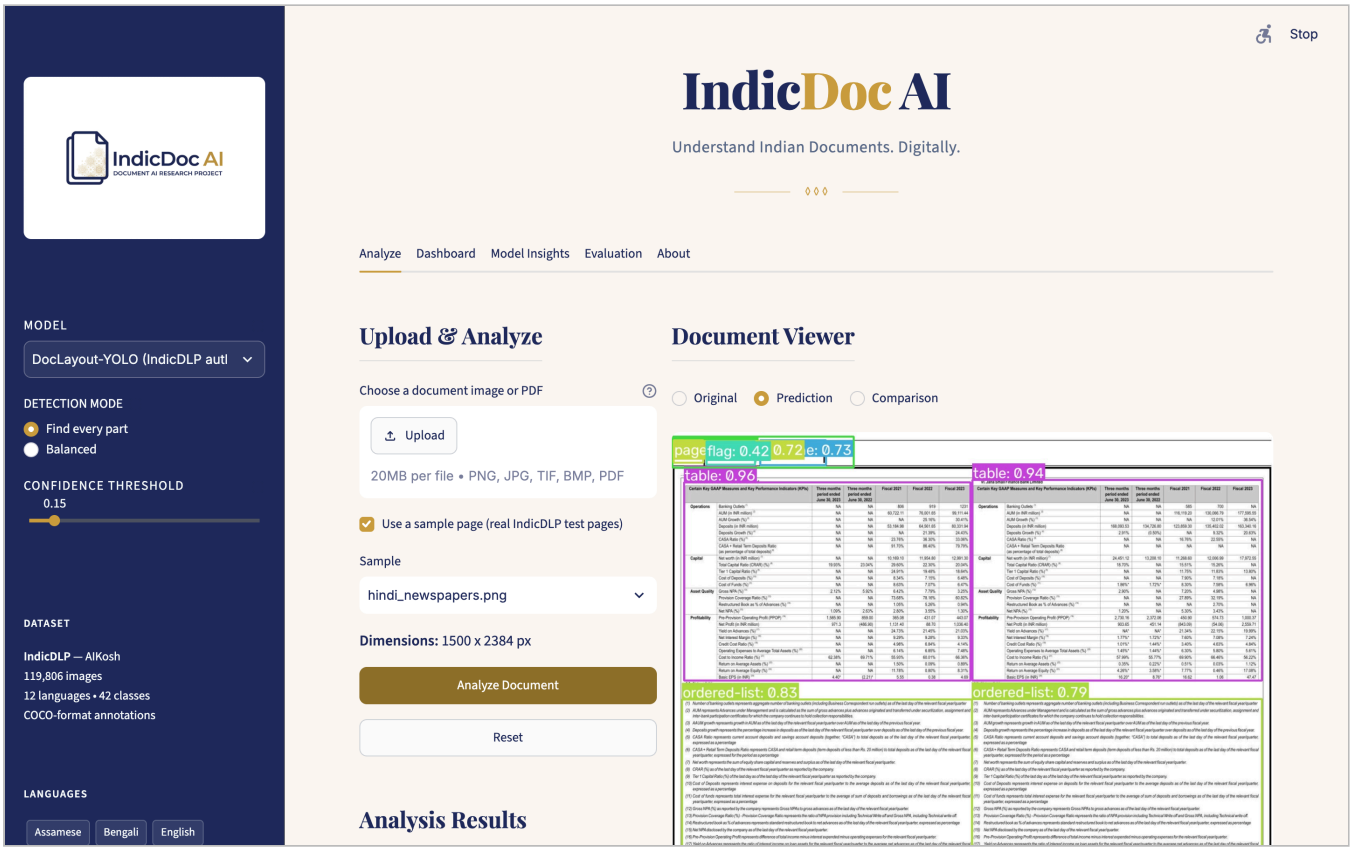


Figure 2.5. Layout regions found by DocLayout-YOLO

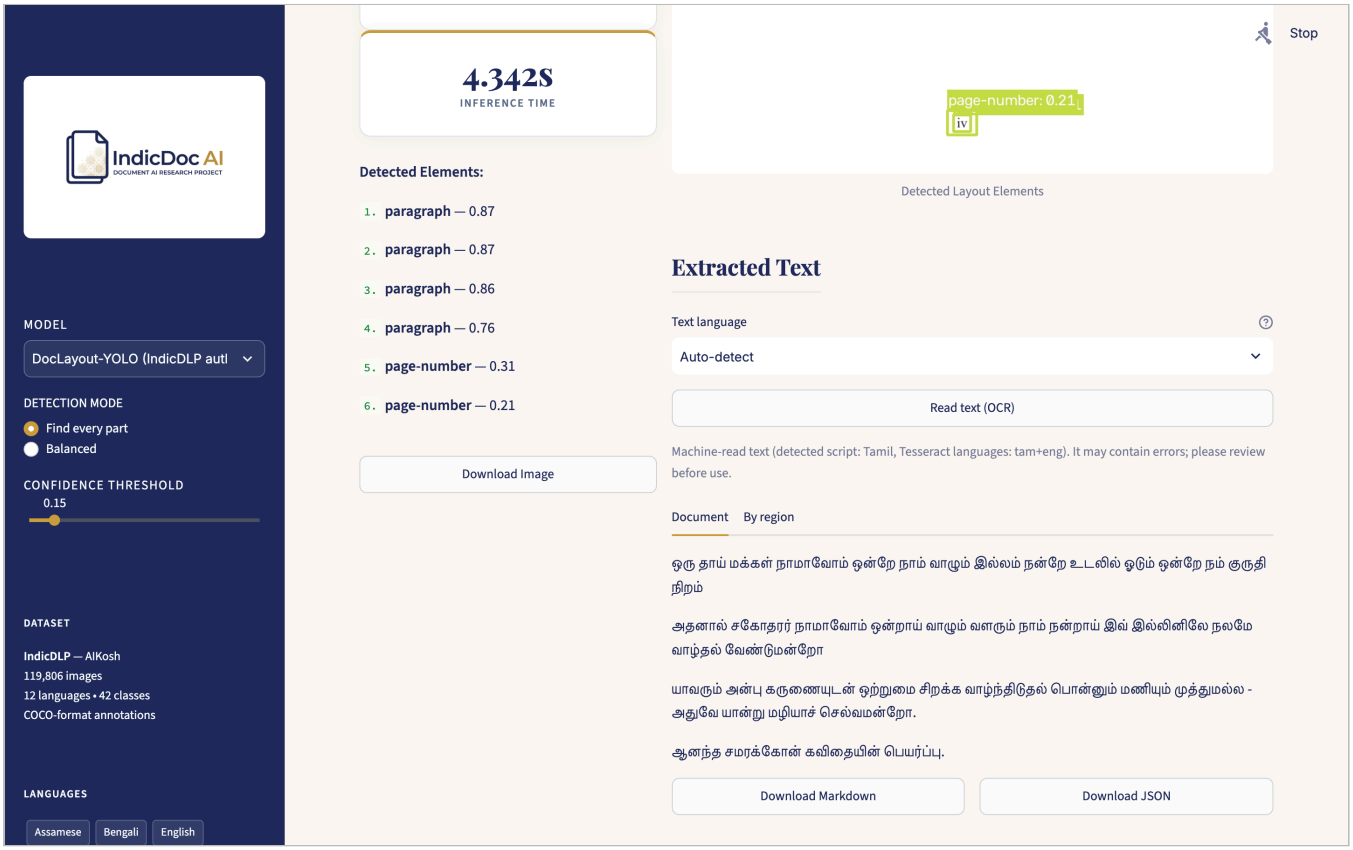


Figure 2.6. Text read from the regions (OCR), language detected per region

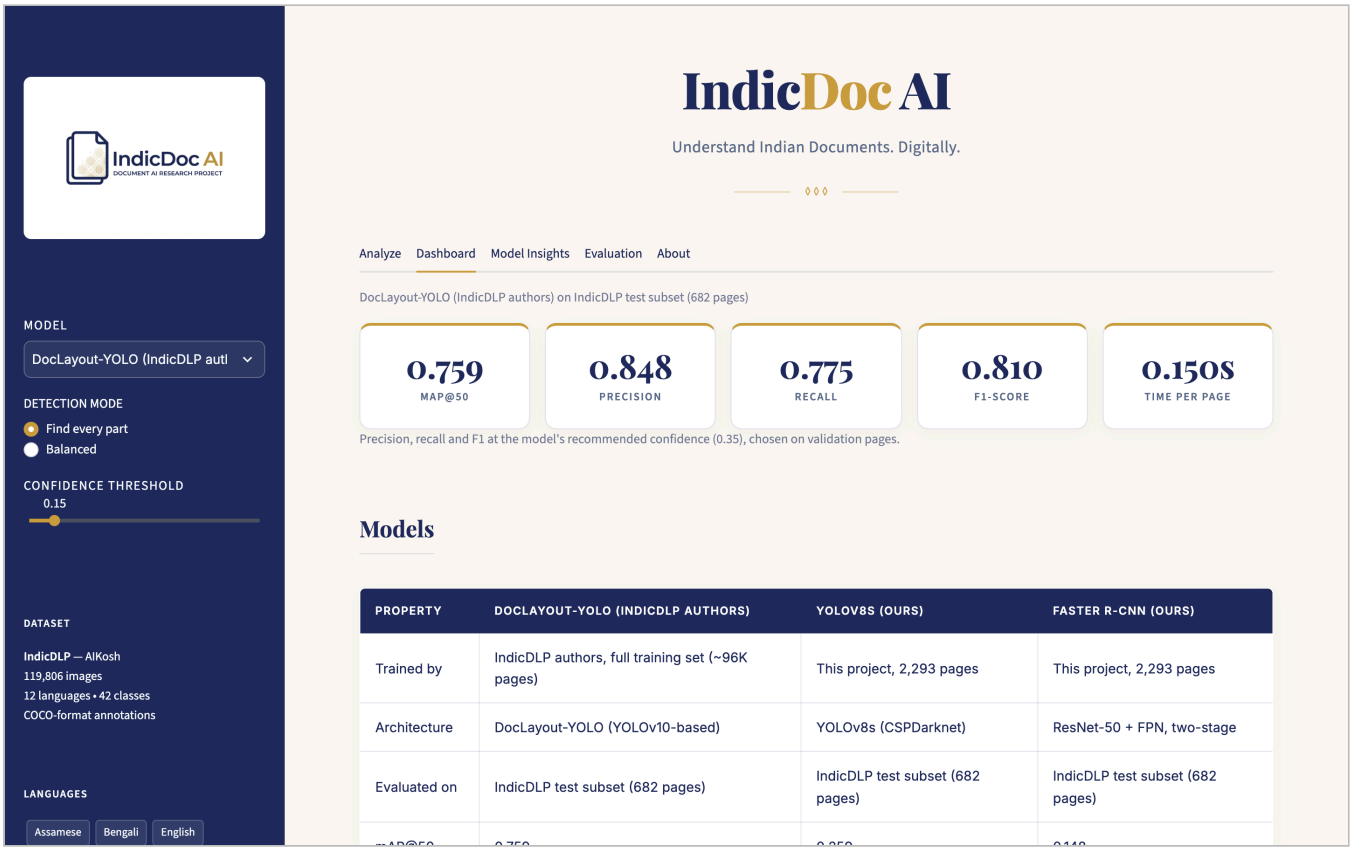


Figure 2.7. Dashboard of the analysed document

2.8 Source code of the core modules

The full code base is in the repository (Part 3). Below are the modules that make up IndicDoc Verify, the OCR stage, the layout models and the API, as they are in the repository.

ml/verify/__init__.py — 76 lines · Entry point: verify() parses the paper, runs every check and returns the report

```python
"""
IndicDoc Verify: pre-print checking of question papers and official documents in 12 Indian languages.

    verify(pages)                                structure checks on one paper
    verify(pages, source_pages=src)              + parity with the source-language version
    verify(pages, bank=[...])                    + questions repeated from earlier papers
    verify(pages, profile='office_memorandum')   format check of any document type against a rule profile
    verify(bilingual_pages, split_scripts=...)   one bilingual paper, checked version against version

`pages` are extract_document() results (layout regions with OCR text), one per page, so the checks run on top of
the same detector and OCR as the rest of the product, offline.
"""
from typing import Callable, Dict, List, Optional, Sequence

from ml.verify.checks import ERROR, INFO, WARNING, Issue, align, check_paper, compare_versions, find_repeats
from ml.verify.paper import Paper, Question, parse_paper
from ml.verify.profiles import check_format, list_profiles, load_profile
from ml.verify.text import split_lines_by_script


def split_bilingual(pages: List[Dict], scripts: Dict[str, str]) -> Dict[str, List[Dict]]:
    """
    One bilingual paper (both languages on the same pages) as one page list per language: every region's lines
    go to the stream of their script; lines without letters (numbers, formulas, option values) go to both.
    scripts: e.g. {'Latin': 'english', 'Devanagari': 'hindi'}.
    """
    out = {name: [] for name in scripts.values()}
    for page in pages:
        per = {name: [] for name in scripts.values()}
        for region in page['regions']:
            texts = split_lines_by_script(region.get('text', '') or '', scripts)
            for name, text in texts.items():
                if text.strip() or region['label'] in ('figure', 'table', 'formula'):
                    per[name].append({**region, 'text': text})
        for name in out:
            out[name].append({**page, 'regions': per[name]})
    return out


def verify(pages: List[Dict], source_pages: Optional[List[Dict]] = None, bank: Optional[Sequence[Dict]] = None,
           profile=None, split_scripts: Optional[Dict[str, str]] = None,
           semantic: Optional[Callable[[str, str], float]] = None, meta: Optional[Dict] = None) -> Dict:
    """
    Run every applicable check and return {'summary', 'issues', 'paper', 'source', 'meta'} (JSON-serialisable).
    With split_scripts, `pages` is one bilingual paper: the first script's stream is the source, the second the
    translation.
    """
    if split_scripts:
        streams = split_bilingual(pages, split_scripts)
        names = list(split_scripts.values())
        source_pages, pages = streams[names[0]], streams[names[1]]

    paper = parse_paper(pages)
    issues: List[Issue] = []
    # Question-level checks run for question papers: no profile, or a profile that asks for them
    if profile is None or load_profile(profile).get('question_checks', False):
        issues += check_paper(paper, version='target' if source_pages else 'paper')
    source = None
    if source_pages is not None:
        source = parse_paper(source_pages)
        issues += check_paper(source, version='source')
        issues += compare_versions(source, paper, semantic=semantic)
    if bank:
        issues += find_repeats(paper, bank)
    if profile is not None:
        issues += check_format(pages, profile)

    out = [i.to_dict() for i in issues]
    summary = {'questions': len(paper.questions), 'errors': sum(i['severity'] == ERROR for i in out),
               'warnings': sum(i['severity'] == WARNING for i in out), 'info': sum(i['severity'] == INFO for i in
```

```
out)}
    return {'summary': summary, 'issues': out, 'paper': paper.to_dict(),
            'source': source.to_dict() if source else None, 'meta': meta or {}}


__all__ = ['verify', 'parse_paper', 'check_paper', 'compare_versions', 'find_repeats', 'check_format',
           'list_profiles', 'load_profile', 'split_bilingual', 'align', 'Paper', 'Question', 'Issue']
```

ml/verify/text.py — 226 lines · Text helpers: Indian digits, numerals, question numbers, marks, option splitting, similarity

```python
"""
Text helpers for verification: digits in any Indian script, question numbers, marks, option markers, scripts.

Everything here works on the OCR text of detected regions and needs no model, so a translated paper can be
compared with its source without translating it: numbers, option markers and structure are the same in every
language even when the words are not.
"""
import re
import unicodedata
from collections import Counter
from typing import Dict, List, Optional, Tuple

# Zero digit of each Indian script (and Latin); the next nine code points are 1..9
_ZEROS = [0x0030, 0x0966, 0x09E6, 0x0A66, 0x0AE6, 0x0B66, 0x0BE6, 0x0C66, 0x0CE6, 0x0D66]
_DIGIT_MAP = {z + i: str(i) for z in _ZEROS for i in range(10)}

# Unicode blocks of the scripts the product reads; 'Latin' is ASCII letters only
SCRIPT_RANGES = {'Latin': [(0x41, 0x5A), (0x61, 0x7A)], 'Devanagari': [(0x0900, 0x097F)],
                 'Bengali': [(0x0980, 0x09FF)], 'Gurmukhi': [(0x0A00, 0x0A7F)], 'Gujarati': [(0x0A80, 0x0AFF)],
                 'Oriya': [(0x0B00, 0x0B7F)], 'Tamil': [(0x0B80, 0x0BFF)], 'Telugu': [(0x0C00, 0x0C7F)],
                 'Kannada': [(0x0C80, 0x0CFF)], 'Malayalam': [(0x0D00, 0x0D7F)]}
_DIGIT_CODEPOINTS = set(_DIGIT_MAP)
# Letters, vowel signs and viramas of all Indian scripts (Python's \w misses the combining vowel signs)
INDIC = '\u0900-\u0DFF'


_ZERO_WIDTH = {0x200B: None, 0x200C: None, 0x200D: None, 0xFEFF: None}


def normalize_digits(text: str) -> str:
    """
    '੪੨ १२' -> '42 12': every Indian-script digit becomes its ASCII digit. Zero-width joiners (Tesseract
    writes U+200C after Tamil and Malayalam viramas) are removed so that patterns match across them.
    """
    return ''.join(_DIGIT_MAP.get(ord(ch), ch) for ch in text.translate(_ZERO_WIDTH))


def script_of(text: str) -> Optional[str]:
    """The script with the most letters in `text` (digits and punctuation do not count); None if no letters."""
    counts = Counter()
    for ch in text:
        cp = ord(ch)
        if cp in _DIGIT_CODEPOINTS:
            continue
        for script, ranges in SCRIPT_RANGES.items():
            if any(lo <= cp <= hi for lo, hi in ranges):
                counts[script] += 1
                break
    return counts.most_common(1)[0][0] if counts else None


# ---------------------------------------------------------------------------------------------- numbers
# A letter before the digits makes them part of a name ('CO2', 'H2O'); a unit after them does not ('3A', '5kg')
_NUMBER = re.compile(r'(?<![\w.])(\d{1,3}(?:,\d{2,3})+|\d+)(?:\.(\d+))?(?!\d)')


def numerals(text: str) -> Counter:
    """
    Multiset of the numbers written in `text`, script-independent: '१,२०० kg and 2.50 m' -> {'1200': 1, '2.5': 1}.
    Thousands separators are dropped (Indian 1,00,000 and Western 100,000 compare equal) and trailing decimal
    zeros removed, so typesetting differences between versions do not count as mismatches.
    """
    out = Counter()
    for m in _NUMBER.finditer(normalize_digits(text)):
        whole = m.group(1).replace(',', '').lstrip('0') or '0'
        frac = (m.group(2) or '').rstrip('0')
        out[f'{whole}.{frac}' if frac else whole] += 1
    return out


# ------------------------------------------------------------------------------------ question numbers
```

```python
# 'Q.12', 'Q 12', 'Question 12', '12.', '12)', 'प्रश्न 12', 'प्र. 12', 'கேள்வி 12', 'ప్రశ్న 12' at the start of a region
_Q_WORDS = r'(?:Q(?:ue(?:stion)?)?\s*(?:No\.?)?|प्रश्न\s*(?:सं(?:ख्या)?\.?)?|प्र\.|கேள்வி|வினா|ప్రశ్న|ಪ್ರಶ್ನ|പ്രശ്ന|પ્રશ્ન|ପ୍ରଶ୍ନ|প্রশ্ন|ਪ੍ਰਸ਼ਨ|ప్ర|ಪ್ರ|പ്ര|ચો(?:દ્યો)?)'
_QNUM = re.compile(rf'^\s*(?:{_Q_WORDS}\s*[.:\-]?\s*(\d{{1,3}})\b\s*[.):\-]?|(\d{{1,3}})\s*[.):](?!\d))', re.IGNORECASE)


def question_number(text: str) -> Tuple[Optional[int], str]:
    """Number at the start of a question and the text after it: '12. Find x' -> (12, 'Find x')."""
    t = normalize_digits(text)
    m = _QNUM.match(t)
    if not m:
        return None, t.strip()
    return int(m.group(1) or m.group(2)), t[m.end():].strip()


# ---------------------------------------------------------------------------------------------- marks
_MARK_WORDS = r'(?:marks?|mks?|अंक|गुण|मदिप्पेण्[\w\u0900-\u0DFF]*|नम्बर|नम्बर|ಗುಣ|ગુણ|ଅଙ୍କ|ਅੰਕ|ನಮ್ಬರ|మార్కు|மார்க்[\w\u0900-\u0DFF]*)'
# Marks are printed in brackets or at the end of a line: '(2 marks)', '[2 अंक]', '(2M)', '... 2 Marks', '... [2]'.
# A number followed by 'marks' in the middle of a sentence ('she scored 45 marks in ...') is part of the question.
_MARKS_BRACKETED = re.compile(rf'[\[(]\s*(\d{{1,2}}(?:\.5)?)\s*(?:{_MARK_WORDS}|M)\s*[\])]', re.IGNORECASE)
_MARKS_EOL = re.compile(rf'(\d{{1,2}}(?:\.5)?)\s*{_MARK_WORDS}\s*[.)]?\s*$', re.IGNORECASE | re.MULTILINE)
_MARKS_BRACKET_EOL = re.compile(r'\[\s*(\d{1,2}(?:\.5)?)\s*\]\s*$', re.MULTILINE)
_TOTAL = re.compile(rf'(?:max(?:imum)?\.?\s*marks?|total\s*marks?|full\s*marks?|पूर्णां[ंँ]?क|अधिकतम\s*अंक|कुल\s*अंक|'
                    rf'மொத்த\s*மதிப்பெண்[\w\u0900-\u0DFF]*|அதிகபட்ச\s*மதிப்பெண்[\w\u0900-\u0DFF]*|పూర్ణాంక|మొత్\s*నమ్బర్|ಕುಲ\s*ಗುಣ|ବୁଲ\s*ଅଙ୍କ|'
                    rf'మొత్\s*నమ్బర్|మీతం\s*మార్కులు|ఒట్టు\s*అంక[\w\u0900-\u0DFF]*|ಆಕె\s*ಮಾರ್ಕ್[\w\u0900-\u0DFF]*)\s*[:\--]?\s*(\d{{1,4}})',
                    re.IGNORECASE)
_EACH = re.compile(rf'(?:each\s+question\s+carries|प्रत्येक\s+प्रश्न|ஒவ்வொரு\s+(?:கேள்விக்கும்|வினாவுக்கும்))\D{{0,20}}?'
                   rf'(\d{{1,2}}(?:\.5)?)[\s।]*{_MARK_WORDS}', re.IGNORECASE)  # '1 । अंक': OCR adds a danda


def marks_in(text: str) -> Optional[float]:
    """Marks printed with a question ('(2 marks)', '[2 अंक]', '... [2]' at a line end), or None."""
    t = normalize_digits(text)
    m = _MARKS_BRACKETED.search(t) or _MARKS_EOL.search(t) or _MARKS_BRACKET_EOL.search(t)
    return float(m.group(1)) if m else None


def strip_marks(text: str) -> str:
    t = normalize_digits(text)
    for pattern in (_MARKS_BRACKETED, _MARKS_EOL, _MARKS_BRACKET_EOL):
        t = pattern.sub(' ', t)
    return t


def declared_total(text: str) -> Optional[int]:
    """'Maximum Marks: 80' / 'पूर्णांक : 80' / 'மொத்த மதிப்பெண்கள்: 80' -> 80."""
    m = _TOTAL.search(normalize_digits(text))
    return int(m.group(1)) if m else None


def marks_each(text: str) -> Optional[float]:
    """'Each question carries 4 marks' / 'प्रत्येक प्रश्न 4 अंक का है' -> 4.0."""
    m = _EACH.search(normalize_digits(text))
    return float(m.group(1)) if m else None


# ---------------------------------------------------------------------------------------------- options
# Families of option markers, each in printed order. A paper uses one family per question.
OPTION_FAMILIES = {
    'upper': list('ABCDEF'), 'lower': list('abcdef'), 'digit': list('123456'),
    'devanagari': ['क', 'ख', 'ग', 'घ', 'ङ'], 'tamil': ['அ', 'ஆ', 'இ', 'ஈ', 'உ'],
    'bengali': ['ক', 'খ', 'গ', 'ঘ', 'ঙ'], 'roman': ['i', 'ii', 'iii', 'iv', 'v'],
}
_OPT_MARKER = re.compile(r'(?:(?<=\s)|^)(\s*([A-Fa-f1-6]|[कखगघङ]|[அஆஇஈஉ]|[কখগঘঙ]|i{1,3}|iv|v)\s*\)'
                         r'|(?:(?<=\s)|^)([A-Fa-f]|[कखगघङ]|[அஆஇஈஉ]|[কখগঘঙ])\s*[.)](?=\s)')


_BRACKET_TOKEN = re.compile(r'(?:(?<=\s)|^)\(((?:[^\s()]{1,3}\)|(?=\S))')


def split_options(text: str, options_box: bool = False) -> List[Tuple[str, str]]:
    """
    Options printed in `text` as (marker, option text): '(1) 40 km (2) 60 km' -> [('1', '40 km'), ('2', '60 km')].
    Only the longest run that starts at a family's first marker and follows its order counts, so a stray '(2)'
    inside a stem is not taken for an option. Digits must be in brackets ('(1)'), because '1.' starts a question.
    options_box: the text is a region the layout model labelled 'options'. There every word that opens with a
    bracket starts an option, which still counts options whose marker OCR garbled ('(ஈ)' read as '(FF)').
```

```
    """
    t = normalize_digits(text)
    if options_box:
        starts = [m.start() for m in _BRACKET_TOKEN.finditer(t)]
        family = split_options(text)
        if len(starts) >= 2 and len(starts) > len(family):
            out = []
            for k, start in enumerate(starts):
                chunk = t[start:starts[k + 1] if k + 1 < len(starts) else len(t)]
                close = chunk.find(')')
                marker, body = (chunk[1:close], chunk[close + 1:]) if 0 < close <= 4 else (str(k + 1), chunk[1:])
                out.append((marker.strip() or str(k + 1), ' '.join(body.split())))
            return out
        return family
    hits = [(m.start(), m.end(), m.group(1) or m.group(2)) for m in _OPT_MARKER.finditer(t)]
    best: List[Tuple[int, int, str]] = []
    for family in OPTION_FAMILIES.values():
        run: List[Tuple[int, int, str]] = []
        for hit in hits:
            if len(run) < len(family) and hit[2] == family[len(run)]:
                run.append(hit)
        if len(run) >= 2 and len(run) > len(best):
            best = run
    out = []
    for k, (start, end, marker) in enumerate(best):
        stop = best[k + 1][0] if k + 1 < len(best) else len(t)
        out.append((marker, ' '.join(t[end:stop].split())))
    return out


def stem_before_options(text: str) -> str:
    """The question text before its first option marker (the whole text if it has no options)."""
    opts = split_options(text)
    if not opts:
        return text
    t = normalize_digits(text)
    m = re.search(r'(?:(?<=\s)|^)\(?\s*' + re.escape(opts[0][0]) + r'\s*[.)]', t)
    return t[:m.start()] if m else t


# ---------------------------------------------------------------------------------------- similarity
def normalized(text: str) -> str:
    """Lower-case, NFC, digits normalised, punctuation and spacing removed: for comparing two question texts."""
    t = unicodedata.normalize('NFC', normalize_digits(text)).lower()
    t = re.sub(rf'[^\w\s{INDIC}]', ' ', t)
    return ' '.join(t.split())


def ngrams(text: str, n: int = 3) -> Counter:
    t = normalized(text).replace(' ', '_')
    return Counter(t[i:i + n] for i in range(max(0, len(t) - n + 1)))


def similarity(a: str, b: str) -> float:
    """Cosine similarity of character 3-gram counts (robust to OCR slips; 1.0 = same text)."""
    ga, gb = ngrams(a), ngrams(b)
    if not ga or not gb:
        return 0.0
    dot = sum(v * gb.get(k, 0) for k, v in ga.items())
    na = sum(v * v for v in ga.values()) ** 0.5
    nb = sum(v * v for v in gb.values()) ** 0.5
    return dot / (na * nb)


def split_lines_by_script(text: str, keep: Dict[str, str]) -> Dict[str, str]:
    """
    Lines of a bilingual region sorted into one text per script. Lines with no letters (a question number, a
    formula, '(1) 40') go to every script, because both versions need them.
    keep: script name -> stream name, e.g. {'Latin': 'english', 'Devanagari': 'hindi'}.
    """
    out = {name: [] for name in keep.values()}
    for line in text.splitlines():
        s = script_of(line)
        if s is None:
            for lines in out.values():
                lines.append(line)
        elif s in keep:
            out[keep[s]].append(line)
    return {name: '\n'.join(lines) for name, lines in out.items()}
```

```python
"""
Turn the layout + OCR output of a question paper (one extract_document() result per page) into questions.

The layout model already labels question papers: first-, second- and third-level questions, options and answers
(IndicDLP classes; DocLayout-YOLO reaches 0.81-0.90 AP@50 on them, see outputs/metrics/authors_metrics.json). A
question is a first-level-question region plus everything that follows it until the next one: sub-questions,
option regions, figures, tables and formulas. Regions the detector missed are recovered from their text when they
start with a question number.
"""
import re
from collections import Counter
from dataclasses import dataclass, field
from typing import Dict, List, Optional

from ml.verify.text import (declared_total, marks_each, marks_in, numerals, question_number, split_options,
                            stem_before_options, strip_marks)

QUESTION = {'first-level-question'}
SUB_QUESTION = {'second-level-question', 'third-level-question'}
OPTIONS = {'options'}
VISUALS = {'figure': 'figures', 'table': 'tables', 'formula': 'formulas'}
TEXT_FALLBACK = {'paragraph', 'unlabelled-text', 'ordered-list', 'answer'}
FURNITURE = {'header', 'footer', 'page-number', 'folio', 'website-link', 'jumpline'}


@dataclass
class Question:
    index: int                      # position in the paper, 0-based
    number: Optional[int]           # printed number, None if it could not be read
    page: int
    box: List[float]
    stem: str = ''
    options: List[str] = field(default_factory=list)
    option_markers: List[str] = field(default_factory=list)
    sub_questions: int = 0
    marks: Optional[float] = None
    sub_marks: float = 0.0          # marks printed on the sub-questions, used when the question has none
    figures: int = 0
    tables: int = 0
    formulas: int = 0
    section: str = ''
    boxes: List[Dict] = field(default_factory=list)  # every region of the question: {'page', 'box', 'label'}

    @property
    def text(self) -> str:
        return ' '.join([self.stem] + self.options)

    @property
    def stem_numerals(self) -> Counter:
        return numerals(self.stem)

    def to_dict(self) -> Dict:
        return {'index': self.index, 'number': self.number, 'page': self.page, 'box': self.box, 'stem': self.stem,
                'options': self.options, 'sub_questions': self.sub_questions, 'marks': self.marks,
                'figures': self.figures, 'tables': self.tables, 'formulas': self.formulas, 'section': self.section}


@dataclass
class Paper:
    questions: List[Question]
    declared_total: Optional[int]
    marks_each: Optional[float]
    page_numbers: List[Optional[int]]
    pages: int
    script: Optional[str] = None

    def to_dict(self) -> Dict:
        return {'questions': [q.to_dict() for q in self.questions], 'declared_total': self.declared_total,
                'marks_each': self.marks_each, 'page_numbers': self.page_numbers, 'pages': self.pages,
                'script': self.script}


def _starts_question(region: Dict) -> bool:
    if region['label'] in QUESTION:
        return True
    # The detector missed it, but the text starts like a question ('12.', 'Q.12', 'प्र 12')
    return region['label'] in TEXT_FALLBACK and question_number(strip_marks(region.get('text', '')))[0] is not None


```

```python
def parse_paper(pages: List[Dict]) -> Paper:
    """
    pages: extract_document() results in page order (each has 'regions' in reading order, with 'label', 'box' and
    'text'). Returns the questions with their numbers, options, marks and attached figures/tables/formulas.
    """
    questions: List[Question] = []
    header_text, page_numbers, section = [], [], ''
    current: Optional[Question] = None

    for p, page in enumerate(pages):
        page_no = None
        for region in page['regions']:
            label, text = region['label'], region.get('text', '') or ''
            ref = {'page': p, 'box': region['box'], 'label': label}
            if label == 'page-number':
                n = numerals(text)
                page_no = int(float(next(iter(n)))) if len(n) == 1 else page_no
                continue
            if label in FURNITURE:
                header_text.append(text)
                continue
            if label in ('section-title', 'sub-section-title', 'headline', 'chapter-title'):
                section = ' '.join(text.split())
                header_text.append(text)
                continue
            if _starts_question(region):
                # Right-aligned marks ('[2 marks]') are often read as the region's first line: drop them first
                number, rest = question_number(strip_marks(text))
                current = Question(index=len(questions), number=number, page=p, box=region['box'], section=section,
                                   boxes=[ref], marks=marks_in(text))
                questions.append(current)
                _absorb_text(current, rest)
                continue
            if current is None:
                header_text.append(text)  # instructions, title, maximum marks ... before question 1
                continue
            current.boxes.append(ref)
            if label in SUB_QUESTION:
                current.sub_questions += 1
                _absorb_text(current, text, sub=True)
            elif label in OPTIONS:
                _absorb_options(current, text, region.get('words'))
            elif label in VISUALS:
                setattr(current, VISUALS[label], getattr(current, VISUALS[label]) + 1)
            else:
                _absorb_text(current, text)
        page_numbers.append(page_no)

    _split_merged_numbers(questions)
    head = '\n'.join(header_text)
    each = marks_each(head)
    for q in questions:
        if q.marks is None and q.sub_marks:
            q.marks = q.sub_marks
        if q.marks is None and each is not None:
            q.marks = each
    script = next((pg.get('script') for pg in pages if pg.get('script')), None)
    return Paper(questions=questions, declared_total=declared_total(head), marks_each=each,
                 page_numbers=page_numbers, pages=len(pages), script=script)


_MERGED = re.compile(r'^(\d{1,3})\.(\d+)\b\s*(.*)$', re.DOTALL)


def _split_merged_numbers(questions: List[Question]) -> None:
    """
    OCR often drops the space after a question number when the stem starts with a number: '5. 16 V ...' is read as
    '5.16 V ...', which is not a question number. When a question has no number and its text starts with 'N.M'
    where N is exactly the next number in sequence, it is split back into the number N and the stem 'M ...'.
    """
    prev = None
    for q in questions:
        if q.number is None and prev is not None:
            m = _MERGED.match(q.stem)
            if m and int(m.group(1)) == prev + 1:
                q.number, q.stem = int(m.group(1)), f'{m.group(2)} {m.group(3)}'.strip()
        prev = q.number if q.number is not None else (prev + 1 if prev is not None else None)


def _absorb_text(q: Question, text: str, sub: bool = False) -> None:
    """Add question text; options printed inside the same region are split off, marks recorded."""
```

```
    m = marks_in(text)
    if m is not None:
        if sub:
            q.sub_marks += m
        elif q.marks is None:
            q.marks = m
    clean = strip_marks(text)
    opts = split_options(clean)
    if opts and not q.options:
        q.stem = ' '.join((q.stem + ' ' + stem_before_options(clean)).split())
        q.option_markers = [m for m, _ in opts]
        q.options = [t for _, t in opts]
    else:
        q.stem = ' '.join((q.stem + ' ' + clean).split())


_MARKER_TOKEN = re.compile(r'^[(\[]?[^\s()\[\]]{1,3}[)\].]$|^[(\[][^\s()\[\]]{1,3}$|^[^\s()]{1,2}$|^\(\)$')


def option_cells(words: List[Dict]) -> List[List[Dict]]:
    """
    The options of a question from the positions of its words. Options are typeset as separate cells (a grid or
    one per line), so a horizontal gap much wider than a word space, or a new line, starts a new option. This keeps
    working when OCR garbles or drops the option markers themselves ('(4)' read as '4)' or lost).
    """
    if not words:
        return []
    height = sorted(w['height'] for w in words)[len(words) // 2]
    lines: List[List[Dict]] = []
    for w in sorted(words, key=lambda w: w['top'] + w['height'] / 2):
        cy = w['top'] + w['height'] / 2
        if lines and abs(cy - (lines[-1][0]['top'] + lines[-1][0]['height'] / 2)) < 0.5 * height:
            lines[-1].append(w)
        else:
            lines.append([w])
    cells = []
    for line in lines:
        line.sort(key=lambda w: w['left'])
        cell = [line[0]]
        for prev, w in zip(line, line[1:]):
            if w['left'] - (prev['left'] + prev['width']) > 2.0 * height:
                cells.append(cell)
                cell = [w]
            else:
                cell.append(w)
        cells.append(cell)
    return cells


# An option marker that OCR ran into the option's value, and what is left of the value
_GLUED = [re.compile(r'^\(?[^\s()]{1,3}\)(\S+)$'),   # '(3)12', '2)8', '(I)3A'
          re.compile(r'^\([^\s()](\d\S*)$'),          # '(128' = '(1' + '28', '(M3A'
          re.compile(r'^[^\w\s(\[](\S+)$'),             # '©2A' ('(c)' read as one symbol), '@7A'
          re.compile(r'^[A-Za-z](\d\S*)$')]             # 'M4A' ('(a)' read as 'M')


def strip_marker(tokens: List[str]) -> List[str]:
    """Remove the option marker from the start of an option: a token of its own, or glued to the value."""
    if not tokens:
        return tokens
    if len(tokens) > 1 and _MARKER_TOKEN.match(tokens[0]):
        return tokens[1:]
    for pattern in _GLUED:
        m = pattern.match(tokens[0])
        if m:
            return [m.group(1)] + tokens[1:]
    return tokens


def _cell_options(cell: List[Dict]) -> List[str]:
    """One cell's option text without its marker; a cell holding several marked options ('(a) x (b) y') is split."""
    text = ' '.join(w['text'] for w in cell)
    inline = split_options(text, options_box=True)
    if len(inline) >= 2:
        return [t for _, t in inline]
    return [' '.join(strip_marker(text.split()))]


def _absorb_options(q: Question, text: str, words: Optional[List[Dict]] = None) -> None:
    """
    An options region: several options in one box (separated by layout when word positions are known, else by
```

```
    their markers), or one option per box (appended in order).
    """
    if words:
        opts = [o for cell in option_cells(words) for o in _cell_options(cell)]
        q.options += opts
        q.option_markers += [str(len(q.option_markers) + k + 1) for k in range(len(opts))]
        return
    opts = split_options(text, options_box=True)
    if len(opts) >= 2:
        q.option_markers += [m for m, _ in opts]
        q.options += [t for _, t in opts]
    elif text.strip():
        q.options.append(' '.join(text.split()))
        q.option_markers.append(str(len(q.options)))
```

ml/verify/checks.py — 277 lines · Structure checks, alignment of two language versions (translation parity), repeats

```
"""
Checks run on parsed question papers. Each check reflects an error that has reached candidates in a real Indian exam
(see docs/PRODUCT.md for the cases):

  check_paper      one paper: numbering gaps / repeats, duplicated questions, missing or repeated options,
                   marks that do not add up to the printed total, page numbering
  compare_versions two language versions of the same paper: missing / extra questions, option count, marks,
                   sub-questions, the numbers in every stem and option, figures / tables / formulas
  find_repeats     questions that repeat an earlier paper (a question bank)

Issues carry the question, page and box, so the report can point the human vetter at the exact place.
"""
from collections import Counter
from dataclasses import asdict, dataclass, field
from typing import Callable, Dict, List, Optional, Sequence, Tuple

from ml.verify.paper import Paper, Question
from ml.verify.text import numerals, normalized, similarity

ERROR, WARNING, INFO = 'error', 'warning', 'info'

# Two questions whose text is at least this similar (character 3-gram cosine) are the same question.
# Chosen on the benchmark's clean papers, where no two different questions came close (scripts/evaluate_verify.py).
DUPLICATE_SIMILARITY = 0.9
REPEAT_SIMILARITY = 0.8


@dataclass
class Issue:
    code: str
    severity: str
    message: str
    question: Optional[int] = None       # printed question number (or position + 1 if unreadable)
    page: Optional[int] = None
    box: Optional[List[float]] = None
    version: str = 'paper'               # 'paper', 'source' or 'target'
    details: Dict = field(default_factory=dict)

    def to_dict(self) -> Dict:
        return asdict(self)


def _qid(q: Question) -> int:
    return q.number if q.number is not None else q.index + 1


def _issue(code, severity, message, q: Optional[Question] = None, version='paper', **details) -> Issue:
    return Issue(code, severity, message, question=_qid(q) if q else None, page=q.page if q else None,
                 box=q.box if q else None, version=version, details=details)


# ---------------------------------------------------------------------------- one paper
def check_paper(paper: Paper, version: str = 'paper') -> List[Issue]:
    issues: List[Issue] = []
    qs = paper.questions
    if not qs:
        return [Issue('NO_QUESTIONS', ERROR, 'No questions were found on these pages.', version=version)]

    # Numbering: every number once, no gaps. Sections may restart at 1, so a reset is not a gap.
    numbered = [q for q in qs if q.number is not None]
    for q in qs:
        if q.number is None:
            issues.append(_issue('NUMBER_UNREADABLE', WARNING, f'Question at position {q.index + 1} has no '
                                 'readable number.', q, version))
```

```
    prev = None
    seen = {}
    for q in numbered:
        if prev is not None and q.number not in (1, prev.number + 1) and q.section == prev.section:
            if q.number > prev.number + 1:
                missing = list(range(prev.number + 1, q.number))
                issues.append(_issue('NUMBER_GAP', ERROR, f'Question numbers jump from {prev.number} to {q.number} '
                                     f'(missing {", ".join(map(str, missing[:5]))}).', q, version, missing=missing))
            elif q.number <= prev.number:
                issues.append(_issue('NUMBER_ORDER', ERROR, f'Question {q.number} comes after question '
                                     f'{prev.number}.', q, version))
        key = (q.section, q.number)
        if key in seen:
            issues.append(_issue('NUMBER_REPEATED', ERROR, f'Question number {q.number} is used twice '
                                 f'(pages {seen[key].page + 1} and {q.page + 1}).', q, version))
        seen[key] = q
        prev = q

    # The same question printed twice (e.g. an MCQ repeated in an MBBS paper, AP High Court, July 2026)
    for i, a in enumerate(qs):
        for b in qs[i + 1:]:
            if len(normalized(a.stem)) >= 15 and similarity(a.stem, b.stem) >= DUPLICATE_SIMILARITY \
                    and a.stem_numerals == b.stem_numerals:
                issues.append(_issue('DUPLICATE_QUESTION', ERROR, f'Question {_qid(b)} repeats question {_qid(a)}.',
                                     b, version, first=_qid(a), similarity=round(similarity(a.stem, b.stem), 3)))

    # Options: MCQs in one paper normally all have the same number of options
    counts = Counter(len(q.options) for q in qs if q.options)
    if counts:
        usual, n_usual = counts.most_common(1)[0]
        for q in qs:
            if q.options and len(q.options) != usual and n_usual >= 3:
                issues.append(_issue('OPTION_COUNT', ERROR, f'Question {_qid(q)} has {len(q.options)} options; the '
                                     f'other questions have {usual}.', q, version, found=len(q.options),
                                     expected=usual))
        for q in qs:
            texts = [normalized(o) for o in q.options if normalized(o)]
            dup = [t for t, c in Counter(texts).items() if c > 1]
            if dup:
                issues.append(_issue('OPTION_REPEATED', ERROR, f'Question {_qid(q)} prints the same option twice '
                                     f'("{dup[0][:40]}").', q, version))
            if any(not normalized(o) for o in q.options):
                issues.append(_issue('OPTION_EMPTY', WARNING, f'Question {_qid(q)} has an empty option.', q,
version))

    # Marks: printed per-question marks should add up to the printed maximum
    with_marks = [q for q in qs if q.marks is not None]
    if paper.declared_total is not None and with_marks:
        if len(with_marks) == len(qs):
            total = sum(q.marks for q in qs)
            if abs(total - paper.declared_total) > 1e-6:
                issues.append(Issue('MARKS_TOTAL', ERROR, f'Question marks add up to {total:g}, but the paper says '
                                    f'maximum marks {paper.declared_total}.', version=version,
                                    details={'sum': total, 'declared': paper.declared_total}))
        else:
            missing = [_qid(q) for q in qs if q.marks is None]
            issues.append(Issue('MARKS_MISSING', WARNING, f'{len(missing)} question(s) show no marks '
                                f'({", ".join(map(str, missing[:8]))}), so the total cannot be checked.',
                                version=version, details={'questions': missing}))

    # Page numbers: printed page numbers should run without gaps
    printed = [(i, n) for i, n in enumerate(paper.page_numbers) if n is not None]
    for (i, a), (j, b) in zip(printed, printed[1:]):
        if b - a != j - i:
            issues.append(Issue('PAGE_NUMBERS', WARNING, f'Printed page number goes from {a} to {b} between '
                                f'sheets {i + 1} and {j + 1}.', page=j, version=version))
    return issues


# ---------------------------------------------------------------------------- two versions
def _skeleton(q: Question) -> Tuple:
    return (len(q.options), q.sub_questions, q.marks, q.figures + q.tables + q.formulas)


def align(src: Sequence[Question], tgt: Sequence[Question]) -> List[Tuple[Optional[int], Optional[int]]]:
    """
    Pair the questions of two versions with a global alignment (Needleman-Wunsch). The score uses what is the same
    in every language: the printed question number, the numbers in the stem and options, the option count,
    sub-questions, marks and visuals. Content keeps the pairing right even when a dropped question shifted the
    numbering of every later one. Returns (source index, target index) pairs; None = no partner.
    """
```

```
    def nums(q: Question) -> Counter:
        out = Counter(q.stem_numerals)
        for o in q.options:
            out.update(numerals(o))
        return out

    src_n, tgt_n = [nums(q) for q in src], [nums(q) for q in tgt]

    def score(i: int, j: int) -> float:
        a, b, na, nb = src[i], tgt[j], src_n[i], tgt_n[j]
        content = (sum((na & nb).values()) / max(1, sum((na | nb).values()))) if (na or nb) else 0.5
        number = 0.0 if a.number is None or b.number is None else (1.0 if a.number == b.number else -0.5)
        return 2 * content + number + 0.5 * (_skeleton(a) == _skeleton(b))

    n, m, gap = len(src), len(tgt), -0.6
    dp = [[0.0] * (m + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        dp[i][0] = i * gap
    for j in range(1, m + 1):
        dp[0][j] = j * gap
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            dp[i][j] = max(dp[i - 1][j - 1] + score(i - 1, j - 1), dp[i - 1][j] + gap, dp[i][j - 1] + gap)
    pairs, i, j = [], n, m
    while i > 0 or j > 0:
        if i > 0 and j > 0 and abs(dp[i][j] - (dp[i - 1][j - 1] + score(i - 1, j - 1))) < 1e-9:
            pairs.append((i - 1, j - 1)); i, j = i - 1, j - 1
        elif i > 0 and abs(dp[i][j] - (dp[i - 1][j] + gap)) < 1e-9:
            pairs.append((i - 1, None)); i -= 1
        else:
            pairs.append((None, j - 1)); j -= 1
    return pairs[::-1]


def _diff(a: Counter, b: Counter) -> Tuple[List[str], List[str]]:
    return sorted((a - b).elements()), sorted((b - a).elements())


def compare_versions(source: Paper, target: Paper,
                     semantic: Optional[Callable[[str, str], float]] = None,
                     semantic_threshold: float = 0.6) -> List[Issue]:
    """
    Differences between a source paper and its translation that a candidate would notice. No translation is
    needed: numbers, option counts, marks and structure must match in every language.
    semantic: optional cross-lingual similarity (e.g. a LaBSE sentence-embedding cosine); low scores are flagged
    for the vetter as possible changes of meaning.
    """
    issues: List[Issue] = []
    if source.declared_total is not None and target.declared_total is not None \
            and source.declared_total != target.declared_total:
        issues.append(Issue('TOTAL_MISMATCH', ERROR, f'Maximum marks: {source.declared_total} in the source, '
                            f'{target.declared_total} in the translation.', version='target'))
    if len(source.questions) != len(target.questions):
        issues.append(Issue('QUESTION_COUNT', ERROR, f'{len(source.questions)} questions in the source, '
                            f'{len(target.questions)} in the translation.', version='target'))

    for i, j in align(source.questions, target.questions):
        if j is None:
            q = source.questions[i]
            issues.append(_issue('MISSING_IN_TRANSLATION', ERROR, f'Question {_qid(q)} of the source is not in the '
                                 'translation.', q, 'source'))
            continue
        if i is None:
            q = target.questions[j]
            issues.append(_issue('EXTRA_IN_TRANSLATION', ERROR, f'Question {_qid(q)} of the translation has no '
                                 'counterpart in the source.', q, 'target'))
            continue
        s, t = source.questions[i], target.questions[j]
        qn = _qid(s)
        if len(s.options) != len(t.options):
            issues.append(_issue('OPTION_COUNT_MISMATCH', ERROR, f'Question {qn}: {len(s.options)} options in the '
                                 f'source, {len(t.options)} in the translation.', t, 'target',
                                 source=len(s.options), target=len(t.options)))
        if s.sub_questions != t.sub_questions:
            issues.append(_issue('SUBQUESTION_MISMATCH', ERROR, f'Question {qn}: {s.sub_questions} sub-questions in
'
                                 f'the source, {t.sub_questions} in the translation.', t, 'target'))
        if s.marks is not None and t.marks is not None and s.marks != t.marks:
            issues.append(_issue('MARKS_MISMATCH', ERROR, f'Question {qn}: {s.marks:g} marks in the source, '
                                 f'{t.marks:g} in the translation.', t, 'target'))
        only_s, only_t = _diff(s.stem_numerals, t.stem_numerals)
```

```
        if only_s or only_t:
            issues.append(_issue('NUMBER_MISMATCH', ERROR, f'Question {qn}: numbers differ - source has '
                                 f'{", ".join(only_s) or "nothing extra"}, translation has '
                                 f'{", ".join(only_t) or "nothing extra"}.', t, 'target',
                                 source_only=only_s, target_only=only_t))
        # One finding per question (the vetter checks the question once), listing every option that differs
        differing = []
        for k, (so, to) in enumerate(zip(s.options, t.options)):
            os_, ot = _diff(numerals(so), numerals(to))
            if os_ or ot:
                differing.append({'option': k + 1, 'source_only': os_, 'target_only': ot})
        if differing:
            parts = '; '.join(f'option {d["option"]}: source {", ".join(d["source_only"]) or "-"}, translation '
                              f'{", ".join(d["target_only"]) or "-"}' for d in differing)
            issues.append(_issue('OPTION_NUMBER_MISMATCH', ERROR, f'Question {qn}: option numbers differ ({parts});
'
                                 'the answer key may no longer hold.', t, 'target', options=differing))
        for attr in ('figures', 'tables', 'formulas'):
            if getattr(s, attr) != getattr(t, attr):
                issues.append(_issue('VISUAL_MISMATCH', WARNING, f'Question {qn}: {getattr(s, attr)} {attr} in the '
                                     f'source, {getattr(t, attr)} in the translation.', t, 'target', kind=attr))
        if semantic is not None and s.stem and t.stem:
            score = semantic(s.stem, t.stem)
            if score < semantic_threshold:
                issues.append(_issue('MEANING_CHECK', WARNING, f'Question {qn}: the translation may not say the same
'
                                     f'thing (cross-lingual similarity {score:.2f}).', t, 'target',
                                     similarity=round(score, 3)))
    return issues


# ---------------------------------------------------------------------------- question bank
def find_repeats(paper: Paper, bank: Sequence[Dict], threshold: float = REPEAT_SIMILARITY) -> List[Issue]:
    """
    Questions that already appeared in earlier papers (UGC-NET June 2026 papers were re-held partly over questions
    repeated from earlier years). bank: [{'text': ..., 'source': 'UGC-NET Dec 2025, Q14'}, ...] in the same
language.
    """
    issues = []
    entries = [(b, None) for b in bank]
    for q in paper.questions:
        if len(normalized(q.stem)) < 15:
            continue
        best, best_score = None, 0.0
        for entry, _ in entries:
            # Same wording AND same numbers: a question with new values is a new question
            if numerals(entry.get('text', '')) != q.stem_numerals:
                continue
            sc = similarity(q.stem, entry.get('text', ''))
            if sc > best_score:
                best, best_score = entry, sc
        if best is not None and best_score >= threshold:
            issues.append(_issue('REPEATED_FROM_BANK', WARNING, f'Question {_qid(q)} closely matches '
                                 f'{best.get("source", "an earlier paper")} (similarity {best_score:.2f}).', q,
                                 similarity=round(best_score, 3), source=best.get('source')))
    return issues
```

ml/verify/profiles.py

107 lines · Format profiles: any document type described as JSON rules

```
"""
Format verification with profiles: a JSON rule file says which elements a kind of document must contain, where,
and in what order (e.g. an Office Memorandum per the Central Secretariat Manual of Office Procedure, or the front
matter of a question paper). Elements are found by layout label, by a pattern in the OCR text, or both, so the
same engine works for any document type a customer describes, in any of the 12 languages.
"""
import json
import re
from collections import Counter
from pathlib import Path
from typing import Dict, List, Optional

from ml.verify.checks import ERROR, INFO, WARNING, Issue
from ml.verify.text import SCRIPT_RANGES, normalize_digits

PROFILE_DIR = Path(__file__).parent / 'profiles'


def list_profiles() -> Dict[str, str]:
    """Profile id -> display name, for every JSON file in ml/verify/profiles."""
    return {p.stem: json.loads(p.read_text())['name'] for p in sorted(PROFILE_DIR.glob('*.json'))}

```

```python

def load_profile(profile) -> Dict:
    """A profile id ('office_memorandum'), a path to a JSON file, or an already-loaded dict."""
    if isinstance(profile, dict):
        return profile
    path = Path(profile)
    if not path.suffix:
        path = PROFILE_DIR / f'{profile}.json'
    return json.loads(path.read_text())


def _find(element: Dict, pages: List[Dict]) -> Optional[Dict]:
    """First region (in reading order) that satisfies the element's label and text rules, with its position."""
    labels = set(element.get('labels', []))
    pattern = re.compile(element['pattern'], re.IGNORECASE | re.MULTILINE) if element.get('pattern') else None
    where = element.get('where')
    page_range = range(len(pages))
    if where == 'first_page':
        page_range = range(min(1, len(pages)))
    elif where == 'last_page':
        page_range = range(max(0, len(pages) - 1), len(pages))
    for p in page_range:
        for k, region in enumerate(pages[p]['regions']):
            if labels and region['label'] not in labels:
                continue
            if pattern and not pattern.search(normalize_digits(region.get('text', '') or '')):
                continue
            return {'page': p, 'order': k, 'box': region['box'], 'label': region['label']}
    return None


def _script_shares(pages: List[Dict]) -> Dict[str, float]:
    counts = Counter()
    for page in pages:
        for region in page['regions']:
            for ch in region.get('text', '') or '':
                for script, ranges in SCRIPT_RANGES.items():
                    if any(lo <= ord(ch) <= hi for lo, hi in ranges):
                        counts[script] += 1
                        break
    total = sum(counts.values())
    return {s: c / total for s, c in counts.items()} if total else {}


def check_format(pages: List[Dict], profile) -> List[Issue]:
    """Issues for missing, misplaced or out-of-order elements, and for the profile's bilingual rule."""
    prof = load_profile(profile)
    issues: List[Issue] = []
    found = {}
    for el in prof['elements']:
        hit = _find(el, pages)
        if hit:
            found[el['id']] = hit
        elif el.get('required', True):
            where = {'first_page': ' on the first page', 'last_page': ' on the last page'}.get(el.get('where'), '')
            issues.append(Issue('FORMAT_MISSING', ERROR, f'{el["name"]} not found{where}.', version='paper',
                                details={'element': el['id']}))
        else:
            issues.append(Issue('FORMAT_OPTIONAL_MISSING', INFO, f'{el["name"]} not found (optional).',
                                version='paper', details={'element': el['id']}))

    names = {el['id']: el['name'] for el in prof['elements']}
    present = [e for e in prof.get('order', []) if e in found]
    for a, b in zip(present, present[1:]):
        pa, pb = found[a], found[b]
        if (pb['page'], pb['order']) < (pa['page'], pa['order']):
            issues.append(Issue('FORMAT_ORDER', WARNING, f'{names[b]} appears before {names[a]}.', page=pb['page'],
                                box=pb['box'], version='paper', details={'before': b, 'after': a}))

    rule = prof.get('bilingual')
    if rule:
        shares = _script_shares(pages)
        low = [s for s in rule['scripts'] if shares.get(s, 0.0) < rule.get('min_share', 0.15)]
        if low:
            issues.append(Issue('BILINGUAL', WARNING, f'Not bilingual: little or no {" / ".join(low)} text. '
                                f'{rule.get("rule", "")}'.strip(), version='paper',
                                details={'shares': {k: round(v, 3) for k, v in shares.items()}}))

    if prof.get('page_numbers') and len(pages) > 1:
        unnumbered = [i + 1 for i, pg in enumerate(pages) if not any(r['label'] == 'page-number'
                                                                     for r in pg['regions'])]
```

```
        if unnumbered:
            issues.append(Issue('PAGE_NUMBER_MISSING', WARNING, f'No printed page number on sheet(s) '
                                f'{", ".join(map(str, unnumbered[:10]))}.', version='paper'))
    return issues
```

ml/verify/profiles/question_paper.json

42 lines · The question-paper profile

```
{
  "name": "Question paper (pre-print check)",
  "description": "Front matter every question paper must carry before it is sent for printing. Question-level checks (numbering, options, marks, translation parity) run separately.",
  "question_checks": true,
  "elements": [
    {
      "id": "max_marks",
      "name": "Maximum marks",
      "pattern": "(max(imum)?\\.?\\s*marks?|total\\s*marks?|full\\s*marks?|पूर्ण[ाी]?क|अधिकतम\\s*अंक|மொத்த\\s*மதிப்பெண்|அதிகபட்ச\\s*மதிப்பெண்|పూర్ణ\\s*ాంక|ಪೂರ್ಣ\\s*ಾಂಕ|ആകെ\\s*മാർക്ക്|మొత్తం\\s*మార్కులు|ഗൊ\\s*ന്മ്ഴ)",
      "where": "first_page",
      "required": true
    },
    {
      "id": "time",
      "name": "Time allowed",
      "pattern": "(time\\s*(allowed)?\\s*[:\\-]|duration|\\bhours?\\b|समय|घंटे|நேரம்|மணி|సమయ|ಸಮಯ|സമയം|సమయం|ಸಮಯ|ସମୟ)",
      "where": "first_page",
      "required": true
    },
    {
      "id": "instructions",
      "name": "Instructions to candidates",
      "pattern": "(instructions?|निर्देश|அறிவுரை|அறிவுறுத்தல்|నిర్దేశ|ಸೂಚನ|നിർദ്ദേശ|సూచన|ನಿರ್ದೇಶ)",
      "where": "first_page",
      "required": true
    },
    {
      "id": "questions",
      "name": "Questions",
      "labels": [
        "first-level-question"
      ],
      "required": true
    }
  ],
  "order": [
    "max_marks",
    "instructions",
    "questions"
  ],
  "page_numbers": true
}
```

ml/verify/report.py

87 lines · The report: findings outlined on the pages, HTML output

```
"""
The verification report a paper-setter or vetter reads: a verdict, the issues sorted by severity with the question
and page, and the pages with each problem outlined. Self-contained HTML (images embedded), safe to e-mail or print.
"""
import base64
import html
import io
from typing import Dict, List, Optional

import numpy as np
from PIL import Image, ImageDraw

SEVERITY_ORDER = {'error': 0, 'warning': 1, 'info': 2}
_COLOURS = {'error': (200, 30, 45), 'warning': (220, 140, 0), 'info': (60, 110, 200)}


def annotate(image: np.ndarray, numbered_issues: List) -> Image.Image:
    """Page image (RGB) with the box of every (number, issue) pair outlined in its severity colour and numbered."""
    img = Image.fromarray(image).convert('RGB')
    draw = ImageDraw.Draw(img)
    width = max(3, img.width // 300)
    for n, issue in numbered_issues:
        if not issue.get('box'):
            continue
```

```
        x1, y1, x2, y2 = issue['box']
        colour = _COLOURS[issue['severity']]
        draw.rectangle([x1 - 4, y1 - 4, x2 + 4, y2 + 4], outline=colour, width=width)
        tag = f' {n} '
        draw.rectangle([x1 - 4, y1 - 4 - 14 * width // 3, x1 - 4 + 9 * width * len(tag) // 3, y1 - 4], fill=colour)
        draw.text((x1 - 2, y1 - 4 - 13 * width // 3), tag, fill=(255, 255, 255))
    return img


def _png(img: Image.Image, max_width: int = 900) -> str:
    if img.width > max_width:
        img = img.resize((max_width, int(img.height * max_width / img.width)))
    buf = io.BytesIO()
    img.save(buf, format='PNG', optimize=True)
    return '' + base64.b64encode(buf.getvalue()).decode()


def to_html(report: Dict, page_images: Optional[Dict[str, List[np.ndarray]]] = None, title: str = 'Verification report') -> str:
    """
    report: the dict returned by ml.verify.verify(). page_images: {'paper': [...], 'source': [...]} RGB arrays, to
    show where each issue is; issues of the translation ('target') are drawn on the 'paper' pages.
    """
    issues = sorted(report['issues'], key=lambda i: (SEVERITY_ORDER[i['severity']], i.get('question') or 0))
    numbered = list(enumerate(issues, 1))
    s = report['summary']
    verdict = ('Ready for printing: no problems found' if s['errors'] == 0 and s['warnings'] == 0 else
               'Check before printing' if s['errors'] == 0 else 'Do not print: errors found')
    colour = '#1B7F3B' if s['errors'] == 0 and s['warnings'] == 0 else '#B26A00' if s['errors'] == 0 else '#B3261E'
    rows = ''.join(
        f'<tr><td>{n}</td><td><span class="sev {i["severity"]}">{i["severity"]}</span></td>'
        f'<td>{html.escape(i["code"].replace("_", " ").lower())}</td><td>{i.get("question") or ""}</td>'
        f'<td>{"" if i.get("page") is None else i["page"] + 1}</td><td>{html.escape(i["version"])}</td>'
        f'<td>{html.escape(i["message"])}</td></tr>' for n, i in numbered)
    pages_html = ''
    for version, images in (page_images or {}).items():
        for p, image in enumerate(images):
            on_page = [(n, i) for n, i in numbered if i.get('page') == p and
                       (i['version'] == version or (version == 'paper' and i['version'] in ('paper', 'target')))]
            pages_html += (f'<figure><img src="{_png(annotate(image, on_page))}"><figcaption>{html.escape(version)} '
                           f'page {p + 1} &middot; {len(on_page)} issue(s) marked</figcaption></figure>')
    meta = ''.join(f'<span><b>{html.escape(k)}</b> {html.escape(str(v))}</span>' for k, v in report.get('meta', {}).items())
    return f'''<!doctype html><html lang="en"><head><meta charset="utf-8"><title>{html.escape(title)}</title>
<style>
body {{ font-family: Inter, -apple-system, 'Segoe UI', 'Noto Sans', sans-serif; color: #1E293B; margin: 28px; }}
h1 {{ font-size: 22px; margin: 0 0 4px; }} .meta span {{ margin-right: 18px; font-size: 13px; color: #475569; }}
.verdict {{ margin: 16px 0; padding: 14px 18px; border-radius: 10px; color: #fff; background: {colour}; font-size: 18px; font-weight: 700; }}
.kpis {{ display: flex; gap: 12px; margin-bottom: 18px; }} .kpis div {{ flex: 1; border: 1px solid #E2E8F0; border-radius: 10px; padding: 10px 14px; }}
.kpis b {{ display: block; font-size: 24px; }} table {{ border-collapse: collapse; width: 100%; font-size: 13px; }}
th {{ text-align: left; background: #14284B; color: #fff; padding: 7px; }} td {{ border-bottom: 1px solid #E2E8F0; padding: 6px 7px; vertical-align: top; }}
.sev {{ padding: 2px 8px; border-radius: 10px; color: #fff; font-size: 11px; text-transform: uppercase; font-weight: 700; }}
.sev.error {{ background: #B3261E; }} .sev.warning {{ background: #B26A00; }} .sev.info {{ background: #2F5FAF; }}
figure {{ margin: 18px 0; }} figure img {{ max-width: 100%; border: 1px solid #CBD5E1; }} figcaption {{ font-size: 12px; color: #475569; }}
</style></head><body>
<h1>{html.escape(title)}</h1><div class="meta">{meta}</div>
<div class="verdict">{verdict}</div>
<div class="kpis"><div><b>{s["questions"]}</b>questions read</div><div><b>{s["errors"]}</b>errors</div>
<div><b>{s["warnings"]}</b>warnings</div><div><b>{s["info"]}</b>notes</div></div>
<table><thead><tr><th>#</th><th>Severity</th><th>Check</th><th>Q</th><th>Page</th><th>Version</th><th>Finding</th></tr></thead>
<tbody>{rows or '<tr><td colspan="7">No issues.</td></tr>'}</tbody></table>
{pages_html}
<p style="font-size:12px;color:#64748B;margin-top:24px">IndicDoc Verify checks structure, numbers, marks and repeats. It does
not judge whether a question is correct or well translated in meaning; every finding is for a human vetter to confirm.</p>
</body></html>'''
```

ml/ocr/__init__.py — 415 lines · Tesseract OCR of the detected regions, script detection, the number pass, reading order

```
"""
OCR for detected layout regions, with Tesseract (Smith, "An Overview of the Tesseract OCR Engine", ICDAR 2007).

The page is read ONCE with Tesseract (word boxes in TSV form); each word is then assigned to the detected layout
region that contains its centre. One call per page instead of one per region keeps a 60-region newspaper page fast,
```

```
and Tesseract sees whole lines, which it reads better than tight crops.

Models: tessdata_best for the 12 IndicDLP languages, kept in outputs/models/tessdata (download with
scripts/download_models.py). The page's script is detected by reading one text region with each script's model
and keeping the most confident reading (detect_script_by_probe).
"""
import io
import os
import re
import shutil
import subprocess
from collections import defaultdict
from typing import Dict, List, Optional

import numpy as np
from PIL import Image

from ml.config import CLASS_NAMES, MODEL_DIR

TESSDATA_DIR = MODEL_DIR / 'tessdata'

# Display name -> Tesseract code, for the 12 IndicDLP languages
OCR_LANGUAGES = {'English': 'eng', 'Hindi': 'hin', 'Marathi': 'mar', 'Bengali': 'ben', 'Assamese': 'asm',
                 'Gujarati': 'guj', 'Punjabi': 'pan', 'Kannada': 'kan', 'Malayalam': 'mal', 'Tamil': 'tam',
                 'Telugu': 'tel', 'Odia': 'ori'}

# Script reported by Tesseract OSD -> languages written in it (English is added: Indian documents mix it in)
_SCRIPT_LANGUAGES = {'Devanagari': 'hin+mar', 'Bengali': 'ben+asm', 'Gujarati': 'guj', 'Gurmukhi': 'pan',
                     'Kannada': 'kan', 'Malayalam': 'mal', 'Oriya': 'ori', 'Tamil': 'tam', 'Telugu': 'tel'}

# Regions that are not plain text: OCR of pictures and mathematics only produces junk
_NO_TEXT = {'figure', 'flag', 'formula'}
# Page furniture: kept, but listed after the main content in Markdown
_FURNITURE = {'header', 'footer', 'page-number', 'folio', 'footnote', 'jumpline', 'website-link', 'contact-info'}
_HEADINGS = {'headline': '#', 'chapter-title': '#', 'section-title': '##', 'sub-headline': '##',
             'sub-section-title': '###', 'subsub-headline': '###', 'subsub-section-title': '####'}
# Bullet glyphs Tesseract reads in front of list items ('«' is its usual misreading of a round bullet)
_BULLET = re.compile(r'^[\s«»•·●○◦■□▶▸▹-\-\*—–+]+')
# Start of a numbered item: '1.', '2)', '(a)', 'iv.' ...
_NUMBERED = re.compile(r'^\s*(\(?\d{1,3}[.)]|\([a-zA-Z]{1,4}\)|[a-zA-Z][.)]\s|[ivxIVX]{1,4}[.)])')


def _list_items(text: str) -> List[str]:
    """
    Split an OCR'd list into items: a line starting with a bullet glyph or a number opens a new item, other lines
    continue the previous one (wrapped text). If no line has a marker, each line is its own item.
    """
    lines = [l.strip() for l in text.splitlines() if l.strip()]
    if not any(_BULLET.match(l) or _NUMBERED.match(l) for l in lines):
        return lines
    items: List[str] = []
    for line in lines:
        if _BULLET.match(line) or _NUMBERED.match(line) or not items:
            items.append(_BULLET.sub('', line).strip())
        else:
            items[-1] += ' ' + line
    return [i for i in items if i]
_LISTS = {'ordered-list': '', 'unordered-list': '', 'options': '', 'sub-ordered-list': '  ',
          'sub-unordered-list': '  ', 'subsub-ordered-list': '    ', 'subsub-unordered-list': '    '}


def tesseract_available() -> bool:
    return shutil.which('tesseract') is not None and (TESSDATA_DIR / 'eng.traineddata').exists()


def languages_for_script(script: str) -> str:
    """Tesseract language string for a script name from OSD (unknown scripts fall back to English)."""
    langs = _SCRIPT_LANGUAGES.get(script)
    return f'{langs}+eng' if langs else 'eng'


def _run_tesseract(image: np.ndarray, args: List[str], timeout: int = 180, single_thread: bool = False) -> str:
    buf = io.BytesIO()
    Image.fromarray(image).save(buf, format='PNG')
    env = {**os.environ, 'TESSDATA_PREFIX': str(TESSDATA_DIR)}
    if single_thread:
        # Tesseract builds with OpenMP (e.g. Debian's) oversubscribe the CPU when several run in parallel
        env['OMP_THREAD_LIMIT'] = '1'
    proc = subprocess.run(['tesseract', 'stdin', 'stdout', *args], input=buf.getvalue(),
                          capture_output=True, env=env, timeout=timeout)
    if proc.returncode != 0:
```

```
        raise RuntimeError(f"tesseract failed: {proc.stderr.decode(errors='replace')[:300]}")
    return proc.stdout.decode('utf-8', errors='replace')


def read_words(image: np.ndarray, languages: str) -> List[Dict]:
    """Every word Tesseract finds on the page, with its box (page pixels) and line identifiers."""
    h, w = image.shape[:2]
    # Tesseract reads best around 300 dpi; small renders are upscaled and the boxes scaled back
    scale = 2 if max(h, w) < 1600 else 1
    if scale > 1:
        image = np.array(Image.fromarray(image).resize((w * scale, h * scale), Image.LANCZOS))
    # -c tessedit_create_tsv=1 rather than the 'tsv' config file, which lives outside our tessdata folder
    tsv = _run_tesseract(image, ['-l', languages, '--psm', '3', '-c', 'tessedit_create_tsv=1'])
    rows = tsv.splitlines()
    header = rows[0].split('\t')
    words = []
    for row in rows[1:]:
        cols = row.split('\t')
        if len(cols) != len(header):
            continue
        r = dict(zip(header, cols))
        if r['level'] != '5' or not r['text'].strip():
            continue
        words.append({'text': r['text'].strip(), 'conf': float(r['conf']),
                      'left': int(r['left']) / scale, 'top': int(r['top']) / scale,
                      'width': int(r['width']) / scale, 'height': int(r['height']) / scale,
                      'block_num': int(r['block_num']), 'par_num': int(r['par_num']), 'line_num':
int(r['line_num'])})
    return words


# A token that is a number, possibly with brackets, punctuation and a short unit: '1947', '(1)', '24.', '8A',
'120°C'
_NUMERIC_TOKEN = re.compile(r'^[(\[]?[0-9][0-9.,:/%°\-]*[)\].,:;]?[A-Za-z°/]{0,4}[)\].,;]?$')
NUMBER_PASS_MIN_CONF = 55
_GLUED_MARKER = re.compile(r'^(?:\(?[^\s()0-9]{1,3}\)|\([0-9]\)|[A-Za-z])(?=[0-9])')
_INDIC_DIGIT = re.compile(r'[\u0966-\u096F\u09E6-\u09EF\u0A66-\u0A6F\u0AE6-\u0AEF\u0B66-\u0B6F\u0BE6-\u0BEF'
                          r'\u0C66-\u0C6F\u0CE6-\u0CEF\u0D66-\u0D6F]')
# Indic letters and vowel signs, without the Indian digits and the danda ('।' is how the Hindi model reads '1')
_INDIC_LETTER = re.compile(r'[\u0900-\u0963\u0970-\u09E5\u09F0-\u0A65\u0A70-\u0AE5\u0AF0-\u0B65\u0B70-\u0BE5'
                           r'\u0BF0-\u0C65\u0C70-\u0CE5\u0CF0-\u0D65\u0D70-\u0D7F]')
_LATIN_UNIT = re.compile(r'^[A-Za-z°/%.]{1,4}$')
_BARE_NUMBER = re.compile(r'^[0-9][0-9.,]*$')  # Latin digits only ('\d' also matches Indian digits)
_HAS_DIGIT = re.compile(r'[0-9\u0966-\u096F\u09E6-\u09EF\u0A66-\u0A6F\u0AE6-\u0AEF\u0B66-\u0B6F\u0BE6-\u0BEF'
                        r'\u0C66-\u0C6F\u0CE6-\u0CEF\u0D66-\u0D6F]')


def _overlap(a: Dict, b: Dict) -> float:
    """Intersection over the smaller of two word boxes."""
    ix = min(a['left'] + a['width'], b['left'] + b['width']) - max(a['left'], b['left'])
    iy = min(a['top'] + a['height'], b['top'] + b['height']) - max(a['top'], b['top'])
    if ix <= 0 or iy <= 0:
        return 0.0
    return ix * iy / max(1.0, min(a['width'] * a['height'], b['width'] * b['height']))


def _digits(text: str) -> str:
    return ''.join(ch for ch in text if ch.isascii() and ch.isdigit())


def merge_number_reading(words: List[Dict], latin_words: List[Dict]) -> List[Dict]:
    """
    Fix the numbers of an Indic-language reading with a second reading by the English model.
    Indic Tesseract models misread Latin digits in mixed text (the Hindi model reads '1947' as '947' and '10' as
    '70', and sometimes reads a letter as a digit); the English model reads Latin digits reliably but not Indic
    letters. So for Latin digits the English reading decides:
      - a numeric token of the English reading replaces the overlapping word of the main reading when that word is
        a number with different digits (a bracket the main reading had, as in an option marker, is kept);
      - it is inserted when the main reading has no word there;
      - a bare Latin number of the main reading that no English numeric token confirms is dropped.
    Indic words and Indian-script digits ('१२०') are never touched.
    """
    out = [dict(w) for w in words]
    # An option marker run into the value is not part of the number: '(3)6A' -> '6A', 'M2A' ('(1) 2A') -> '2A'
    latin_words = [{**lw, 'text': _GLUED_MARKER.sub('', lw['text'])} for lw in latin_words]
    numeric_latin = [lw for lw in latin_words if _NUMERIC_TOKEN.match(lw['text'])]
    # Pages printed with Indian digits ('१२०') keep them: the English model cannot read them. On pages printed with
    # Latin digits, an Indian digit in the Indic reading is a misread ('1A' -> '८') and the English reading decides.
    indic_digits = sum(bool(_INDIC_DIGIT.search(w['text'])) for w in out)
    latin_digits = sum(bool(re.search('[0-9]', w['text'])) for w in out)
    indic_page = indic_digits > latin_digits
```

```
    # Any English token with the same digits confirms a number, even when it ran into a marker ('(m30' for '(π) 30')
    with_digits = [lw for lw in latin_words if _digits(lw['text'])]
    for w in out:
        if _BARE_NUMBER.match(w['text']) and not any(_overlap(w, lw) > 0.3 and _digits(w['text']) in
_digits(lw['text'])
                                                     for lw in with_digits):
            w['text'] = '·'
        # Latin units read by the Indic model as digits ('A' -> '८', '°C' -> '९६' or '“6'): a confident short Latin
        # reading without digits at the same place wins
        elif _HAS_DIGIT.search(w['text']) and not _BARE_NUMBER.match(w['text']) \
                and not (indic_page and _INDIC_DIGIT.search(w['text'])):
            near = [lw for lw in latin_words if _overlap(w, lw) > 0.5]
            if len(near) == 1 and near[0].get('conf', 0) >= 80 and _LATIN_UNIT.match(near[0]['text']):
                w['text'] = near[0]['text']
    # Only confident English readings may change the text: where the English model "reads" digits into Indic
    # letters its confidence is low (median 32 on the benchmark's development pages, against 93 for real digits)
    for lw in numeric_latin:
        confident = lw.get('conf', 100) >= NUMBER_PASS_MIN_CONF
        hits = [w for w in out if _overlap(w, lw) > 0.3]
        if not hits:
            if not confident:
                continue
            same_line = [w for w in out if abs((w['top'] + w['height'] / 2) - (lw['top'] + lw['height'] / 2))
                         < 0.6 * max(w['height'], lw['height'])]
            ids = min(same_line, key=lambda w: abs(w['left'] - lw['left'])) if same_line else None
            if ids is not None:
                out.append({**lw, 'block_num': ids['block_num'], 'par_num': ids['par_num'], 'line_num':
ids['line_num']})
                continue
        w = hits[0]
        # A word of the Indic reading without Indic letters (a number, 'ו' for '1', a stray symbol) takes the
        # English number; a word with Indic letters only when the English reading is confident or more confident
        letters = bool(_INDIC_LETTER.search(w['text']))
        if indic_page and _INDIC_DIGIT.search(w['text']):
            continue
        if len(hits) == 1 and (_HAS_DIGIT.search(w['text']) or not letters or len(w['text']) <= 2) \
                and _digits(w['text']) != _digits(lw['text']) \
                and (not letters or confident or lw.get('conf', 0) > w.get('conf', 0)):
            # '(m2' (marker and number run together) + English '12' -> '(?) 12': the option marker survives
            w['text'] = ('(?) ' + lw['text']) if w['text'].startswith('(') and not lw['text'].startswith('(') \
                else lw['text']
    return out


def assign_words(regions: List[Dict], words: List[Dict], min_conf: float = 0.0,
                 include_words: bool = False) -> List[Dict]:
    """
    Put each word into the smallest region containing its centre and build that region's text, line by line.
    Returns copies of the regions with a 'text' field, in the same order (and their 'words' with boxes when
    include_words is set: verification uses the word positions to separate the options of a question).
    """
    per_region = defaultdict(list)
    for word in words:
        if not word['text'].strip() or word['conf'] < min_conf:
            continue
        cx, cy = word['left'] + word['width'] / 2, word['top'] + word['height'] / 2
        containing = [i for i, r in enumerate(regions)
                      if r['box'][0] <= cx <= r['box'][2] and r['box'][1] <= cy <= r['box'][3]]
        if containing:
            area = lambda i: (regions[i]['box'][2] - regions[i]['box'][0]) * (regions[i]['box'][3] - regions[i]
['box'][1])
            per_region[min(containing, key=area)].append(word)

    out = []
    for i, region in enumerate(regions):
        lines = defaultdict(list)
        for word in per_region[i]:
            lines[(word['block_num'], word['par_num'], word['line_num'])].append(word)
        ordered = sorted(lines.values(), key=lambda ws: np.mean([w['top'] for w in ws]))
        text = '\n'.join(' '.join(w['text'] for w in sorted(ws, key=lambda w: w['left'])) for ws in ordered)
        out.append({**region, 'text': text})
        if include_words:
            out[-1]['words'] = [{k: w[k] for k in ('text', 'left', 'top', 'width', 'height')} for w in
per_region[i]]
    return out


def _contains(region: Dict, word: Dict) -> bool:
    cx, cy = word['left'] + word['width'] / 2, word['top'] + word['height'] / 2
    return region['box'][0] <= cx <= region['box'][2] and region['box'][1] <= cy <= region['box'][3]
```

```python

def leftover_regions(regions: List[Dict], words: List[Dict], min_conf: float = 0.0) -> List[Dict]:
    """
    Text the layout model put in no box at all, grouped by Tesseract paragraph, as 'unlabelled-text' regions.
    Keeps the extracted document complete when the detector misses an element (e.g. a heading).
    """
    groups = defaultdict(list)
    for word in words:
        if word['text'].strip() and word['conf'] >= min_conf and not any(_contains(r, word) for r in regions):
            groups[(word['block_num'], word['par_num'])].append(word)
    out = []
    for ws in groups.values():
        lines = defaultdict(list)
        for w in ws:
            lines[w['line_num']].append(w)
        ordered = sorted(lines.values(), key=lambda line: np.mean([w['top'] for w in line]))
        text = '\n'.join(' '.join(w['text'] for w in sorted(line, key=lambda w: w['left'])) for line in ordered)
        box = [min(w['left'] for w in ws), min(w['top'] for w in ws),
               max(w['left'] + w['width'] for w in ws), max(w['top'] + w['height'] for w in ws)]
        out.append({'label': 'unlabelled-text', 'box': [round(float(v), 1) for v in box], 'score': None,
                    'text': text, 'detected': False})
    return sorted(out, key=lambda r: r['box'][1])


def reading_order(regions: List[Dict]) -> List[Dict]:
    """Top to bottom; regions that overlap vertically by at least half form a row, read left to right."""
    rows: List[List[Dict]] = []
    for region in sorted(regions, key=lambda r: r['box'][1]):
        y1, y2 = region['box'][1], region['box'][3]
        if rows:
            row_top = min(r['box'][1] for r in rows[-1])
            row_bottom = min(r['box'][3] for r in rows[-1])
            overlap = min(y2, row_bottom) - max(y1, row_top)
            if overlap >= 0.5 * min(y2 - y1, row_bottom - row_top):
                rows[-1].append(region)
                continue
        rows.append([region])
    return [r for row in rows for r in sorted(row, key=lambda r: r['box'][0])]


def to_markdown(regions: List[Dict]) -> str:
    """Regions (already in reading order) as Markdown: headings, lists, tables, captions; page furniture last."""
    body, furniture = [], []
    for r in regions:
        label, text = r['label'], r.get('text', '').strip()
        if label in _NO_TEXT:
            body.append(f'[{label}]')
            continue
        if not text:
            continue
        if label in _FURNITURE:
            furniture.append(f"- {label}: {' '.join(text.split())}")
        elif label in _HEADINGS:
            body.append(f"{_HEADINGS[label]} {' '.join(text.split())}")
        elif label in _LISTS:
            body.append('\n'.join(f'{_LISTS[label]}- {item}' for item in _list_items(text)))
        elif label == 'table':
            body.append(f'```\n{text}\n```')
        elif label in ('figure-caption', 'table-caption'):
            body.append(f'*{" ".join(text.split())}*')
        elif label == 'quote':
            body.append('\n'.join(f'> {line}' for line in text.splitlines()))
        else:
            body.append(text)
    md = '\n\n'.join(body)
    if furniture:
        md += '\n\n---\n\n' + '\n'.join(furniture)
    return md + '\n'


def extract_document(image: np.ndarray, detections: Dict, languages: Optional[str] = 'auto',
                     number_pass: bool = False) -> Dict:
    """
    OCR a page given its layout detections (0-indexed labels, boxes as x1, y1, x2, y2 in page pixels).
    languages: a Tesseract language string such as 'hin+eng', or 'auto' to detect the script first.
    number_pass: read the page a second time with the English model and use its digits (merge_number_reading);
    verification needs every number exact, so it turns this on for Indic-language pages.
    """
    script = None
    if languages in (None, 'auto'):
        script = detect_script_by_probe(image, detections)
```

```
        languages = languages_for_script(script)
    if number_pass and languages != 'eng':
        # The Indic model alone reads Indic words best (with '+eng' Tesseract sometimes swaps a Hindi word for an
        # English guess: 'पुणीक' -> 'guia'); the English number pass then supplies the Latin digits
        indic = '+'.join(code for code in languages.split('+') if code != 'eng')
        words = merge_number_reading(read_words(image, indic), read_words(image, 'eng'))
    else:
        words = read_words(image, languages)
    regions = [{'label': CLASS_NAMES[int(l)], 'box': [round(float(v), 1) for v in box], 'score': round(float(s), 3)}
               for box, l, s in zip(detections['boxes'], detections['labels'], detections['scores'])]
    regions = [{**r, 'detected': True} if r['label'] not in _NO_TEXT else {**r, 'text': '', 'detected': True}
               for r in assign_words(regions, words, include_words=number_pass)]
    # Words in no detected box are kept as 'unlabelled-text' regions, so no text is silently lost
    regions += leftover_regions(regions, words)
    ordered = reading_order(regions)
    assigned = sum(len(r['text'].split()) for r in ordered if r['detected'])
    return {'script': script, 'languages': languages, 'regions': ordered, 'markdown': to_markdown(ordered),
            'words_on_page': len(words), 'words_in_regions': assigned}


# One representative Tesseract model per script for the language probe (Marathi shares Devanagari with Hindi,
# Assamese shares the Bengali script)
_PROBE_LANGUAGES = {'Latin': 'eng', 'Devanagari': 'hin', 'Bengali': 'ben', 'Gujarati': 'guj', 'Gurmukhi': 'pan',
                    'Kannada': 'kan', 'Malayalam': 'mal', 'Oriya': 'ori', 'Tamil': 'tam', 'Telugu': 'tel'}
# Unicode ranges of each script, to count only the characters a model reads in ITS OWN script
# (the Indic models also contain Latin letters and read English text confidently)
_SCRIPT_RANGES = {'Latin': [(0x41, 0x5A), (0x61, 0x7A)], 'Devanagari': [(0x0900, 0x097F)],
                  'Bengali': [(0x0980, 0x09FF)], 'Gurmukhi': [(0x0A00, 0x0A7F)], 'Gujarati': [(0x0A80, 0x0AFF)],
                  'Oriya': [(0x0B00, 0x0B7F)], 'Tamil': [(0x0B80, 0x0BFF)], 'Telugu': [(0x0C00, 0x0C7F)],
                  'Kannada': [(0x0C80, 0x0CFF)], 'Malayalam': [(0x0D00, 0x0D7F)]}


def _in_script(ch: str, script: str) -> bool:
    return any(lo <= ord(ch) <= hi for lo, hi in _SCRIPT_RANGES[script])


_TEXT_LABELS_FOR_PROBE = {'paragraph', 'ordered-list', 'unordered-list', 'answer', 'first-level-question',
                          'second-level-question', 'options', 'sidebar', 'quote'}


def _probe_crop(image: np.ndarray, detections: Optional[Dict]) -> np.ndarray:
    """The largest detected body-text region (most text, least noise); the page centre if there is none."""
    h, w = image.shape[:2]
    best = None
    if detections is not None:
        for box, label in zip(detections['boxes'], detections['labels']):
            if CLASS_NAMES[int(label)] in _TEXT_LABELS_FOR_PROBE:
                x1, y1, x2, y2 = (int(v) for v in box)
                if best is None or (x2 - x1) * (y2 - y1) > (best[2] - best[0]) * (best[3] - best[1]):
                    best = (max(0, x1), max(0, y1), min(w, x2), min(h, y2))
    if best is None or (best[2] - best[0]) < 40 or (best[3] - best[1]) < 20:
        best = (w // 6, h // 4, 5 * w // 6, 3 * h // 4)
    x1, y1, x2, y2 = best
    crop = image[y1:y2, x1:x2]
    if crop.shape[0] > 700:  # enough text to judge; keeps the probe fast
        crop = crop[:700]
    return crop


def detect_script_by_probe(image: np.ndarray, detections: Optional[Dict] = None) -> str:
    """
    Script of the page: read one text region with each script's model and keep the most confident reading.
    A model shown the wrong script produces few, low-confidence words in its own script, so the mean
    confidence of words containing that script's characters, weighted by how many such characters were
    read, separates the scripts well. Accuracy on IndicDLP test pages is measured by scripts/evaluate_ocr.py (during
    development Tesseract's own orientation-and-script detector got only 70 of 120 pages right).
    """
    from concurrent.futures import ThreadPoolExecutor
    crop = _probe_crop(image, detections)
    if crop.shape[0] < 600:
        crop = np.array(Image.fromarray(crop).resize((crop.shape[1] * 2, crop.shape[0] * 2), Image.LANCZOS))

    def score(item):
        script, lang = item
        try:
            tsv = _run_tesseract(crop, ['-l', lang, '--psm', '6', '-c', 'tessedit_create_tsv=1'], timeout=120,
                                 single_thread=True)
        except (RuntimeError, subprocess.TimeoutExpired):
            return script, 0.0
        confs, chars = [], 0
        for row in tsv.splitlines()[1:]:
```

```
            cols = row.split('\t')
            if len(cols) == 12 and cols[0] == '5' and cols[11].strip() and float(cols[10]) >= 0:
                own = sum(_in_script(ch, script) for ch in cols[11])
                if own:
                    confs.append(float(cols[10]))
                    chars += own
        return script, (np.mean(confs) if confs else 0.0) * np.log1p(chars)

    with ThreadPoolExecutor(max_workers=len(_PROBE_LANGUAGES)) as pool:
        scores = dict(pool.map(score, _PROBE_LANGUAGES.items()))
    return max(scores, key=scores.get)
```

ml/inference/predict.py — 152 lines · Loads a layout detector (DocLayout-YOLO, YOLOv8, Faster R-CNN) and predicts regions

```python
import torch
import time
import numpy as np
from PIL import Image
from typing import Dict, Any, List
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import matplotlib.patches as patches
import os

def disable_runtime_installs() -> None:
    """
    Stop Ultralytics from pip-installing packages while serving. It patches PIL.Image.open so that any image that
    fails to open triggers `pip install pi-heif` from the internet, which a bad upload would set off in production.
    The flag is read when Ultralytics is imported, so it is also switched off on an already-imported Ultralytics.
    """
    import os
    import sys
    os.environ["YOLO_AUTOINSTALL"] = "False"
    for name in ("ultralytics.utils", "ultralytics.utils.checks", "doclayout_yolo.utils",
"doclayout_yolo.utils.checks"):
        if name in sys.modules:
            sys.modules[name].AUTOINSTALL = False

def load_model(model_path: str, model_type: str, device: str = 'cpu') -> Any:
    """Load Faster R-CNN or YOLOv8 model."""
    if model_type.lower() == 'fasterrcnn':
        # Training scripts save a state_dict, so rebuild the architecture and load the weights into it
        from ml.config import NUM_CLASSES
        from ml.models.baseline_frcnn import get_baseline_model
        model = get_baseline_model(NUM_CLASSES + 1, pretrained=False)
        model.load_state_dict(torch.load(model_path, map_location=device, weights_only=True))
        model.to(device)
        model.eval()
        return model
    elif model_type.lower() == 'yolov8':
        from ultralytics import YOLO
        model = YOLO(model_path)
        model.to(device)
        return model
    elif model_type.lower() == 'doclayout':
        # DocLayout-YOLO (a YOLOv10 fork); the IndicDLP authors' checkpoint uses the same 42 labels in the same
order
        from doclayout_yolo import YOLOv10
        model = YOLOv10(model_path)
        model.to(device)
        return model
    else:
        raise ValueError("Unsupported model type")

def predict_single(model: Any, image: Any, model_type: str, device: str = 'cpu', conf: float = None) -> Dict:
    """
    Predict on a single image. Labels are 0-indexed class ids (index into CLASS_NAMES) for both models.
    conf: optional minimum score. Evaluation passes a very low value, since mAP needs low-confidence detections too.
    """
    if model_type.lower() == 'fasterrcnn':
        if isinstance(image, Image.Image):
            import torchvision.transforms as T
            image = T.ToTensor()(image)
        image = image.to(device)
        with torch.no_grad():
            output = model([image])[0]
        keep = output['scores'] >= conf if conf is not None else slice(None)
        return {
```

```
            'boxes': output['boxes'][keep].cpu().numpy(),
            # Faster R-CNN reserves 0 for background, so shift back to 0-indexed class ids
            'labels': output['labels'][keep].cpu().numpy() - 1,
            'scores': output['scores'][keep].cpu().numpy()
        }
    elif model_type.lower() in ('yolov8', 'doclayout'):
        kwargs = {'conf': conf} if conf is not None else {}
        if model_type.lower() == 'doclayout':
            from ml.config import AUTHORS_IMG_SIZE
            kwargs['imgsz'] = AUTHORS_IMG_SIZE
        results = model.predict(image, verbose=False, device=device, **kwargs)[0]
        boxes = results.boxes
        return {
            'boxes': boxes.xyxy.cpu().numpy(),
            'labels': boxes.cls.cpu().numpy(),
            'scores': boxes.conf.cpu().numpy()
        }
    return {}

def demo_inference(image: np.ndarray) -> Dict:
    """
    Placeholder detections for when no trained checkpoint exists. The boxes are random and
    flagged with is_demo=True so every UI can label them as synthetic, not model output.
    """
    h, w = image.shape[:2]
    rng = np.random.RandomState(42)  # local RNG: same boxes every call, no global side effects
    n = rng.randint(4, 9)
    boxes, labels, scores = [], [], []
    for _ in range(n):
        bw = rng.randint(w // 6, w // 2)
        bh = rng.randint(h // 8, h // 3)
        x1 = rng.randint(0, max(1, w - bw))
        y1 = rng.randint(0, max(1, h - bh))
        boxes.append([x1, y1, x1 + bw, y1 + bh])
        labels.append(rng.randint(0, 10))
        scores.append(round(rng.uniform(0.55, 0.97), 2))
    return {
        'boxes': np.array(boxes), 'labels': np.array(labels),
        'scores': np.array(scores), 'inference_time': 0.0, 'is_demo': True
    }

def predict_batch(model: Any, images: List, model_type: str, device: str = 'cpu') -> List[Dict]:
    """Predict on a batch of images."""
    return [predict_single(model, img, model_type, device) for img in images]

def measure_inference_time(model: Any, image: Any, model_type: str, num_runs: int = 10, warmup: int = 3, device: str
= 'cpu') -> float:
    """Measure average inference time."""
    for _ in range(warmup):
        predict_single(model, image, model_type, device)

    start_time = time.time()
    for _ in range(num_runs):
        predict_single(model, image, model_type, device)
    end_time = time.time()

    return (end_time - start_time) / num_runs

def save_annotated_image(image: Any, predictions: Dict, save_path: str) -> None:
    """Save image with drawn predictions."""
    if isinstance(image, torch.Tensor):
        image = image.permute(1, 2, 0).cpu().numpy()
    elif isinstance(image, Image.Image):
        image = np.array(image)

    fig, ax = plt.subplots(1)
    ax.imshow(image)

    for box, label, score in zip(predictions.get('boxes', []), predictions.get('labels', []),
predictions.get('scores', [])):
        rect = patches.Rectangle((box[0], box[1]), box[2] - box[0], box[3] - box[1], linewidth=2, edgecolor='r',
facecolor='none')
        ax.add_patch(rect)
        ax.text(box[0], box[1] - 5, f"cls {int(label)}: {score:.2f}", color='red', fontsize=8)

    os.makedirs(os.path.dirname(save_path), exist_ok=True)
    plt.savefig(save_path)
    plt.close(fig)

def dedupe_boxes(results: Dict, iou_threshold: float = 0.7) -> Dict:
    """
```

```
    Class-agnostic duplicate removal: when boxes overlap at IoU > iou_threshold, keep only the most confident one.
    DocLayout-YOLO scores every label separately, so at low thresholds one heading can come back as section-title,
    sub-section-title and subsub-section-title; a page element should be shown, and read, once.
    """
    import torchvision
    boxes = np.asarray(results['boxes'], dtype=np.float32).reshape(-1, 4)
    if len(boxes) == 0:
        return {**results, 'boxes': boxes, 'labels': np.asarray(results['labels']), 'scores':
np.asarray(results['scores'])}
    scores = np.asarray(results['scores'], dtype=np.float32)
    keep = torchvision.ops.nms(torch.from_numpy(boxes), torch.from_numpy(scores), iou_threshold).numpy()
    return {**results, 'boxes': boxes[keep], 'labels': np.asarray(results['labels'])[keep], 'scores': scores[keep]}
```

ml/models/proposed_yolov8.py — 68 lines · YOLOv8s model builder and COCO-to-YOLO conversion

```python
import os
import json
from pathlib import Path
try:
    from ultralytics import YOLO
except ImportError:
    YOLO = None

def get_proposed_model(model_size: str = 'n', weights_dir: str = None):
    """
    Initialize YOLOv8 model for fine-tuning.
    Args:
        model_size (str): 'n', 's', 'm', 'l', or 'x' for YOLOv8 model sizes.
        weights_dir (str, optional): Folder to download/cache the COCO-pretrained weights in.
            Without it, ultralytics downloads into the current working directory.
    """
    if YOLO is None:
        raise ImportError("ultralytics is not installed. Run: pip install ultralytics")

    # Load a pretrained YOLOv8 model
    model_name = f'yolov8{model_size}.pt'
    if weights_dir:
        Path(weights_dir).mkdir(parents=True, exist_ok=True)
        model_name = str(Path(weights_dir) / model_name)
    model = YOLO(model_name)
    return model

def coco_to_yolo(coco_json_path: str, output_dir: str):
    """
    Converts COCO format annotations to YOLO format txt files.
    """
    with open(coco_json_path, 'r', encoding='utf-8') as f:
        coco = json.load(f)

    output_dir = Path(output_dir)
    output_dir.mkdir(parents=True, exist_ok=True)

    # Mapping image id to file
    img_dict = {img['id']: img for img in coco['images']}

    # Collect every label line first, then write each file once with 'w',
    # so re-running the conversion overwrites instead of appending duplicates
    lines_per_file = {}

    for ann in coco['annotations']:
        img_id = ann['image_id']
        cat_id = ann['category_id'] # Note: YOLO classes are 0-indexed. Adjust if necessary.
        img_info = img_dict[img_id]

        img_w = img_info['width']
        img_h = img_info['height']

        # COCO: [x_min, y_min, width, height]
        x_min, y_min, w, h = ann['bbox']

        # YOLO: [x_center, y_center, width, height] (normalized)
        x_center = (x_min + w / 2) / img_w
        y_center = (y_min + h / 2) / img_h
        norm_w = w / img_w
        norm_h = h / img_h

        txt_filename = Path(img_info['file_name']).stem + '.txt'
        lines_per_file.setdefault(txt_filename, []).append(
            f"{cat_id} {x_center} {y_center} {norm_w} {norm_h}\n")

    for txt_filename, lines in lines_per_file.items():
```

```
        with open(output_dir / txt_filename, 'w', encoding='utf-8') as txt_f:
            txt_f.writelines(lines)
```

ml/training/train_yolo.py — 88 lines · Fine-tuning YOLOv8s on the IndicDLP subset

```python
import argparse
import shutil
import yaml
from pathlib import Path

import sys
sys.path.append(str(Path(__file__).parent.parent.parent))

from ml.config import *
from ml.models.proposed_yolov8 import get_proposed_model, coco_to_yolo

def create_yolo_yaml(train_img_dir: str, val_img_dir: str, num_classes: int, class_names: list, output_path: str):
    # Absolute paths: ultralytics resolves relative paths against the YAML's own folder, not the working directory
    data = {
        'train': str(Path(train_img_dir).resolve()),
        'val': str(Path(val_img_dir).resolve()),
        'nc': num_classes,
        'names': class_names
    }
    with open(output_path, 'w') as f:
        yaml.dump(data, f, sort_keys=False)

def main(args) -> Path:
    project = Path(args.project)
    project.mkdir(parents=True, exist_ok=True)
    output = Path(args.output)
    output.parent.mkdir(parents=True, exist_ok=True)

    # 1. Convert COCO to YOLO if needed. Ultralytics finds labels by swapping 'images' for 'labels' in each image path
    if args.convert_annotations:
        print("Converting COCO annotations to YOLO format...")
        coco_to_yolo(args.train_ann, str(Path(args.train_img).parent / 'labels'))
        coco_to_yolo(args.val_ann, str(Path(args.val_img).parent / 'labels'))

    # 2. Create YAML
    yaml_path = project / 'yolo_dataset.yaml'
    create_yolo_yaml(args.train_img, args.val_img, NUM_CLASSES, CLASS_NAMES, str(yaml_path))

    # 3. Train
    print("Initializing YOLO model...")
    model = get_proposed_model(model_size=args.model_size, weights_dir=args.weights_dir)

    print("Starting training...")
    model.train(
        data=str(yaml_path),
        epochs=args.epochs,
        imgsz=args.imgsz,
        batch=args.batch,
        device=args.device,
        project=str(project),
        name='indicdoc_yolo',
        exist_ok=True,
        seed=SEED,
        workers=NUM_WORKERS,
        fliplr=0.0,  # no mirror flips: a mirrored page is not a realistic document
        **({'time': args.hours} if getattr(args, 'hours', None) else {}),  # ultralytics fits the schedule to this budget
    )

    # 4. Copy the best weights to the path the app and backend load from
    best = Path(model.trainer.save_dir) / 'weights' / 'best.pt'
    shutil.copy(best, output)
    print(f"Training complete. Best weights copied to {output}")
    history = getattr(args, 'history', None)
    if history:  # ultralytics logs per-epoch losses and val mAP in results.csv
        Path(history).parent.mkdir(parents=True, exist_ok=True)
        shutil.copy(Path(model.trainer.save_dir) / 'results.csv', history)
    return output

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument('--train_ann', type=str, required=True, help="Path to train annotations JSON")
    parser.add_argument('--train_img', type=str, required=True, help="Path to train images directory")
    parser.add_argument('--val_ann', type=str, required=True, help="Path to val annotations JSON")
    parser.add_argument('--val_img', type=str, required=True, help="Path to val images directory")
```

```
    parser.add_argument('--convert_annotations', action='store_true', help="Convert COCO to YOLO before training")
    parser.add_argument('--epochs', type=int, default=NUM_EPOCHS)
    parser.add_argument('--batch', type=int, default=BATCH_SIZE)
    parser.add_argument('--imgsz', type=int, default=IMG_SIZE)
    parser.add_argument('--model_size', type=str, default='s', choices=['n', 's', 'm', 'l', 'x'])
    parser.add_argument('--device', type=str, default=get_device())
    parser.add_argument('--weights_dir', type=str, default=str(MODEL_DIR), help="Where pretrained COCO weights are cached")
    parser.add_argument('--project', type=str, default=str(OUTPUT_DIR / 'yolo_runs'))
    parser.add_argument('--output', type=str, default=str(PROPOSED_CHECKPOINT), help="Where to copy the best weights")
    parser.add_argument('--hours', type=float, default=None, help="Time budget in hours (overrides --epochs)")
    parser.add_argument('--history', type=str, default=str(METRICS_DIR / 'proposed_training_history.csv'),
                        help="Where to copy ultralytics' per-epoch results.csv")
    args = parser.parse_args()
    main(args)
```

ml/models/baseline_frcnn.py — 35 lines · Faster R-CNN (ResNet-50 FPN) baseline builder

```python
import torchvision
from torchvision.models.detection import FasterRCNN
from torchvision.models.detection.backbone_utils import _resnet_fpn_extractor
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor
from torchvision.ops import misc as misc_nn_ops
from ml.config import IMG_SIZE

def get_baseline_model(num_classes: int, pretrained: bool = True):
    """
    Returns a Faster R-CNN model modified for the specified number of classes.
    Note: num_classes should include the background class (e.g., 42 IndicDLP classes + 1 bg = 43).

    pretrained=True starts from COCO weights (for training). pretrained=False builds the same architecture
    without fetching any weights, for loading a trained checkpoint (works offline).
    """
    # min_size/max_size: torchvision's 800/1333 default upscales our 640px pages ~1.8x internally;
    # working near the training resolution keeps a CPU/laptop-GPU training budget realistic
    size = dict(min_size=IMG_SIZE, max_size=1024)
    if pretrained:
        # Load a model pre-trained on COCO
        model = torchvision.models.detection.fasterrcnn_resnet50_fpn(weights='DEFAULT', **size)
    else:
        # fasterrcnn_resnet50_fpn(weights=None) would still download ImageNet backbone weights, and dropping
        # them would switch to plain BatchNorm and change the state_dict. Build the trained layout directly:
        # ResNet-50 with frozen batch norm + FPN (3 trainable stages, as when training from COCO weights)
        backbone = torchvision.models.resnet50(weights=None, norm_layer=misc_nn_ops.FrozenBatchNorm2d)
        model = FasterRCNN(_resnet_fpn_extractor(backbone, 3), num_classes=91, **size)

    # Get number of input features for the classifier
    in_features = model.roi_heads.box_predictor.cls_score.in_features

    # Replace the pre-trained head with a new one
    model.roi_heads.box_predictor = FastRCNNPredictor(in_features, num_classes)

    return model
```

backend/main.py — 274 lines · FastAPI REST API: /analyze (layout + OCR) and /verify (pre-print check)

```python
import json
import os
import sys
import threading
import time
from contextlib import asynccontextmanager
from functools import lru_cache
from pathlib import Path
from typing import Literal, Optional

import cv2
import numpy as np
from fastapi import FastAPI, File, HTTPException, Query, UploadFile
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import HTMLResponse
from PIL import Image
from starlette.concurrency import run_in_threadpool

# Allow importing the ml package from the project root
PROJECT_ROOT = Path(__file__).resolve().parent.parent
sys.path.append(str(PROJECT_ROOT))

from ml.config import AUTHORS_CHECKPOINT, BASELINE_CHECKPOINT, CLASS_NAMES, METRICS_DIR, PROPOSED_CHECKPOINT,
```

```
get_device
from ml.inference.predict import dedupe_boxes, demo_inference, load_model, predict_single
from ml.preprocessing.preprocess import decode_image, pdf_page_count, render_pdf_page
from ml.ocr import OCR_LANGUAGES, extract_document, tesseract_available
from ml.verify import list_profiles, verify
from ml.verify.report import to_html as verify_report_html
from ml.inference.predict import disable_runtime_installs

disable_runtime_installs()  # serving must never pip-install packages (see the function's docstring)

CHECKPOINTS = {'doclayout': AUTHORS_CHECKPOINT, 'yolov8': PROPOSED_CHECKPOINT, 'fasterrcnn': BASELINE_CHECKPOINT}
# Saved evaluation results of each model, written by scripts/evaluate.py
METRICS_NAMES = {'doclayout': 'authors_metrics', 'yolov8': 'proposed_metrics', 'fasterrcnn': 'baseline_metrics'}

# Deployment settings, overridable through environment variables
MAX_UPLOAD_BYTES = int(float(os.environ.get('INDICDOC_MAX_UPLOAD_MB', '20')) * 1024 * 1024)
MAX_PIXELS = int(float(os.environ.get('INDICDOC_MAX_MEGAPIXELS', '40')) * 1_000_000)
# One lock per model: Ultralytics predictors are not safe to share between threads
MODEL_LOCKS = {name: threading.Lock() for name in CHECKPOINTS}
OCR_CODES = set(OCR_LANGUAGES.values())
MAX_VERIFY_PAGES = int(os.environ.get('INDICDOC_MAX_VERIFY_PAGES', '60'))
# A whole scanned paper is much larger than one page, so /verify has its own upload limit
MAX_VERIFY_BYTES = int(float(os.environ.get('INDICDOC_MAX_VERIFY_MB', '100')) * 1024 * 1024)
VERIFY_DPI = 200  # every digit must be read exactly, so papers are rendered finer than for layout analysis
SCRIPTS = {'Latin', 'Devanagari', 'Bengali', 'Gurmukhi', 'Gujarati', 'Oriya', 'Tamil', 'Telugu', 'Kannada',
'Malayalam'}
CORS_ORIGINS = [o.strip() for o in os.environ.get('INDICDOC_CORS_ORIGINS', 'http://localhost:5173').split(',') if
o.strip()]



@asynccontextmanager
async def lifespan(app: FastAPI):
    # Load the available models before serving, so the first request does not wait ~20 s for a 166 MB checkpoint
    for model_type, path in CHECKPOINTS.items():
        if path.exists():
            await run_in_threadpool(get_model, model_type)
    yield


app = FastAPI(title="IndicDoc AI API", version="1.0.0", lifespan=lifespan)

app.add_middleware(
    CORSMiddleware,
    allow_origins=CORS_ORIGINS,  # set INDICDOC_CORS_ORIGINS to the sites allowed to call this API
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)


@lru_cache(maxsize=2)
def get_model(model_type: str):
    """Load each model once per process instead of on every request."""
    device = get_device()
    model = load_model(str(CHECKPOINTS[model_type]), model_type, device)
    disable_runtime_installs()  # again, now that ultralytics may have been imported
    return model, device


@app.get("/")
def read_root():
    return {"message": "Welcome to IndicDoc AI API"}


@app.post("/analyze")
async def analyze_document(
    file: UploadFile = File(...),
    model_type: Literal['doclayout', 'yolov8', 'fasterrcnn'] = 'doclayout',
    conf_threshold: Optional[float] = Query(None, ge=0.0, le=1.0,
                                            description="Defaults to the model's recommended threshold"),
    mode: Literal['balanced', 'all'] = Query('balanced', description="'all' = the model's high-recall threshold"),
    ocr: bool = Query(False, description="Also read the text of every detected region (Tesseract)"),
    ocr_lang: str = Query('auto', pattern=r'^(auto|[a-z]{3}(\+[a-z]{3})*)$',
                          description="'auto' or Tesseract codes joined by '+', e.g. 'hin+eng'"),
):
    # Read at most one byte past the limit, so an oversized upload is never loaded in full
    contents = await file.read(MAX_UPLOAD_BYTES + 1)
    if len(contents) > MAX_UPLOAD_BYTES:
        raise HTTPException(status_code=413, detail=f"File larger than {MAX_UPLOAD_BYTES // (1024 * 1024)} MB")
    try:
```

```
        image = decode_image(contents, max_pixels=MAX_PIXELS)
    except ValueError as e:
        too_large = 'too large' in str(e)
        raise HTTPException(status_code=413 if too_large else 400, detail=str(e) if too_large else "Invalid image
file")
    if ocr:
        if ocr_lang != 'auto' and not set(ocr_lang.split('+')) <= OCR_CODES:
            raise HTTPException(status_code=400, detail=f"ocr_lang must be 'auto' or codes from
{sorted(OCR_CODES)}")
        if not tesseract_available():
            raise HTTPException(status_code=503, detail="OCR is not available on this server")
    image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    if conf_threshold is None:
        conf_threshold = recommended_conf(model_type, mode)

    if CHECKPOINTS[model_type].exists():
        def infer():
            with MODEL_LOCKS[model_type]:
                model, device = get_model(model_type)
                start = time.time()
                # PIL keeps channel order unambiguous for both torchvision and ultralytics
                out = predict_single(model, Image.fromarray(image_rgb), model_type, device, conf=conf_threshold)
                out['inference_time'] = time.time() - start
                return out

        # Run in a worker thread so a slow page does not block other requests (health checks, metrics)
        results = await run_in_threadpool(infer)
        keep = results['scores'] >= conf_threshold
        for key in ('boxes', 'labels', 'scores'):
            results[key] = results[key][keep]
        results = dedupe_boxes(results)  # one box per page element, not one per candidate label
        results['is_demo'] = False
    else:
        # No trained checkpoint yet: synthetic boxes, flagged so clients can label them
        results = demo_inference(image_rgb)

    document = None
    if ocr:
        # Tesseract is a separate process; run it off the event loop like inference
        document = await run_in_threadpool(extract_document, image_rgb, results, ocr_lang)

    labels = [int(label) for label in results["labels"]]
    return {
        "status": "success",
        "results": {
            "boxes": results["boxes"].tolist(),
            "labels": labels,
            "class_names": [CLASS_NAMES[label] for label in labels],
            "scores": results["scores"].tolist(),
            "inference_time": results["inference_time"],
            "is_demo": results["is_demo"],
            "model_type": model_type,
            "mode": mode,
            "conf_threshold": conf_threshold,
            "document": document,
            "image_width": image.shape[1],
            "image_height": image.shape[0],
        },
    }


def _read_metrics(name: str) -> Optional[dict]:
    path = METRICS_DIR / f"{name}.json"
    return json.loads(path.read_text()) if path.exists() else None


def recommended_conf(model_type: str, mode: str = 'balanced') -> float:
    """Threshold chosen on the validation pages by scripts/evaluate.py (best F1, or high recall for 'all'), else
0.5."""
    metrics = _read_metrics(METRICS_NAMES[model_type]) or {}
    balanced = metrics.get('recommended_conf', 0.5)
    return metrics.get('high_recall_conf', balanced) if mode == 'all' else balanced


@app.get("/metrics")
def get_metrics():
    # Saved evaluation results; null until a model has been evaluated
    return {
        "authors": _read_metrics("authors_metrics"),
        "baseline": _read_metrics("baseline_metrics"),
        "proposed": _read_metrics("proposed_metrics"),
```

```
    }


# ---------------------------------------------------------------------------------------------- verification
async def _read_upload(file: UploadFile, limit: int = MAX_VERIFY_BYTES) -> bytes:
    contents = await file.read(limit + 1)
    if len(contents) > limit:
        raise HTTPException(status_code=413, detail=f"File larger than {limit // (1024 * 1024)} MB")
    return contents


def _pages_rgb(contents: bytes) -> list:
    """Every page of an uploaded PDF (or the one image) as RGB arrays."""
    if contents[:5] == b'%PDF-':
        n = pdf_page_count(contents)
        if n > MAX_VERIFY_PAGES:
            raise HTTPException(status_code=413, detail=f"At most {MAX_VERIFY_PAGES} pages per paper")
        return [cv2.cvtColor(render_pdf_page(contents, i, dpi=VERIFY_DPI, max_pixels=MAX_PIXELS), cv2.COLOR_BGR2RGB)
                for i in range(n)]
    try:
        return [cv2.cvtColor(decode_image(contents, max_pixels=MAX_PIXELS), cv2.COLOR_BGR2RGB)]
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e))


def _read_paper(images: list, lang: str) -> list:
    """Layout (DocLayout-YOLO, high-recall threshold) + OCR with the number pass, page by page."""
    conf = recommended_conf('doclayout', 'all')
    pages = []
    for image_rgb in images:
        with MODEL_LOCKS['doclayout']:
            model, device = get_model('doclayout')
            out = predict_single(model, Image.fromarray(image_rgb), 'doclayout', device, conf=conf)
        keep = out['scores'] >= conf
        out = dedupe_boxes({**out, 'boxes': out['boxes'][keep], 'labels': out['labels'][keep],
                            'scores': out['scores'][keep]})
        pages.append(extract_document(image_rgb, out, lang, number_pass=True))
    return pages


@app.get("/verify/profiles")
def verify_profiles():
    """Format profiles available to POST /verify (ml/verify/profiles/*.json)."""
    return list_profiles()


@app.post("/verify")
async def verify_paper(
    file: UploadFile = File(..., description="The paper to check (PDF or image)"),
    source: Optional[UploadFile] = File(None, description="The source-language version, for translation parity"),
    bank: Optional[UploadFile] = File(None, description="Earlier papers' questions: JSON [{'text', 'source'}]"),
    lang: str = Query('auto', pattern=r'^(auto|[a-z]{3}(\+[a-z]{3})*)$', description="OCR languages of `file`"),
    source_lang: str = Query('auto', pattern=r'^(auto|[a-z]{3}(\+[a-z]{3})*)$'),
    bilingual: Optional[str] = Query(None, description="One bilingual paper: its two scripts, e.g.
'Latin,Devanagari'"),
    profile: Optional[str] = Query(None, description="Format profile id from GET /verify/profiles"),
    output: Literal['json', 'html'] = Query('json'),
):
    """
    Pre-print verification of a question paper: numbering, options, marks, duplicates; with `source`, parity with
the
    source-language version; with `bank`, repeats of earlier questions; with `profile`, the document's format.
    """
    if not CHECKPOINTS['doclayout'].exists() or not tesseract_available():
        raise HTTPException(status_code=503, detail="Verification needs the layout model and OCR on this server")
    for code in (lang, source_lang):
        if code != 'auto' and not set(code.split('+')) <= OCR_CODES:
            raise HTTPException(status_code=400, detail=f"OCR languages must be 'auto' or codes from
{sorted(OCR_CODES)}")
    if profile is not None and profile not in list_profiles():
        raise HTTPException(status_code=400, detail=f"Unknown profile; choose from {sorted(list_profiles())}")
    split = None
    if bilingual:
        scripts = [s.strip() for s in bilingual.split(',')]
        if len(scripts) != 2 or not set(scripts) <= SCRIPTS or source is not None:
            raise HTTPException(status_code=400, detail=f"bilingual takes two scripts from {sorted(SCRIPTS)},
without `source`")
        split = {scripts[0]: scripts[0].lower(), scripts[1]: scripts[1].lower()}
    bank_items = None
    if bank is not None:
        try:
```

```
            bank_items = json.loads(await _read_upload(bank))
            assert isinstance(bank_items, list) and all(isinstance(b, dict) and 'text' in b for b in bank_items)
        except (ValueError, AssertionError):
            raise HTTPException(status_code=400, detail="bank must be a JSON list of {'text', 'source'} objects")

    images = _pages_rgb(await _read_upload(file))
    src_images = _pages_rgb(await _read_upload(source)) if source is not None else None
    pages = await run_in_threadpool(_read_paper, images, lang)
    src_pages = await run_in_threadpool(_read_paper, src_images, source_lang) if src_images else None
    meta = {'File': file.filename or 'paper', 'Pages': len(images)}
    if source is not None:
        meta['Checked against'] = source.filename or 'source'
    report = verify(pages, source_pages=src_pages, bank=bank_items, profile=profile, split_scripts=split, meta=meta)
    if output == 'html':
        imgs = {'paper': images, **({'source': src_images} if src_images else {})}
        return HTMLResponse(verify_report_html(report, imgs, title='IndicDoc Verify: pre-print report'))
    return report
```

This page is intentionally left blank.

PART 3

GitHub Repository

The project's code repository on GitHub, `github.com/va-ada/IndicDoc-AI`.

va-ada / **IndicDoc-AI** Private

branch: ccr-1ee1d4be-rwi3u2 2 branches 23 commits `https://github.com/va-ada/IndicDoc-AI.git`

`f4be430` Final verification pass: claims, citations, upload limit		24 Oct 2026
.streamlit	Make IndicDoc AI a working, tested, deployable pro...	19 Oct 2026
app	Add IndicDoc Verify: pre-print checking of questio...	24 Oct 2026
backend	Final verification pass: claims, citations, upload...	24 Oct 2026
data	Make IndicDoc AI a working, tested, deployable pro...	19 Oct 2026
dataset	Add course deliverables: dataset CSVs, Phase-1/Pha...	22 Oct 2026
docs	Final verification pass: claims, citations, upload...	24 Oct 2026
frontend	Make IndicDoc AI a working, tested, deployable pro...	19 Oct 2026
ml	IndicDoc Verify: benchmark results, Phase-2 deck...	24 Oct 2026
notebooks	Make IndicDoc AI a working, tested, deployable pro...	19 Oct 2026
outputs	IndicDoc Verify: benchmark results, Phase-2 deck...	24 Oct 2026
paper	Final verification pass: claims, citations, upload...	24 Oct 2026
scripts	Final verification pass: claims, citations, upload...	24 Oct 2026
tests	Add IndicDoc Verify: pre-print checking of questio...	24 Oct 2026
.dockerignore	Make IndicDoc AI a working, tested, deployable pro...	19 Oct 2026
.gitignore	Add IndicDoc Verify: pre-print checking of questio...	24 Oct 2026
docker-compose.yml	Make IndicDoc AI a working, tested, deployable pro...	19 Oct 2026
Dockerfile	Make IndicDoc AI a working, tested, deployable pro...	19 Oct 2026
README.md	IndicDoc Verify: benchmark results, Phase-2 deck...	24 Oct 2026
requirements.txt	Make IndicDoc AI a working, tested, deployable pro...	19 Oct 2026

About

Layout detection + OCR for multilingual Indian documents (IndicDLP): DocLayout-YOLO, YOLOv8, Faster R-CNN, Tesseract; Streamlit app + FastAPI in Docker

Languages

- Python **92.4%**
- TeX **5.4%**
- JavaScript **1.3%**
- CSS **0.8%**

Branches

`main`
`ccr-1ee1d4be-rwi3u2` (this submission)

Open the repository

Latest commits on `ccr-1ee1d4be-rwi3u2`

Commit	Message	Author	Date
`f4be430`	Final verification pass: claims, citations, upload limit	Claude	24 Oct 2026
`681cd4d`	IndicDoc Verify: benchmark results, Phase-2 deck, lab report and IEEE paper	Claude	24 Oct 2026
`bec2bec`	Add IndicDoc Verify: pre-print checking of question papers and their translations	Claude	24 Oct 2026
`40e7b08`	Lab report: drop the SFIT logo and institute header; plain title block, Expt.-11 structure kept	Claude	24 Oct 2026
`5064aff`	Add Expt.-11 lab report in the SFIT format (Phase-2 sections, IEEE style)	Claude	24 Oct 2026
`4009557`	Add scripts/pdf_to_epub.py: whole scanned PDF to one accessible EPUB3, script detected per page	Aditya Soni	22 Oct 2026
`185b3b3`	Plan: focus on accessible Indian-language study material; add accessible HTML/EPUB3 export	Aditya Soni	22 Oct 2026
`2dd653e`	Add course deliverables: dataset CSVs, Phase-1/Phase-2 slides, research paper, robustness study	Aditya Soni	22 Oct 2026
`030cf52`	Make IndicDoc AI a working, tested, deployable product	Aditya Soni	19 Oct 2026
`137aa96`	style: Apply Liquid Glass/Glassmorphism theme to React frontend components	Prashantj44	26 Sep 2026

README.md

IndicDoc AI · IndicDoc Verify

Check an Indian question paper and its translations before it is printed. Offline, 12 Indian languages.

Indian exams are set in one language and translated into Hindi and regional languages; every version is checked by hand, and errors keep reaching candidates (NEET 2018: 49 wrong questions in Tamil; KPSC 2024: 59 errors and a re-exam; UGC-NET June 2026: three papers re-tested). IndicDoc Verify reads the printed paper and reports:

- **Structure:** numbering gaps and repeats, the same question printed twice, a question with a missing or repeated option, marks that do not add up to the printed maximum, page numbering
- **Translation parity:** a question missing in a version, and every number, option, mark, sub-question or figure that differs from the source version, question by question, **without translating anything**
- **Repeats:** questions copied from earlier papers (a question bank)
- **Format:** any document type described as a JSON profile (e.g. an Office Memorandum, bilingual circulars)

Each finding points to the question and page and is outlined on the page (HTML / JSON report). It runs in one Docker image with no internet access, because a pre-print paper must never leave the exam body.

How it works. A deep-learning layout detector (DocLayout-YOLO, fine-tuned on IndicDLP by its authors) finds the questions, sub-questions, options and answers on each page; Tesseract reads the text, with an English *number pass* that must agree on every digit; the questions are built from the layout and checked. Three layout detectors are compared on the same IndicDLP test pages (DocLayout-YOLO, and a YOLOv8s and a Faster R-CNN we fine-tuned).

Product, evidence and market: docs/PRODUCT.md. Benchmark: Verification results.

> Repository details taken from git and GitHub on 27 October 2026. Clone with `git clone https://github.com/va-ada/IndicDoc-AI.git` (access granted by the owner).

LITERATURE SURVEY · ML MINI PROJECT

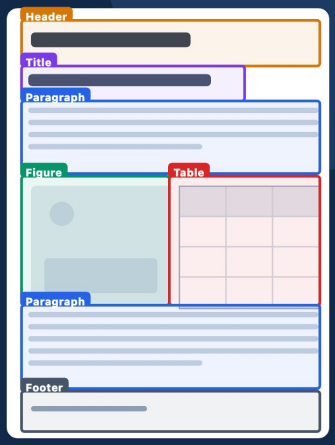
IndicDoc AI

Document Layout Parsing for Multilingual Indian Documents

1 2 3 4 5 6 7 8 **8 research papers**

Prashant Jha (PJ) — Roll No. 41
Aditya Soni (AS) — Roll No. 42
Riddhi Patil (RP) — Roll No. 39
Mariam Badure (MB) — Roll No. 40

Course: ML Mini Project
Guide: Dr. Joanne Gomes
Institute: St. Francis Institute of Technology



WHY THIS WORK

Motivation & SDG Mapping

WHY THIS PROBLEM MATTERS

Many languages, many scripts
22 scheduled languages; scripts have ligatures and diacritics.

Huge paper archives
Circulars, textbooks, newspapers, forms — often scanned.

Tools are built for English
Most layout & table datasets are English-only.

Key point: layout detection is step 1 — wrong regions mean wrong OCR, tables and answers.

MAPPED SDGS

SDG 4 — Quality Education
Digital textbooks in Indian languages

SDG 9 — Industry & Innovation
AI building blocks for Indic documents

SDG 10 — Reduced Inequalities
Document AI beyond English

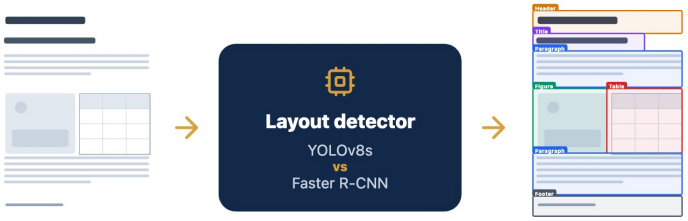
SDG 16 — Strong Institutions
Searchable government records

IndicDoc AI | Literature Survey 2

WHAT WE SOLVE

Problem Statement

Problem: find and label every region on an Indian-language page — paragraphs, headings, tables, figures — so OCR, table extraction and QA can follow.



SCOPE

- Indian-language pages (IndicDLP, via AIKosh)
- Region detection: boxes + labels
- YOLOv8s and Faster R-CNN
- OCR, table QA, grounding → future

RESEARCH OBJECTIVE

Compare one-stage (YOLOv8s) and two-stage (Faster R-CNN) detectors, using IndicDLP's YOLO results as reference.

IndicDoc AI | Literature Survey 3

SELECTED PAPERS

Paper Details

1 IndicDLP
IndicDLP: A Foundational Dataset for Multi-Lingual and Multi-Domain Document Layout Parsing
O. Nath, S. Kukkala, M. Khapra, R. K. Sarvadevabhatla
2025 arXiv preprint
IIT Madras & IIIT Hyderabad

2 PLATTER
PLATTER: A Page-Level Handwritten Text Recognition System for Indic Scripts
B. V. Kasuba, D. Kudale, V. Subramanian, P. Chaudhuri, G. Ramakrishnan
2025 arXiv preprint
IIT Bombay

3 UniTabBank
UniTabBank: A Large Scale Multi-Lingual, Multi-Layout, Multi-Type, Multi-Format Dataset for Table Detection
A. Mondal, S. Mundra, A. Dasgupta, C. V. Jawahar
2026 IEEE/CVF WACV
IIIT Hyderabad

4 Indic Table QA
Table Question Answering for Low-resourced Indic Languages
V. Pal, E. Kanoulas, A. Yates, M. de Rijke
2024 arXiv preprint
University of Amsterdam

5 LayoutXLM
LayoutXLM: Multimodal Pre-training for Multilingual Visually-rich Document Understanding
Y. Xu, T. Lv, L. Cui, G. Wang, Y. Lu, D. Florencio, C. Zhang, F. Wei
2021 arXiv preprint
Microsoft Research Asia

6 DRISHTIKON
DRISHTIKON: Visual Grounding at Multiple Granularities in Documents
B. V. Kasuba, P. Chaudhuri, G. Ramakrishnan
2025 arXiv preprint
IIT Bombay

7 HindiOCR-VLM
Adapting Vision-Language Models for Hindi OCR (HindiOCR-VLM)
S. Bhattacharyya, S. Ghosh, P. Deb, A. Mondal, C. V. Jawahar
2025 Conference paper
IIIT Hyderabad

8 Devanagari Tables
Incorporate Structure and Content for Devanagari Table Extraction
A. R. Dumada, S. G. Shah
2026 IJSAT 17(1)
Deogiri Institute, Chh. Sambhajinagar

IndicDoc AI | Literature Survey 4

PAPER-1 · INTRODUCTION · METHODOLOGY · RESULTS

IndicDLP

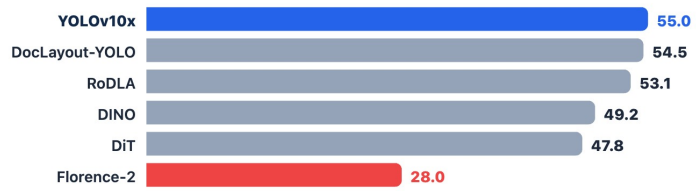
IndicDLP: A Foundational Dataset for Multi-Lingual and Multi-Domain Document Layout Parsing — Nath et al., 2025, IIT Madras & IIIT Hyderabad

PROBLEM
Layout datasets are mostly English; models do poorly on Indic pages.

KEY IDEA
A huge human-labelled layout dataset across 12 languages and 12 document types; benchmark six detectors.

DATA & MODELS
~120K pages · 12 domains · 42 region labels
YOLOv10x · DocLayout-YOLO · RoDLA · DINO · DiT · Florence-2

RESULT mAP@[50:95] on IndicDLP



55.0 best mAP

−20 to −25 mAP on unseen scripts

METHODOLOGY WORKFLOW

1. Collect — 12 domains → 2. Annotate — 50-person team → 3. Review — Superchecks → 4. Split — 80/10/10 → 5. Train — 6 detectors → 6. Evaluate — mAP

Limitation: Vision-only models; rectangular boxes only.

Our project: Our dataset source (AIKosh); its YOLO results are the reference for YOLOv8s vs Faster R-CNN.

IndicDoc AI | Literature Survey 5

PAPER-2 · INTRODUCTION · METHODOLOGY · RESULTS

PLATTER

PLATTER: A Page-Level Handwritten Text Recognition System for Indic Scripts — Kasuba et al., 2025, IIT Bombay

PROBLEM
Indic handwriting work is word-level, compared inconsistently, with little page-level data.

KEY IDEA
Two stages: find words (DBNet), then read them with one of six recognisers.

DATA & MODELS
New CHIPS dataset: ~25K pages, 10 languages
DBNet · CRNN · MASTER · PARSeq · +3 more

RESULT Word accuracy (%): clean words vs words cut from pages

99.7% detection F1

73% → 38% CRNN on page crops

METHODOLOGY WORKFLOW

1. Page — Handwritten → 2. Detect — DBNet words → 3. Crop — Each word → 4. Recognise — 1 of 6 models → 5. Output — Text on page

Limitation: Synthetic pages; language must be named by the user.

Our project: Detection precedes recognition; our layout output can feed an OCR stage.

IndicDoc AI | Literature Survey 6

PAPER-3 · INTRODUCTION · METHODOLOGY · RESULTS

UniTabBank

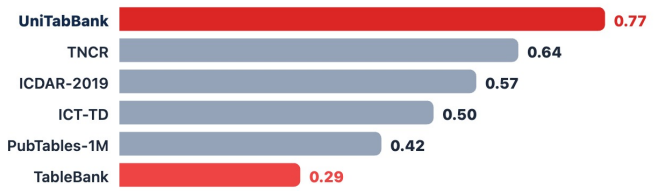
UniTabBank: A Large Scale Multi-Lingual, Multi-Layout, Multi-Type, Multi-Format Dataset for Table Detection — Mondal et al., 2026, IIIT Hyderabad

PROBLEM
Table datasets are English, single-domain, clean PDFs; detectors fail on real documents.

KEY IDEA
One diverse table dataset (PDFs, scans, photos) auto-labelled then fixed by humans; YOLOv11 detector.

DATA & MODELS
55K pages · 82K tables · 28 languages
UniTabDet (YOLOv11) · DocLayout-YOLO · TATR

RESULT AP on unseen UNLV data, by training dataset



8 of 9 benchmarks beaten

0.77 vs 0.42 AP on UNLV

METHODOLOGY WORKFLOW

1. Collect — 28 languages → 2. Auto-label — DocLayout-YOLO → 3. Fix — Annotators → 4. Recheck — 20% sample → 5. Train — UniTabDet → 6. Test — 9 benchmarks

Limitation: Borderless tables and degraded scans.

Our project: Tables are one of our layout classes; YOLO-family detectors work well here.

IndicDoc AI | Literature Survey 7

PAPER-4 · INTRODUCTION · METHODOLOGY · RESULTS

Indic Table QA

Table Question Answering for Low-resourced Indic Languages — Pal et al., 2024, University of Amsterdam

PROBLEM
No table QA data for low-resource languages; translating English tables is error-prone.

KEY IDEA
Auto-generate questions and answer tables from Bengali / Hindi Wikipedia tables using SQL.

DATA & MODELS
BanglaTabQA 2M · HindiTabQA 643K samples
mBART · M2M-100 · Llama-7B · vs GPT-4

RESULT Bengali test set — table exact match (%)

35.9% vs 26.8% mBART vs GPT-4

2M + 643K Bengali + Hindi samples

METHODOLOGY WORKFLOW

1. Tables — Wikipedia → 2. SQL — Templates → 3. Question — SQL2NQ → 4. Answer — Run SQL → 5. Filter — Quality check → 6. Train — mBART...

Limitation: Needs web tables; no multi-table queries; small test sets.

Our project: Detected tables can feed table QA on Indian-language data.

IndicDoc AI | Literature Survey 8

PAPER-5 · INTRODUCTION · METHODOLOGY · RESULTS

LayoutXLM

LayoutXLM: Multimodal Pre-training for Multilingual Visually-rich Document Understanding — Xu et al., 2021, Microsoft Research Asia

PROBLEM
Document AI models were English-only; text-only multilingual models ignore layout.

KEY IDEA
Pre-train one model on text + layout + image across 53 languages; new XFUND benchmark.

DATA & MODELS
30M docs, 53 languages · XFUND: 1,393 forms, 7 languages

LayoutLMv2 · InfoXLM · MMVLM · TIA · TIM

RESULT Average F1 on XFUND + FUNSD (large models)

0.83 vs 0.74 entity F1 vs XLM-R

Zero-shot transfers to new languages

METHODOLOGY WORKFLOW

1. Page — Document → 2. OCR — Words + boxes → 3. Embed — Text+layout+image → 4. Transformer — Pre-train → 5. Fine-tune — Entities

Limitation: No Indic language in XFUND; needs OCR; large model.

Our project: Layout regions help document understanding; Indic scripts not covered.

PAPER-6 · INTRODUCTION · METHODOLOGY · RESULTS

DRISHTIKON

DRISHTIKON: Visual Grounding at Multiple Granularities in Documents — Kasuba et al., 2025, IIT Bombay

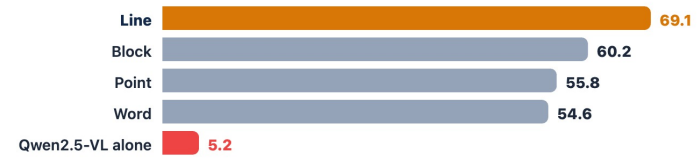
PROBLEM
Document QA gives answers but not where on the page they come from.

KEY IDEA
LLM gives the answer; a matching algorithm finds it on the page at block, line, word or point level.

DATA & MODELS
MGVG: 70 circulars · 509 QA pairs

DocTR / Surya OCR · Llama-3.1-8B · Region matching

RESULT Grounding F1 (%) by level



69.1% line-level F1 (best)

5.2% Qwen2.5-VL alone

METHODOLOGY WORKFLOW

1. Input — Page + question → 2. OCR — Text blocks → 3. Answer — Llama-3.1-8B → 4. Match — Fuzzy score → 5. Ground — Block→point

Limitation: Depends on OCR; single page; small test set.

Our project: Our layout regions can serve as blocks for answer grounding.

PAPER-7 · INTRODUCTION · METHODOLOGY · RESULTS

HindiOCR-VLM

Adapting Vision-Language Models for Hindi OCR (HindiOCR-VLM) — Bhattacharyya et al., 2025, IIIT Hyderabad

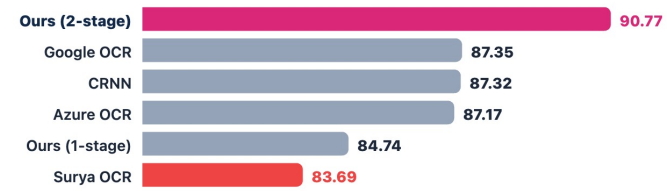
PROBLEM
Indian-script OCR is hard (ligatures, diacritics); each text type needs a separate model.

KEY IDEA
LoRA-tune GOT OCR 2.0 for Hindi, learning step by step: words → lines → blocks → pages.

DATA & MODELS
Printed Hindi: 1.1M words → 3.6K pages (+ handwritten, scene)

GOT OCR 2.0 · LoRA (rank 8) · CRAFT / YOLO

RESULT Printed Hindi — word recognition rate (%) *Axis starts at 80*



90.77 vs 87.35 vs Google OCR

56% faster 350K vs 800K steps

METHODOLOGY WORKFLOW

1. Base — GOT OCR 2.0 → 2. LoRA — Rank 8 → 3. Words — 50K steps → 4. + Lines — Fast transfer → 5. + Pages — From 200K → 6. Done — 350K steps

Limitation: Weak on mixed Hindi-English lines; scene text trails Google OCR (67.15 vs 74.38 WRR); Hindi only (Marathi WRR 36.70).

Our project: Page-level OCR stage that can follow our layout detection for Devanagari text.

IndicDoc AI | Literature Survey 11

PAPER-8 · INTRODUCTION · METHODOLOGY · RESULTS

Devanagari Tables

Incorporate Structure and Content for Devanagari Table Extraction — Dumada & Shah, 2026, Deogiri Institute, Chh. Sambhajinagar

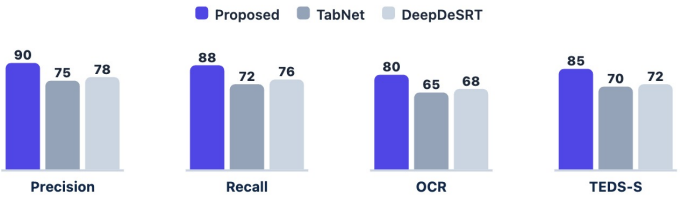
PROBLEM
Table extractors perform poorly on Devanagari: ligatures, noisy scans, scarce data.

KEY IDEA
Combine table structure (lines, cells) with OCR content checks using weighted fusion.

DATA & MODELS
500+ annotated Devanagari documents

Hough deskew · Contours · Tesseract · Weighted fusion

RESULT Proposed vs baselines (%) — self-reported



85% TEDS-S (vs 70% TabNet)

90% structural precision

METHODOLOGY WORKFLOW

1. Input — Document → 2. Pre-process — Denoise, deskew → 3. Structure — Lines & cells → 4. OCR — Tesseract → 5. Fusion — Weighted → 6. Output — Table

Limitation: Dataset, baseline adaptation and fusion weights not described; no ablation; results are self-reported.

Our project: Supporting reference for the table-structure stage; its claims need independent validation.

IndicDoc AI | Literature Survey 12

LITERATURE SURVEY TABLE

Summary: Strengths of Papers

S.N.	Paper	What is good	Innovations	Assumptions
1	**IndicDLP** Nath et al., 2025 · arXiv preprint	Largest human-labelled Indic layout set	42 fine labels (jumpline, flag…) + UED-mini pre-training	Rectangular boxes are enough
2	**PLATTER** Kasuba et al., 2025 · arXiv preprint	Fair comparison of 6 HTR models, 10 languages	Language-agnostic detector + CHIPS dataset	Synthetic pages behave like real ones
3	**UniTabBank** Mondal et al., 2026 · IEEE/CVF WACV	28 languages; scans + photos; generalises	Semi-automatic labelling	Script barely affects table detection
4	**Indic Table QA** Pal et al., 2024 · arXiv preprint	Cheap, automatic data; small models beat GPT-4	SQL → question model + quality filter	Wikipedia tables represent real ones
5	**LayoutXLM** Xu et al., 2021 · arXiv preprint	Text + layout + image together	Language-free token boxes; XFUND	Good OCR is available
6	**DRISHTIKON** Kasuba et al., 2025 · arXiv preprint	Shows where the answer is → trust	Multi-level grounding + MGVG	Answer appears word-for-word in OCR
7	**HindiOCR-VLM** Bhattacharyya et al., 2025 · Conference paper	One model for printed, handwritten, scene	Progressive learning, 56% faster	English/Chinese encoder transfers
8	**Devanagari Tables** Dumada & Shah, 2026 · IJSAT 17(1)	Targets Devanagari tables directly	Structure + content fusion	Tesseract is accurate enough

IndicDoc AI | Literature Survey 13

LITERATURE SURVEY TABLE

Summary: Limitations of Papers

Paper	Weaknesses	Missing aspects	Assumptions
1. IndicDLP Nath et al., 2025 · arXiv preprint	Vision-only; −20–25 mAP on unseen scripts	Polygon boxes; text cues	Look alone tells region type
2. PLATTER Kasuba et al., 2025 · arXiv preprint	Big accuracy drop on page crops	Auto script detection; real pages	User names the language
3. UniTabBank Mondal et al., 2026 · IEEE/CVF WACV	Borderless tables, degraded scans	Row / cell structure	Auto labels nearly correct
4. Indic Table QA Pal et al., 2024 · arXiv preprint	Small test sets (165 Bengali)	Multi-table, complex queries	Clean tables, no OCR noise
5. LayoutXLM Xu et al., 2021 · arXiv preprint	Large model; needs OCR	No Indic language	Synthetic forms look real
6. DRISHTIKON Kasuba et al., 2025 · arXiv preprint	OCR-dependent; small test set	Multi-page, mixed script	Fixed limits suit all documents
7. HindiOCR-VLM Bhattacharyya et al., 2025 · Conference paper	Mixed-language lines; scene text	Other Indic languages	In-house data is representative
8. Devanagari Tables Dumada & Shah, 2026 · IJSAT 17(1)	Self-reported; no ablation	Method details, public data	Baselines adapted fairly

IndicDoc AI | Literature Survey 14

WHAT IS MISSING

Research Gaps

IMPROVEMENTS SUGGESTED BY US		POSSIBLE EXTENSIONS BY US
Only big models benchmarked on IndicDLP	→ Compare lightweight YOLOv8s vs Faster R-CNN	**Auto script detection** Pick the right OCR per region
–20–25 mAP on unseen scripts	→ Train on all languages; report per-language mAP	**Multi-page documents** Grounding & QA beyond one page
Borderless tables, degraded scans fail	→ Add blur / noise augmentation	**Text + layout for Indic** LayoutXLM-style model for Indian scripts
Layout, OCR, tables, QA built separately	→ Output boxes the next stages can use	**Lightweight deployment** Small models on mobile / edge

IndicDoc AI | Literature Survey 15

OUR PROJECT

Our Proposed Idea

NEW APPROACH — WORKFLOW

IndicDLP pages From AIKosh → **Prepare data** Images + region labels → **YOLOv8s** One-stage · fast / **Faster R-CNN** Two-stage · precise → **Evaluate** mAP, speed, per class & language → **Compare** vs IndicDLP YOLOv10x (55.0 mAP)

OUR IMPROVEMENT IDEA

Test a lightweight YOLOv8s against Faster R-CNN on IndicDLP — a pair the IndicDLP paper did not test — with output ready for the next stages.

EXPECTED OUTCOME

- Speed vs accuracy trade-off for Indic pages
- Which regions / scripts are hardest
- Region boxes ready for OCR & tables

NEXT STAGES

- OCR → HindiOCR-VLM / PLATTER
- Tables → UniTabBank + Table QA
- Grounding → DRISHTIKON

IndicDoc AI | Literature Survey 16

KEY LEARNINGS

Conclusion

Data is the bottleneck
Big human-labelled Indic datasets drive progress.

Detect first, then read
Two-stage pipelines win.

YOLO-family is strong
Accurate and fast for layout and tables.

Script matters
–20–25 mAP on unseen scripts.

Small tuned models win
Fine-tuned mBART beat GPT-4.

VLMs can't locate well
Qwen2.5-VL: 5.2% grounding F1.

Takeaway: layout detection is the first building block of Indian document AI — that is what our project builds.

IndicDoc AI | Literature Survey 17

SOURCES

References

PAPER CITATIONS

[1] O. Nath et al., "IndicDLP: A Foundational Dataset for Multi-Lingual and Multi-Domain Document Layout Parsing," arXiv:2512.20236, 2025.

[2] B. V. Kasuba et al., "PLATTER: A Page-Level Handwritten Text Recognition System for Indic Scripts," arXiv:2502.06172, 2025.

[3] A. Mondal et al., "UniTabBank: A Large Scale Multi-Lingual, Multi-Layout, Multi-Type, Multi-Format Dataset for Table Detection," IEEE/CVF WACV, pp. 6486–6495.

[4] V. Pal et al., "Table Question Answering for Low-resourced Indic Languages," arXiv:2410.03576, 2024.

[5] Y. Xu et al., "LayoutXLM: Multimodal Pre-training for Multilingual Visually-rich Document Understanding," arXiv:2104.08836, 2021.

[6] B. V. Kasuba et al., "DRISHTIKON: Visual Grounding at Multiple Granularities in Documents," arXiv:2506.21316, 2025.

[7] S. Bhattacharyya et al., "Adapting Vision-Language Models for Hindi OCR," IIIT Hyderabad, hindiocr-vlm.github.io.

[8] A. R. Dumada, S. G. Shah, "Incorporate Structure and Content for Devanagari Table Extraction," IJSAT, vol. 17, no. 1, 2026.

OTHER SOURCES

- IndicDLP dataset — AIKosh
- indicdlp.github.io
- github.com/iitb-research-code/doctr
- aka.ms/layoutxlm
- github.com/kolk/Low-Resource-TableQA-Indic-languages
- hindiocr-vlm.github.io

IndicDoc AI | Literature Survey 18

Thank You

Questions and discussion

Prashant Jha | Aditya Soni | Riddhi Patil | Mariam Badure

IndicDoc AI | Literature Survey

PHASE-2 · PROJECT IMPLEMENTATION · DL MINI PROJECT

IndicDoc Verify

Deep-learning layout analysis that checks Indian question papers and their translations **before they are printed**: offline, 12 languages

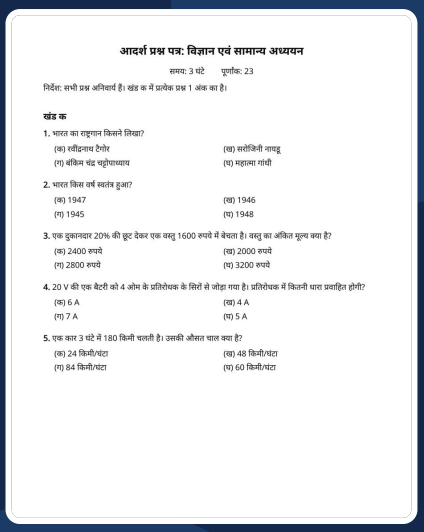
PJ **Prashant Jha** Roll No. 41

AS **Aditya Soni** Roll No. 42

RP **Riddhi Patil** Roll No. 39

MB **Mariam Badure** Roll No. 40

Course: Deep Learning Mini Project (Expt. 11, SDG-based)
Department: Artificial Intelligence and Machine Learning, St. Francis Institute of Technology (SFIT)
Guide: Dr. Joanne Gomes
Academic year: 2026–27 · Semester V (TE AIML)



आदर्श प्रश्न पत्र: विज्ञान एवं सामान्य अध्ययन

समय: 3 घंटे पूर्णांक: 23

निर्देश: सभी प्रश्न अनिवार्य हैं। खंड क में प्रत्येक प्रश्न 1 अंक का है।

खंड क

1. भारत का राष्ट्रगान किसने लिखा?
(क) रवींद्रनाथ टैगोर (ख) सरोजिनी नायडू
(ग) बंकिम चंद्र चट्टोपाध्याय (घ) महात्मा गांधी

2. भारत किस वर्ष स्वतंत्र हुआ?
(क) 1947 (ख) 1946
(ग) 1945 (घ) 1948

3. एक दुकानदार 20% की छूट देकर एक वस्तु 1600 रुपये में बेचता है। वस्तु का अंकित मूल्य क्या है?
(क) 2400 रुपये (ख) 2000 रुपये
(ग) 2800 रुपये (घ) 3200 रुपये

4. 20 V की एक बैटरी को 4 ओम के प्रतिरोधक के सिरों से जोड़ा गया है। प्रतिरोधक में कितनी धारा प्रवाहित होगी?
(क) 6 A (ख) 4 A
(ग) 7 A (घ) 5 A

5. एक कार 3 घंटे में 180 किमी चलती है। उसकी औसत चाल क्या है?
(क) 24 किमी/घंटा (ख) 48 किमी/घंटा
(ग) 84 किमी/घंटा (घ) 60 किमी/घंटा

OVERVIEW

Content

01 Literature Survey Summary Table
02 Research Gaps
03 Problem Statement and Objectives
04 Proposed System, Methodology and Novelty
05 System Block Diagram
06 Dataset Description and Pre-processing
07 Deep Learning Model Architecture
08 Tools, Technologies and Environment
09 Implementation Details
10 Experimental Setup and Training
11 Results and Performance Evaluation
12 Comparison with Baseline / Existing Methods
13 Sample Outputs and Demonstration
14 Challenges and Limitations
15 Conclusion and Future Scope
16 References

PHASE-1 RECAP + TRANSLATION QA

Literature Survey Summary Table

Ref	Author	Problem	Dataset	Method	Limitation
[1]	**Nath et al., 2025**	Layout parsing of Indian-language pages	IndicDLP: 119,806 pages, 12 languages, 42 labels incl. questions and options	Benchmark of YOLOv10x, DocLayout-YOLO, RoDLA, DINO, DiT, Florence-2	Detects regions only; nobody uses the question labels to check papers
[2]	**Kasuba et al., 2025**	Page-level handwritten Indic text recognition	CHIPS, ~25K pages, 10 languages	DBNet word detection + 6 recognisers (CRNN, PARSeq, ...)	User must name the language; text only
[3]	**Mondal et al., 2026**	Multilingual table detection	UniTabBank: 55K pages, 82K tables, 28 languages	Semi-automatic labelling; YOLOv11 detector	Borderless tables and degraded scans fail
[4]	**Pal et al., 2024**	Table question answering, low-resource Indic	BanglaTabQA 2M, HindiTabQA 643K	SQL-generated QA; fine-tuned mBART / M2M / Llama	Wikipedia tables only
[5]	**Xu et al., 2021**	Multilingual document understanding	30M docs; XFUND, 7 languages	LayoutXLM: text + layout + image transformer	No Indian language
[6]	**Kasuba et al., 2025**	Grounding answers on the page	MGVG: 70 circulars, 509 QA pairs	OCR + Llama-3.1-8B + region matching	OCR-dependent; single page
[7]	**Bhattacharyya et al., 2025**	Hindi OCR with vision-language models	1.1M printed Hindi words + pages	LoRA-tuned GOT-OCR 2.0	Hindi only; mixed-language lines are hard
[8]	**Dumada & Shah, 2026**	Devanagari table extraction	500+ Devanagari documents	Structure + Tesseract content, weighted fusion	No ablation; self-reported
[9]	**Babayeva et al., 2025**	Translating a test into other languages with an LLM	Force Concept Inventory, many languages	GPT-4o translation + back-translation + human review	Every item still checked by hand; no page structure
[10]	**Rei et al., 2022**	Translation quality estimation without a reference	WMT 2022 QE shared task	CometKiwi: multilingual encoder predicts quality per sentence	Plain sentences: no question numbers, options or marks

IndicDoc AI | Phase-2: Project Implementation 3

WHAT IS MISSING

Research Gaps

⚠ Exam papers are translated into up to 13 languages and vetted only by hand; errors reach candidates	G1 ✓ **An automatic pre-print check of every language version against the source**
⚠ Translation-quality tools [9, 10] read plain sentences: blind to question numbers, options, marks, figures	G2 ✓ **Compare versions question by question using the page layout itself**
⚠ Indic layout models [1] detect questions and options, but this is not used to check papers	G3 ✓ **Use the detector's question / option regions (AP@50 0.81–0.90) as the backbone of the checker**
⚠ Indic OCR misreads Latin digits in mixed text [7]; a checker of numbers needs every digit right	G4 ✓ **A "number pass": two OCR models must agree on every number**

We also looked at court-filing defect checks: the Supreme Court already uses an AI tool built with IIT Madras, and a commercial tool is reported in 16 High Courts, so we chose the problem **nobody automates yet**: checking a question paper and its translations.

IndicDoc AI | Phase-2: Project Implementation 4

WHAT WE SOLVE

Problem Statement and Objectives

Problem: a question paper is set in one language and translated into Hindi and regional languages. Every version is checked by hand before printing. When a number, an option, a mark or a whole question differs, lakhs of candidates are affected: grace marks, re-exams, court cases.

MAIN OBJECTIVE

Build an offline deep-learning system that reads a printed paper in any of 12 Indian languages and flags every structural or numerical error, and every difference from its source version, before printing.

SPECIFIC OBJECTIVES

- Detect the paper's structure (questions, options, answers) with a layout detector; compare 3 detectors.
- Read text with reliable numbers in 12 languages (number pass).
- Checks: numbering, duplicates, options, marks, translation parity, repeats, format profiles.
- Measure errors found and false alarms on a benchmark; ship app + API in Docker.

EXPECTED OUTPUT

A verdict (print / check / do not print) and each finding with its question and page, outlined on the page, as HTML and JSON.

IT HAS ALREADY HAPPENED

NEET-UG 2018
49 questions wrong in the Tamil version; HC ordered 196 grace marks, SC set it aside

KPSC KAS 2024
59 errors in the Kannada paper; re-exam for ~2 lakh candidates, errors again

APPSC Group-I
51 of 120 questions alleged mistranslated into Telugu; SC notice

UGC-NET Jun 2026
Translation errors + questions repeated from old papers; re-test 9–10 Sep

MBBS (NTRUHS) 2026
The same MCQ printed twice; AP HC ordered a compensatory mark

Sources: The News Minute, Deccan Herald, India Legal, The Week, SCC Online, Medical Dialogues (refs. slide).

IndicDoc AI | Phase-2: Project Implementation 5

OUR APPROACH

Proposed System, Methodology and Novelty

1. **Paper in** — PDF / scans + source version
2. **Find structure** — DocLayout-YOLO: questions, options
3. **Read text** — Indic OCR + English number pass
4. **Build questions** — number, stem, options, marks, figures
5. **Check** — structure, parity, repeats, format
6. **Report** — verdict + findings on the page

How deep learning is used

A convolutional object detector finds every question, sub-question, option and answer on the page (AP@50 0.81–0.90 on these labels). Its regions and the word positions inside them turn a picture of a page into structured questions, in any script.

What is new

Translation parity without translation: numbers, option counts, marks, sub-questions and figures must be the same in every language, so versions are compared question by question with no MT model.
Number pass: two OCR models must agree on every digit.

Why it sells

USP: no tool we found checks an Indian question paper against its translations question by question; IndicDoc Verify does, offline, in 12 languages, before printing. Confidential papers never leave the exam body. Same engine checks any document format via JSON profiles (e.g. bilingual circulars).

IndicDoc AI | Phase-2: Project Implementation 6

ARCHITECTURE

System Block Diagram

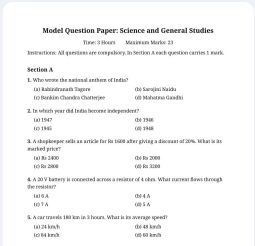
USER	**Streamlit: Verify Paper tab** upload paper, source, bank, profile	**REST API (FastAPI)** POST /verify · GET /verify/profiles	**Docker, offline** papers never leave the exam body
INPUT	**Render pages** PDF at 200 dpi, images	**Source version** optional, any of 12 languages	**Question bank · profile** earlier papers · JSON format rules
DEEP LEARNING	**Layout detector** DocLayout-YOLO (default) · YOLOv8s · Faster R-CNN	**Post-process** high-recall threshold, duplicate removal	**Question regions** questions, sub-questions, options, answers
TEXT	**Indic OCR** Tesseract, the page's own script	**Number pass** English model, digits must agree	**Questions** words → regions → options by layout gaps
CHECKS + OUTPUT	**Structure · repeats** numbering, duplicates, options, marks, bank	**Translation parity** align versions; numbers, options, marks	**Report** verdict, findings, pages outlined; HTML / JSON

IndicDoc AI | Phase-2: Project Implementation 7

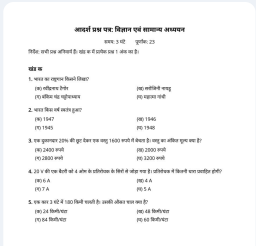
DATA

Dataset Description and Pre-processing

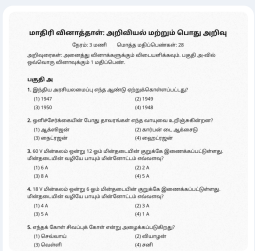
Layout data	**IndicDLP** (Nath et al., ICDAR 2025; Hugging Face `ai4bharat/indicdlp`, AIKosh): 119,806 pages, 12 languages, 12 domains, 42 labels
Our subset	**3,317 pages**, 49,995 boxes, language-balanced, official splits: train 2,293 · val 342 · test 682. 195 question-paper pages; 1,227 question and 1,209 option boxes
Verify benchmark	**132 paper pairs** (English → Hindi, English → Tamil), 18 questions each (MCQ + short answer), rendered at 200 dpi; **108** with one injected error of 9 real kinds, 24 clean; half degraded like scans
Kept apart	A separate **development set** (48 pairs, other seed) was used to build the parser; the test set above was run once
Pre-processing	Parquet → PNG + COCO JSON; boxes clipped; COCO → YOLO labels; letterbox 640 / 1024 px. Verify: PDF → 200 dpi; blur 0.7 px + noise σ 8 + JPEG 55 for "scanned"
Augmentation	YOLOv8: mosaic, HSV, scale. Faster R-CNN: colour jitter, blur. No mirror flips
Challenges	Class imbalance ("paragraph" = 32% of boxes); no OCR ground truth in IndicDLP; no public set of erroneous exam papers, hence the benchmark
Deliverable	`dataset/` CSVs + samples; `scripts/verify_bench/` rebuilds the benchmark



benchmark · English source



benchmark · Hindi version



benchmark · Tamil, scan-degraded

IndicDLP · hindi newspaper

IndicDoc AI | Phase-2: Project Implementation 8

MODELS

Deep Learning Model Architecture

DocLayout-YOLO · 20.0 M params · used by IndicDoc Verify

Input	Backbone	Neck	Head	Output
1024×1024×3 page	CSP conv blocks + GL-CRM (global-to-local receptive fields)	PAN feature pyramid, 3 scales	YOLOv10-style, NMS-free	N × (box, 42-class score)

Fine-tuned on full IndicDLP by its authors (Zenodo 15881917). Question labels: first-level 0.85, second-level 0.90, options 0.84, answer 0.81 AP@50.

YOLOv8s · 11.2 M params · fine-tuned by us

Input	Backbone	Neck	Head	Output
640×640×3	CSPDarknet with C2f blocks + SPPF	PAN-FPN	decoupled, anchor-free	boxes + 42 classes

Transfer learning: COCO-pretrained weights, 80-class head replaced by a 42-class head. Loss: CIoU + DFL (boxes) + BCE (classes).

Faster R-CNN · 41.5 M params · baseline

Input	Backbone	Neck	RPN + RoIAlign	Box head
longest side 640–1024	ResNet-50, frozen batch-norm	FPN, 5 levels	region proposals	43 outputs (42 + background)

Two-stage detector, COCO-pretrained; predictor replaced (43 classes). Loss: RPN + classification CE + smooth-L1 box.

IndicDoc AI | Phase-2: Project Implementation 9

STACK

Tools, Technologies and Environment

Component	Details
Programming language	Python 3.11 (ML, checks, app, API); JavaScript (Node) for the benchmark renderer
DL framework	PyTorch 2 with torchvision; Ultralytics 8 (YOLOv8); doclayout-yolo 0.0.4
Development environment	VS Code / terminal, Git + GitHub, pytest (unit tests for every check)
Libraries	pycocotools, OpenCV, Pillow, NumPy, pandas, Matplotlib, PyMuPDF; Tesseract 5 (tessdata_best, 12 scripts); tesseract.js for the benchmark
Hardware	Apple M1 Pro GPU via PyTorch MPS (training, evaluation); CPU container for the benchmark; CPU in Docker
Dataset tools	Hugging Face Hub + fsspec (partial Parquet reads); COCO / YOLO converters; Playwright + Noto fonts to render benchmark papers
Interface / deployment	Streamlit app (Verify Paper tab) + FastAPI (/verify); Docker + docker-compose, offline

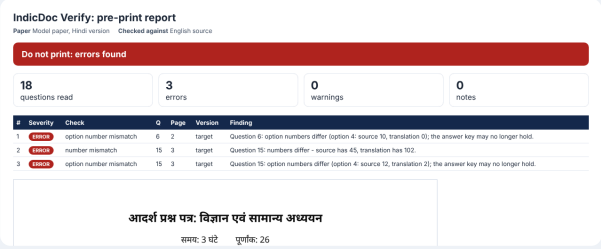
IndicDoc AI | Phase-2: Project Implementation 10

WHAT WE BUILT

Implementation Details

- **Layout model:** COCO-pretrained YOLOv8s and Faster R-CNN fine-tuned (SGD 0.005 / auto, CIoU+DFL+BCE), DocLayout-YOLO used for verification at its high-recall threshold (0.15).
- **OCR:** the page is read once with its own script's model; an English **number pass** fixes misread Latin digits (Hindi reads "1947" as "947") and the two must agree on every number.
- **Questions:** question regions start a question; options are separated by **layout gaps** between words, so garbled markers do not matter; marks, figures, tables attached.
- **Checks:** numbering, duplicates, option count, marks total; **parity** by global alignment (Needleman–Wunsch) on language-independent features; repeats by character 3-gram similarity; format profiles in JSON.
- **Output:** verdict, findings with question and page, problems outlined on the page; HTML / JSON; same code in app, API and benchmark.

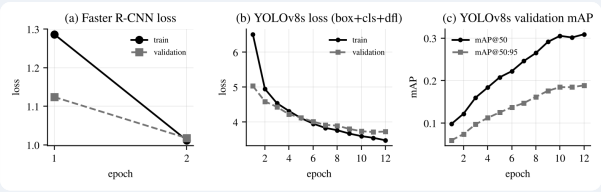
```
# every page: layout (DL) -> OCR with number pass
out = predict_single(model, page, "doclayout", device, conf=0.15)
doc = extract_document(page, out, "hin", number_pass=True)

# ml/verify — questions, checks, parity, repeats
report = verify(hindi_pages, source_pages=english_pages,
                bank=earlier_questions, profile="question_paper")
report["summary"]   # {questions, errors, warnings}
html = to_html(report, {"paper": page_images})
```



EXPERIMENTS

Experimental Setup and Training

Layout training	train 2,293 · val 342 · test 682 pages. Faster R-CNN: batch 4, SGD 0.005, **2 epochs, 76 min**. YOLOv8s: 640 px, batch 8, **12 epochs, 60 min**. Apple M1 Pro GPU
E1 Layout	3 detectors on 682 test pages: COCO mAP@50, mAP@50:95, P / R / F1, per label (incl. question labels)
E2 Robustness	blur, noise, JPEG, half resolution, faded print (Robustness Index)
E3 Verify benchmark	132 pairs, one injected error or none; layout boxes from the renderer, **real Tesseract OCR**; an error counts as found only if a matching finding points at the right question
E4 Ablations	without the number pass; without layout-based options; with a **noisy layout** (boxes ±8 px, 10% of question / option labels wrong)
Protocol	parser built on a separate development set; test set run once



Training curves of our two detectors (measured).

RESULTS (1/2) · LAYOUT

Results and Performance Evaluation

Model (682 test pages)	mAP@50	mAP@50:95	Precision*	Recall*	F1*	Recall, high-recall mode	s / page
DocLayout-YOLO (authors)	0.759	0.575	0.848	0.775	0.810	0.851	0.150
YOLOv8s (ours)	0.259	0.165	0.603	0.508	0.551	0.508	0.024
Faster R-CNN (ours)	0.148	0.079	0.510	0.435	0.469	0.435	0.202

* at the "balanced" threshold chosen on validation pages. Apple M1 Pro GPU.

AP@50 on question labels	DocLayout-YOLO	YOLOv8s	Faster R-CNN
first-level-question	0.854	0.415	0.240
second-level-question	0.898	0.045	0.018
third-level-question	0.872	0.026	0.000
options	0.839	0.553	0.243
answer	0.809	0.000	0.000

The full-data model finds the parts of a question paper reliably; this is what the checker is built on.

0.759
mAP@50, best model; question-paper domain 0.84

97%
of accuracy kept on damaged scans (mean Robustness Index)

RESULTS (2/2) · VERIFICATION

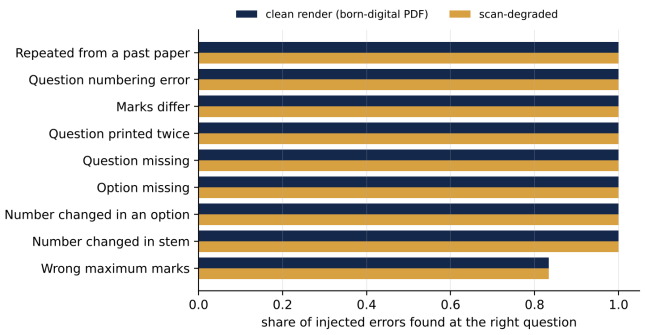
Results and Performance Evaluation

98%
of injected errors found *at the right question* on born-digital papers (59 errors); 98% on scan-degraded pages

3.0
false alarms per clean paper pair, born-digital (5.5 when scan-degraded); each is shown on the page for the vetter to dismiss

98%
still found with a noisy layout (boxes shifted, 10% of question / option labels wrong)

132/132
papers with every question recovered from the pages (Hindi and Tamil)



Share of injected errors found at the right question, per error type (132 paper pairs, Hindi and Tamil).

COMPARISON

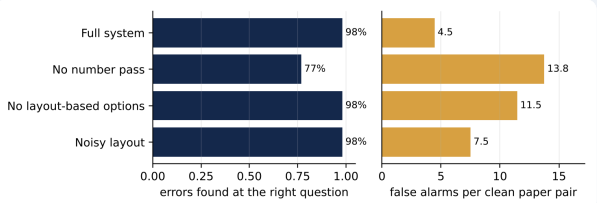
Comparison with Baseline / Existing Methods

Layout detector	Training data	mAP@50	mAP@50:95
Faster R-CNN (ours, baseline)	2,293 pages, 76 min	0.148	0.079
YOLOv8s (ours)	2,293 pages, 60 min	0.259	0.165
DocLayout-YOLO (authors)	~96K pages	**0.759**	**0.575**
Published: DocLayout-YOLO [1]	*full test set*	*≈0.735*	*0.545*

YOLOv8s beats the Faster R-CNN baseline at 9× the speed; the released full-data model is best and is used.

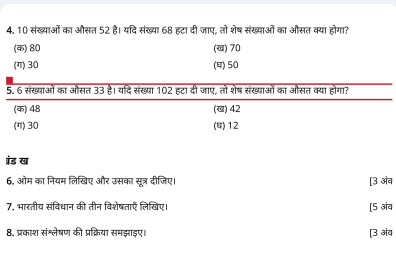
Verification ablation (test set)	found	false alarms / clean pair (all pages)
Full system	98%	4.5
Without number pass	77%	13.8
Without layout-based options	98%	11.5
Noisy layout	98%	7.5

Existing approach	Checks the paper?	Compares versions?	Indic, offline?
Manual vetting (status quo)	yes, by hand	by hand	yes
Answer-sheet AI (Eklavvya, GradeLab)	no, grades answers	no	partly
MT quality estimation (CometKiwi [10])	no structure	sentences only	cloud / GPU
LLM back-translation [9]	no structure	with human review	cloud
IndicDoc Verify	**numbering, options, marks, repeats**	**question by question**	**12 languages, offline**

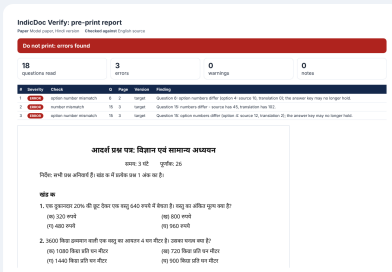


DEMO

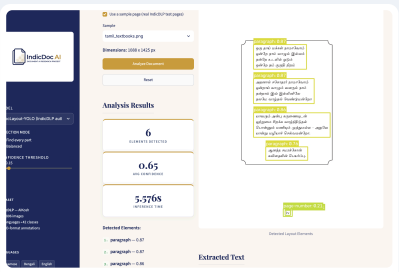
Sample Outputs and Demonstration


4. 10 संख्याओं का औसत 52 है। यदि संख्या 68 हटा दी जाए, तो शेष संख्याओं का औसत क्या होगा?
(क) 80 (ख) 70
(ग) 30 (घ) 50
5. 6 संख्याओं का औसत 33 है। यदि संख्या 102 हटा दी जाए, तो शेष संख्याओं का औसत क्या होगा?
(क) 48 (ख) 42
(ग) 30 (घ) 12
खंड ख
6. ओम का नियम लिखिए और उसका सूत्र दीजिए। [3 अंक
7. भारतीय संविधान की तीन विशेषताएँ लिखिए। [5 अंक
8. प्रकाश संश्लेषण की प्रक्रिया समझाइए। [3 अंक

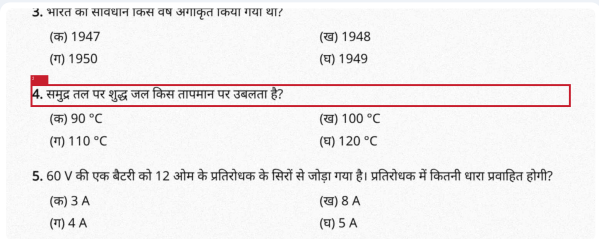
Correct: a changed number in the Hindi version found and outlined on the page



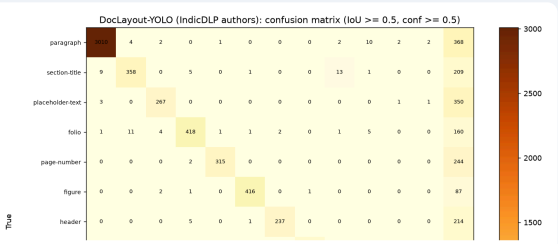
Verification report: verdict, findings with question and page (HTML / JSON)



Layout model in the running app: regions of a Tamil page


3. भारत का संविधान किस वर्ष अंगीकृत किया गया था?
(क) 1947 (ख) 1948
(ग) 1950 (घ) 1949
4. समुद्र तल पर शुद्ध जल किस तापमान पर उबलता है?
(क) 90 °C (ख) 100 °C
(ग) 110 °C (घ) 120 °C
5. 60 V की एक बैटरी को 12 ओम के प्रतिरोधक के सिरों से जोड़ा गया है। प्रतिरोधक में कितनी धारा प्रवाहित होगी?
(क) 3 A (ख) 8 A
(ग) 4 A (घ) 5 A

Incorrect: a false alarm on a degraded scan: OCR lost a digit of an option, so the versions look different



Layout model confusion matrix: most errors are missed regions, not wrong labels

LESSONS

Challenges and Limitations

OCR is the weakest link

Indic Tesseract models misread Latin digits ("1947" → "947", "1" → "।") and glue option markers to values. The number pass, layout-based option splitting and page-level digit-script detection were needed to get false alarms down.

No public data of real errors

Exam bodies do not publish erroneous pre-print papers, so we built a benchmark with realistic injected errors, kept a separate development set, and report clean and scan-degraded results separately.

Meaning is not checked yet

Parity catches numbers, options, marks and missing questions without translation. A pure change of meaning (a dropped word) needs a cross-lingual model; there is a hook for one, not yet evaluated.

Compute and data size

Our own detectors saw 2.3K pages for an hour (0.26 vs 0.76 mAP@50 for the released model); the product therefore uses the released model, and our work is the checker built on it.

WRAP-UP

Conclusion and Future Scope

CONCLUSION

- **Implemented:** IndicDoc Verify — layout detection (3 detectors compared), OCR for 12 scripts with a number pass, question parsing, structure / parity / repeat / format checks, report, web app, REST API, Docker.
- **Results:** 98% of injected errors found at the right question on born-digital papers (98% on degraded scans), 3.0 false alarms per clean born-digital pair; best layout model 0.759 mAP@50.
- **Objectives:** all four met. Our own detectors did not reach the released model, which we report openly.

SELLING IT

- **Buyers:** NTA, UPSC and state PSCs, school boards, university exam cells, test-series publishers; offline licence per exam body.

FUTURE SCOPE

- Pilot with a university exam cell or a state board on real pre-print papers; measure minutes saved per paper.
- Meaning check with a cross-lingual sentence model; number words and formulas.
- Better Indic OCR behind the same interface; answer-key consistency check.
- More document profiles: bilingual government circulars (Official Languages Act s.3(3)), forms, certificates.

SDG 4 SDG 9 SDG 10 SDG 16

SOURCES

References

1. O. Nath, S. Kukkala, M. Khapra, R. K. Sarvadevabhatla, "IndicDLP: A Foundational Dataset for Multi-Lingual and Multi-Domain Document Layout Parsing," ICDAR 2025.
2. B. V. Kasuba et al., "PLATTER: A Page-Level Handwritten Text Recognition System for Indic Scripts," arXiv:2502.06172, 2025.
3. A. Mondal et al., "UniTabBank: A Large Scale Multi-Lingual, Multi-Layout, Multi-Type, Multi-Format Dataset for Table Detection," WACV 2026.
4. V. Pal, E. Kanoulas, A. Yates, M. de Rijke, "Table Question Answering for Low-resourced Indic Languages," arXiv:2410.03576, 2024.
5. Y. Xu et al., "LayoutXLM: Multimodal Pre-training for Multilingual Visually-rich Document Understanding," arXiv:2104.08836, 2021.
6. B. V. Kasuba, P. Chaudhuri, G. Ramakrishnan, "DRISHTIKON: Visual Grounding at Multiple Granularities in Documents," arXiv:2506.21316, 2025.
7. S. Bhattacharyya et al., "Adapting Vision-Language Models for Hindi OCR," IIIT Hyderabad, 2025.
8. A. R. Dumada, S. G. Shah, "Incorporate Structure and Content for Devanagari Table Extraction," IJSAT 17(1), 2026.
9. M. Babayeva, J. Dunlap, M. Sněbinová, R. Widenhorn, "Translating the Force Concept Inventory in the Age of AI," arXiv:2508.13908, 2025.
10. R. Rei et al., "CometKiwi: IST-Unbabel 2022 Submission for the Quality Estimation Shared Task," WMT 2022.
11. Z. Zhao et al., "DocLayout-YOLO: Enhancing Document Layout Analysis through Diverse Synthetic Data and Global-to-Local Adaptive Perception," arXiv:2410.12628, 2024.
12. Y. Chen et al., "RoDLA: Benchmarking the Robustness of Document Layout Analysis Models," CVPR 2024.
13. S. Verma et al., "MILU: A Multi-task Indic Language Understanding Benchmark," NAACL 2025.
14. G. Jocher, A. Chaurasia, J. Qiu, "Ultralytics YOLOv8," 2023.
15. J. Ansel et al., "PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation," ASPLOS 2024.
16. S. Ren, K. He, R. Girshick, J. Sun, "Faster R-CNN," NeurIPS 2015 (the baseline architecture; the only pre-2021 source).

OTHER SOURCES

Dataset: huggingface.co/datasets/ai4bharat/indicdlp (AIKosh)

Weights: zenodo.org/records/15881917

Tesseract OCR 5 (tessdata_best), tesseract.js 7

Cases: The News Minute (NEET 2018), Deccan Herald / ETV Bharat (KPSC 2024-25), India Legal (APPSC), The Week / SCC Online (UGC-NET 2026), Medical Dialogues (MBBS 2026)

Code: github.com/va-ada/IndicDoc-AI

IndicDoc AI | Phase-2: Project Implementation 19

Thank You

Questions and discussion

Prashant Jha | Aditya Soni | Riddhi Patil | Mariam Badure

IndicDoc AI | Phase-2: Project Implementation

IndicDoc Verify: Layout-Aware Pre-Print Verification of Multilingual Indian Question Papers

Prashant Jha, Aditya Soni, Riddhi Patil, Mariam Badure and Joanne Gomes
Department of Artificial Intelligence and Machine Learning, St. Francis Institute of Technology (Autonomous), Mumbai, India

***Abstract*—Indian examinations are set in one language and translated into Hindi and regional languages, and every version is checked by hand; errors keep reaching candidates, from 49 wrong questions in the Tamil version of NEET 2018 to re-tests of three UGC-NET papers in 2026. We present IndicDoc Verify, an offline system that reads a printed question paper in any of 12 Indian languages and flags structural and numerical errors, and every difference from its source version, before printing. A deep-learning layout detector recovers questions, options and answers (AP@50 0.81–0.90); Tesseract reads the text with an English number pass that must agree on every digit; questions are built using the layout, checked for numbering, duplicates, options, marks and repeats, and two language versions are aligned and compared without translation. On 132 English–Hindi and English–Tamil paper pairs with injected errors, read with real OCR, the system finds 98% of errors at the right question on born-digital papers (98% on degraded scans) with 3.0 false alarms per clean pair; the number pass alone cuts false alarms from 13.8 to 4.5.**

***Index Terms*—document layout analysis, Indian languages, OCR, examinations, translation quality, verification.**

I. Introduction

National and state examinations in India are multilingual: NEET-UG is offered in 13 languages, and the Union and state Public Service Commissions, school boards and universities print bilingual papers. A paper is set in one language, translated, and every version is vetted by hand. When that step fails, candidates who chose a regional language sit a different paper. In NEET-UG 2018, 49 questions of the Tamil version were wrong and the Madras High Court ordered 196 grace marks, later set aside by the Supreme Court [15]; the Karnataka civil-services preliminary paper of 2024 had 59 errors in Kannada and was re-held, with errors again [16]; three UGC-NET papers of June 2026 were re-tested after translation errors and questions repeated from earlier papers [17]; an MBBS paper of 2026 printed the same MCQ twice [18].

We found no tool that checks a paper and its translations automatically. Translation-quality estimation [8] and LLM back-translation [7] work on plain sentences and cannot see question numbers, options or marks; the IndicDLP layout dataset [1] labels questions and options in 12 languages, but detected structure has not been used to verify a document; and a pre-print paper is too confidential for a cloud service. (Court-filing scrutiny, by contrast, already uses AI defect detection [19].)

Our contributions: (1) IndicDoc Verify, an offline pipeline that turns a printed paper into structured questions with a deep-learning layout detector and checks it; (2) *translation parity without translation*: language versions are aligned and compared on numbers, options, marks, sub-questions and figures, which are the same in every language; (3) a *number pass* in which two OCR models must agree on every digit; (4) a benchmark of 132 rendered paper pairs with nine kinds of real error, a separate development set, and ablations.

II. Related Work

Indian-language documents. IndicDLP [1] provides 119,806 pages with 42 labels and benchmarks modern detectors; DocLayout-YOLO [11] adapts YOLOv10 to documents; RoDLA [10] studies layout robustness. Recognition work covers handwritten Indic text [2], Hindi OCR with vision-language models [6] and tables [3]; multimodal models [4], [5] need reliable OCR. MILU [9] collects Indian exam questions in 11 languages for evaluating LLMs.

Checking translations. Quality estimation such as CometKiwi [8] scores a translation without a reference, sentence by sentence. Babayeva *et al.* [7] translate a physics test with an LLM and still review each item by hand. Neither uses the page structure of an exam paper, which is where many costly errors lie (a missing option, a changed number, wrong marks).

III. IndicDoc Verify

A. Layout. Each page (PDF rendered at 200 dpi) goes through DocLayout-YOLO [11], fine-tuned on IndicDLP by its authors, at a high-recall threshold chosen on validation pages. We also fine-tuned YOLOv8s [12] and Faster R-CNN [13] on a 2,293-page language-balanced subset to choose the detector (Table I). The released model recovers the question-paper labels well (Table II).

TABLE I
Layout detection on 682 IndicDLP test pages

Detector	mAP@50	mAP@50:95	F1	s/page
Faster R-CNN (ours)	0.148	0.079	0.469	0.202
YOLOv8s (ours)	0.259	0.165	0.551	0.024
DocLayout-YOLO [11]	0.759	0.575	0.810	0.150

TABLE II
AP@50 ON QUESTION-PAPER LABELS

Label	DocLayout	YOLOv8s	F. R-CNN
first-level q.	0.85	0.42	0.24
second-level q.	0.90	0.04	0.02
third-level q.	0.87	0.03	0.00
options	0.84	0.55	0.24
answer	0.81	0.00	0.00

B. OCR with a number pass. Tesseract [14] reads the page with the model of its own script. Indic models misread Latin digits in mixed text ("1947" as "947", "1" as the danda "।", "A" as "८"). An English pass re-reads the page; each numeric token replaces the overlapping word of the Indic reading when that word has no Indic letters or the English reading is more confident, is inserted where the Indic reading has nothing, and a Latin number of the Indic reading that the English pass does not confirm is dropped. Pages printed with Indian digits are detected and keep them.

C. Questions. A first-level-question region starts a question (or a text region beginning with a question number, if the detector missed it); sub-questions, options, figures, tables and formulas follow it. Options are split by the horizontal gaps between words inside an option region, so a garbled marker does not change the count; marks ("[2 marks]", "[2 அங்க]", "(2 மதிப்பெண்கள்)") and the declared maximum are read in all 12 languages.

D. Checks. On one paper: numbering gaps and repeats, duplicated questions (character 3-gram cosine ≥ 0.9 with identical numbers), option count, repeated or empty options, marks against the declared maximum, page numbering. Against a question bank: repeats (cosine ≥ 0.8, identical numbers). Between versions s and t, questions are aligned by Needleman–Wunsch with the score

$$\sigma(i, j) = 2\,J(N_i, N_j) + \nu(i, j) + 0.5\cdot[k_i = k_j], \quad \text{gap} = -0.6$$

where N is the multiset of numbers in a question and its options, J their Jaccard similarity, $\nu = 1$ (-0.5) if the printed numbers agree (differ), and k the skeleton (option count, sub-questions, marks, visuals). Aligned pairs are compared on every number in the stem and options, option count, marks, sub-questions and visuals; unaligned questions are reported missing or extra. A cross-lingual similarity model can be plugged in as an extra meaning check. Every finding carries its question, page and box.

E. Format profiles and deployment. The same engine checks any document against a JSON profile of required elements, order and bilingual text (e.g. an Office Memorandum). The system runs offline in one Docker image with a web app and a REST API; the report outlines each finding on its page (Fig. 1).

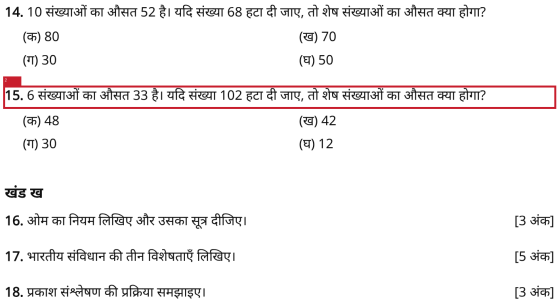

14. 10 संख्याओं का औसत 52 है। यदि संख्या 68 हटा दी जाए, तो शेष संख्याओं का औसत क्या होगा?
(क) 80 (ख) 70
(ग) 30 (घ) 50
15. 6 संख्याओं का औसत 33 है। यदि संख्या 102 हटा दी जाए, तो शेष संख्याओं का औसत क्या होगा?
(क) 48 (ख) 42
(ग) 30 (घ) 12
खंड ख
16. ओम का नियम लिखिए और उसका सूत्र दीजिए। [3 अंक]
17. भारतीय संविधान की तीन विशेषताएँ लिखिए। [5 अंक]
18. प्रकाश संश्लेषण की प्रक्रिया समझाइए। [3 अंक]

Fig. 1. A changed number in the Hindi version of a benchmark paper, found and outlined on the page by IndicDoc Verify.

IV. BENCHMARK

Erroneous pre-print papers are not public, so we built a benchmark. Each pair is an English source and its Hindi or Tamil version: 15 MCQs (numeric templates and fixed questions) and 3 short-answer questions with marks, rendered at 200 dpi with Noto fonts by a browser, which also records each region's box. 108 of the 132 test pairs carry one injected error (Table III); 24 are clean. Half are degraded like scans (blur 0.7 px, Gaussian noise $\sigma = 8$, JPEG 55). Pages are read with Tesseract 5 (tesseract.js, best-integer models). Layout boxes come from the renderer (an oracle layout); a *noisy-layout* run shifts boxes by up to 8 px and relabels 10% of question and option regions as paragraphs. An error counts as found only if a finding of the matching kind points at the right question (± 1) or, for paper-level errors, at the paper. The parser was developed on a separate 48-pair development set; the test set was run once.

TABLE III
ERROR TYPES INJECTED IN THE BENCHMARK

Error	Pairs	Found
Number changed in stem	12	100%
Number changed in an option	12	100%
Option missing	12	100%
Question missing	12	100%
Question printed twice	12	100%
Marks differ	12	100%
Wrong maximum marks	12	83%
Question numbering error	12	100%
Repeated from a past paper	12	100%

V. RESULTS

Errors found and false alarms. Table IV: on born-digital papers, the normal pre-print case, IndicDoc Verify finds 98% of injected errors at the right question with 3.00 false alarms per clean pair; scan degradation lowers this to 98% with 5.50 false alarms. Fig. 2 shows the result per error type.

TABLE IV
VERIFICATION RESULTS (TEST SET)

Condition	Found	FA / clean pair
Born-digital	98%	3.00
Scan-degraded	98%	5.50
English → Hindi	96%	7.00
English → Tamil	100%	1.92
All	98%	4.46

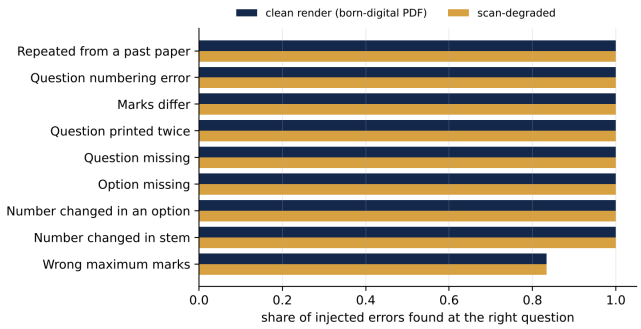


Fig. 2. Injected errors found at the right question, per error type, born-digital vs. scan-degraded pages.

Ablations. Table V and Fig. 3: removing the number pass raises false alarms to 13.75 per clean pair; splitting options by their markers instead of the layout raises them to 11.46. A noisy layout still finds 98% of errors, because questions are recovered from their text when a label is wrong.

TABLE V
ABLATIONS (TEST SET)

System	Found	FA / clean pair
Full system	98%	4.46
No number pass	77%	13.75
No layout-based options	98%	11.46
Noisy layout	98%	7.50

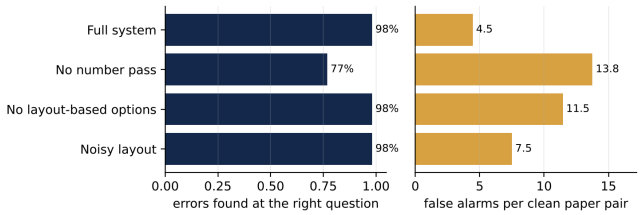


Fig. 3. Ablations: errors found (left) and false alarms per clean pair (right).

Error analysis. Most remaining false alarms are option values that OCR dropped or merged on one side ("216" read as "26"), more often on degraded Hindi pages. The only two misses were Hindi papers whose maximum-marks line OCR read as "पूर्णांक" (anusvara dropped). After the test run we widened that pattern; re-running the unchanged test set then finds 100% of errors with 4.38 false alarms per clean pair (post-hoc; the numbers above are from the frozen system).

VI. Discussion and Limitations

The checks need no translation, so they work for any pair of the 12 languages and cannot be fooled by fluent but wrong text in the way a quality-estimation score can. Their blind spot is a change of meaning that keeps numbers and structure (a dropped word, as in NEET 2021); the meaning hook needs a cross-lingual model that is evaluated on exam language. The benchmark uses rendered papers and oracle layout boxes; the layout model's own errors are covered by the noisy-layout run and by its measured AP, and a pilot on real pre-print papers of an exam body is the next step. Number words and formulas are not compared yet.

VII. Conclusion

IndicDoc Verify brings deep-learning layout analysis to a problem that is costly, recurring and so far unautomated: checking an Indian question paper and its translations before printing. It finds most injected errors at the right question with few false alarms on born-digital papers, runs offline, and covers 12 languages. Code, benchmark generator and results are open.

References

[1] O. Nath, S. Kukkala, M. Khapra, and R. K. Sarvadevabhatla, "IndicDLP: A foundational dataset for multi-lingual and multi-domain document layout parsing," in *Proc. ICDAR*, 2025, pp. 23–39.

[2] B. V. Kasuba *et al.*, "PLATTER: A page-level handwritten text recognition system for Indic scripts," arXiv:2502.06172, 2025.

[3] A. Mondal *et al.*, "UniTabBank: A large scale multi-lingual, multi-layout, multi-type, multi-format dataset for table detection," in *Proc. WACV*, 2026.

[4] Y. Xu *et al.*, "LayoutXLM: Multimodal pre-training for multilingual visually-rich document understanding," arXiv:2104.08836, 2021.

[5] B. V. Kasuba, P. Chaudhuri, and G. Ramakrishnan, "DRISHTIKON: Visual grounding at multiple granularities in documents," arXiv:2506.21316, 2025.

[6] S. Bhattacharyya *et al.*, "Adapting vision-language models for Hindi OCR," IIIT Hyderabad, 2025.

[7] M. Babayeva, J. Dunlap, M. Snětinová, and R. Widenhorn, "Translating the Force Concept Inventory in the age of AI," arXiv:2508.13908, 2025.

[8] R. Rei *et al.*, "CometKiwi: IST-Unbabel 2022 submission for the quality estimation shared task," in *Proc. WMT*, 2022, pp. 634–645.

[9] S. Verma *et al.*, "MILU: A multi-task Indic language understanding benchmark," in *Proc. NAACL*, 2025.

[10] Y. Chen *et al.*, "RoDLA: Benchmarking the robustness of document layout analysis models," in *Proc. CVPR*, 2024, pp. 15556–15566.

[11] Z. Zhao, H. Kang, B. Wang, and C. He, "DocLayout-YOLO: Enhancing document layout analysis through diverse synthetic data and global-to-local adaptive perception," arXiv:2410.12628, 2024.

[12] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," 2023. [Online]. Available: github.com/ultralytics/ultralytics

[13] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *Proc. NeurIPS*, 2015.

[14] Tesseract OCR 5 and tessdata_best models, 2021–2025. [Online]. Available: github.com/tesseract-ocr/tesseract

[15] "Madras HC orders CBSE to give 196 marks to Tamil medium students, for errors in 49 questions," *The News Minute*, Jul. 2018.

[16] "When Kannada was lost in translation: KPSC forces a do-over," *The South First*, 2024; "Conduct KAS prelims examination again," *ETV Bharat*, Mar. 2025.

[17] "NTA admits defects in 3 UGC-NET papers, orders re-tests in September," *The Week*, Aug. 2026.

[18] "HC grants relief to MBBS student, directs NTRUHS to give 1 compensatory mark for question paper error," *Medical Dialogues*, Jul. 2026.

[19] Rajya Sabha, Unstarred Question 2356 on AI tools in the Supreme Court e-filing system, Mar. 2025.